

Slide 6 - First Slide with code cell

Matplotlib: Your Fundamental Canvas

Matplotlib is the most powerful and widely used visualization library in Python. It gives you complete and **granular control** over every element of your plot. While it might seem a bit complex at first, understanding its core concepts opens the door to creating highly customized and professional-grade visualizations. 😊

Understanding Figure and Axes

In Matplotlib, there are two key concepts you need to understand:

1. Figure (The Overall Container) 🖼️

The Figure is essentially the entire "canvas" or "window" where your plots will be displayed.

You can think of it as a sheet of paper on which you're going to draw.

A Figure can contain one or more Axes .

2. Axes (The Plotting Area) 📊

The Axes is the actual plotting area(s) within the Figure. This is where your data is drawn as lines, bars, points, etc.

Each Axes has its own X and Y axes, and you can define separate titles, labels, and scales for it.

If you consider the Figure as a sheet of paper, the Axes is the actual "graph" or "plot" you draw on that paper.

```
In [2]: import matplotlib.pyplot as plt
import numpy as np

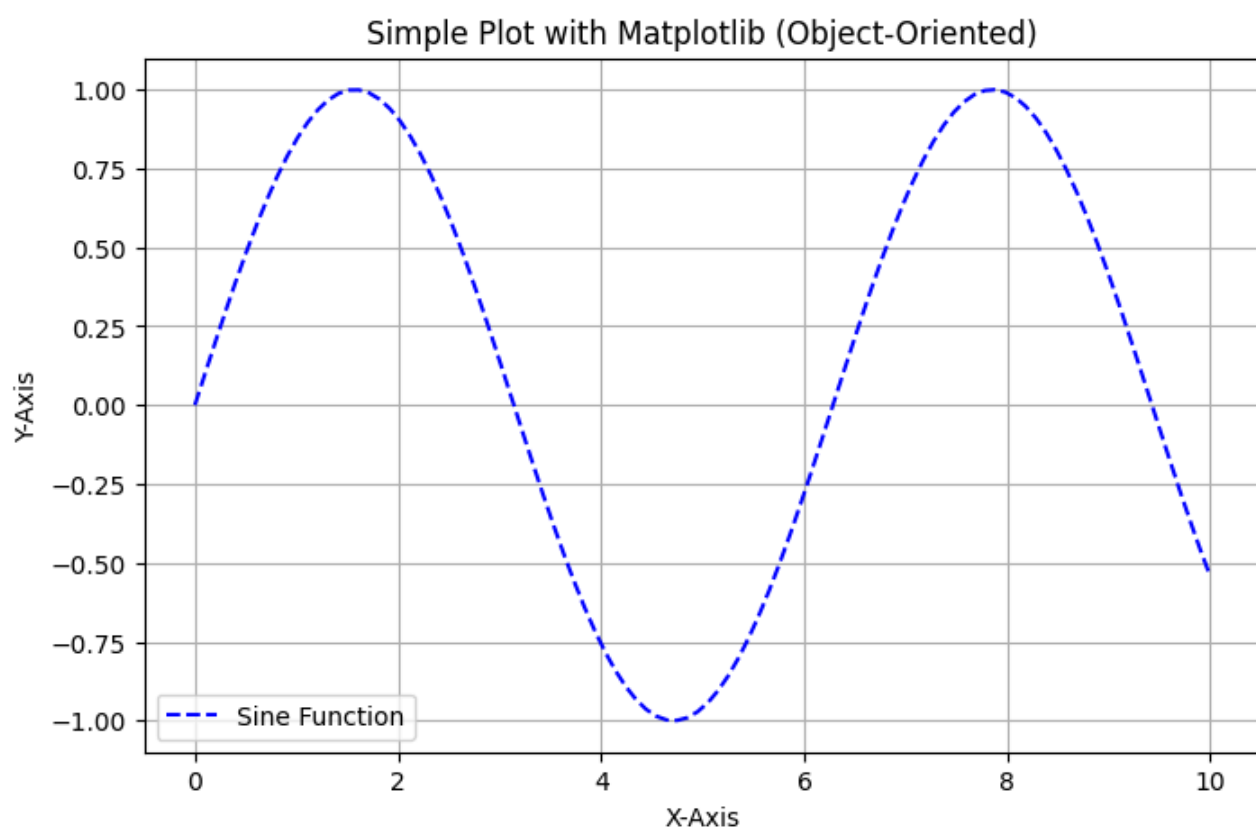
In [4]: # Create a Figure and a set of Axes
# plt.subplots() returns a Figure and one Axes (by default).

fig, ax = plt.subplots(figsize=(8, 5)) # figsize sets the dimensions of the Figure in inches.

# Plot data on the Axes
x = np.linspace(0, 10, 100) # 100 points between 0 and 10
y = np.sin(x) # Sine function for y

ax.plot(x, y, label='Sine Function', color='blue', linestyle='--') # Plot line on Axes
ax.set_title('Simple Plot with Matplotlib (Object-Oriented)') # Axes title
ax.set_xlabel('X-Axis') # X-axis label
ax.set_ylabel('Y-Axis') # Y-axis label
ax.legend() # Display the legend
ax.grid(True) # Show grid

# Display the Figure
plt.show()
```

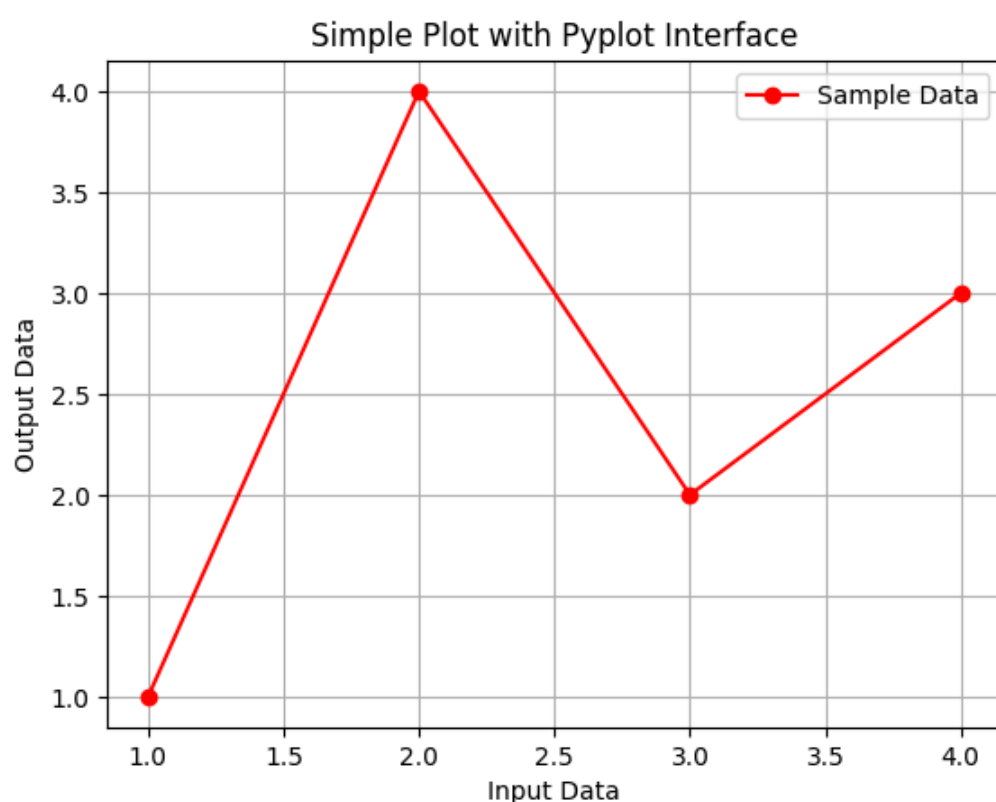


matplotlib.pyplot: The Quick Interface

`matplotlib.pyplot` is a module in Matplotlib that provides a collection of functions styled like MATLAB commands. This interface is very useful for quick starts and simple plots. Many `pyplot` functions **automatically** manage the Figure and Axes for you.

```
In [5]: # Plot using the pyplot interface

plt.plot([1, 2, 3, 4], [1, 4, 2, 3], label='Sample Data', color='red', marker='o')
plt.title('Simple Plot with Pyplot Interface')
plt.xlabel('Input Data')
plt.ylabel('Output Data')
plt.grid(True)
plt.legend()
plt.show()
```



Difference in Approaches:

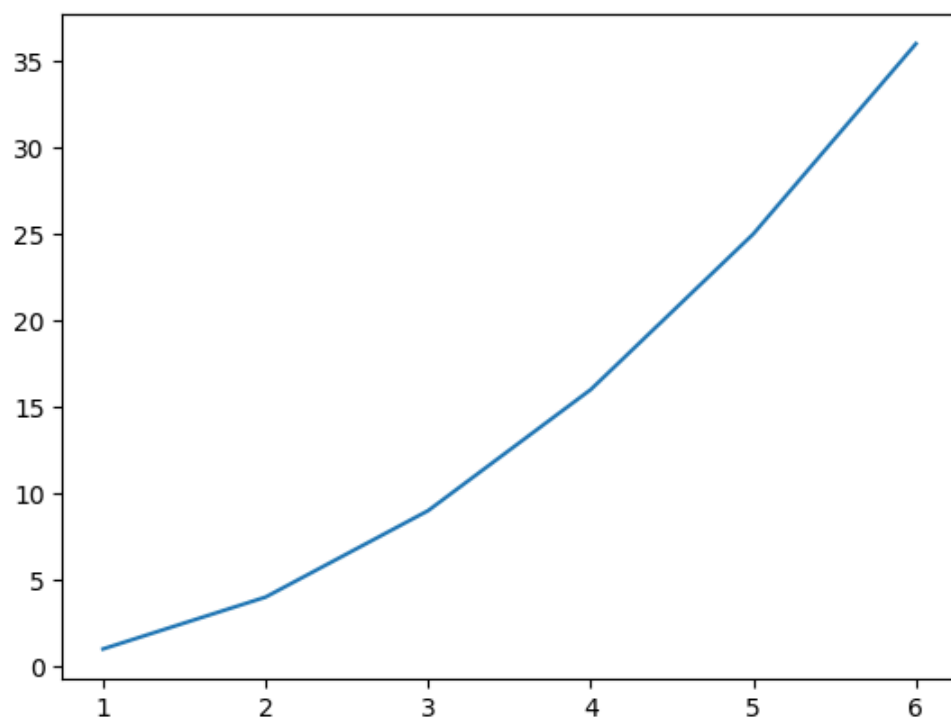
Object-Oriented Approach (`fig, ax = plt.subplots()`): Gives you more control and is recommended for complex plots or when you want to have multiple plots within a single Figure.

pyplot Approach: Suitable for quick and simple plots, and very practical for getting started.

Throughout this notebook, we will explore both approaches, but our emphasis will be on the object-oriented approach to help you create more professional and customized plots.

```
In [6]: import matplotlib.pyplot as plt
fig, ax = plt.subplots() # Create Axes and Figure
```

```
ax.plot([1, 2, 3, 4, 5, 6], [1, 4, 9, 16, 25, 36]) # Plot datas on Axes
plt.show()
```



```
In [7]: import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

np.random.seed(42)

x_scatter = np.random.rand(50) * 10
y_scatter = np.random.rand(50) * 10 + x_scatter
colors = np.random.rand(50)
sizes = np.random.rand(50) * 800 + 100

labels = ['A', 'B', 'C', 'D']
group1_data = np.random.randint(10, 80, 4)
group2_data = np.random.randint(10, 80, 4)
x = np.arange(len(labels))
width = 0.35

years = np.arange(2010, 2016)
data_comp1 = np.random.rand(6) * 100
data_comp2 = np.random.rand(6) * 100
data_comp3 = np.random.rand(6) * 100

matrix_data = np.random.rand(10, 10) * 100

fig, axes = plt.subplots(2, 2, figsize=(12, 9))

scatter = axes[0, 0].scatter(x_scatter, y_scatter, c=colors, s=sizes, cmap='viridis', alpha=0.7, edgecolors='w', linewidth=0.5)
axes[0, 0].set_title('Scatter Plot with Color & Size')
axes[0, 0].set_xlabel('Feature 1')
axes[0, 0].set_ylabel('Feature 2')
fig.colorbar(scatter, ax=axes[0, 0], label='Value')

axes[0, 1].bar(x - width/2, group1_data, width, label='Group 1', color='skyblue')
axes[0, 1].bar(x + width/2, group2_data, width, label='Group 2', color='lightcoral')
axes[0, 1].set_title('Grouped Bar Chart')
axes[0, 1].set_ylabel('Scores')
axes[0, 1].set_xticks(x)
axes[0, 1].set_xticklabels(labels)
axes[0, 1].legend()

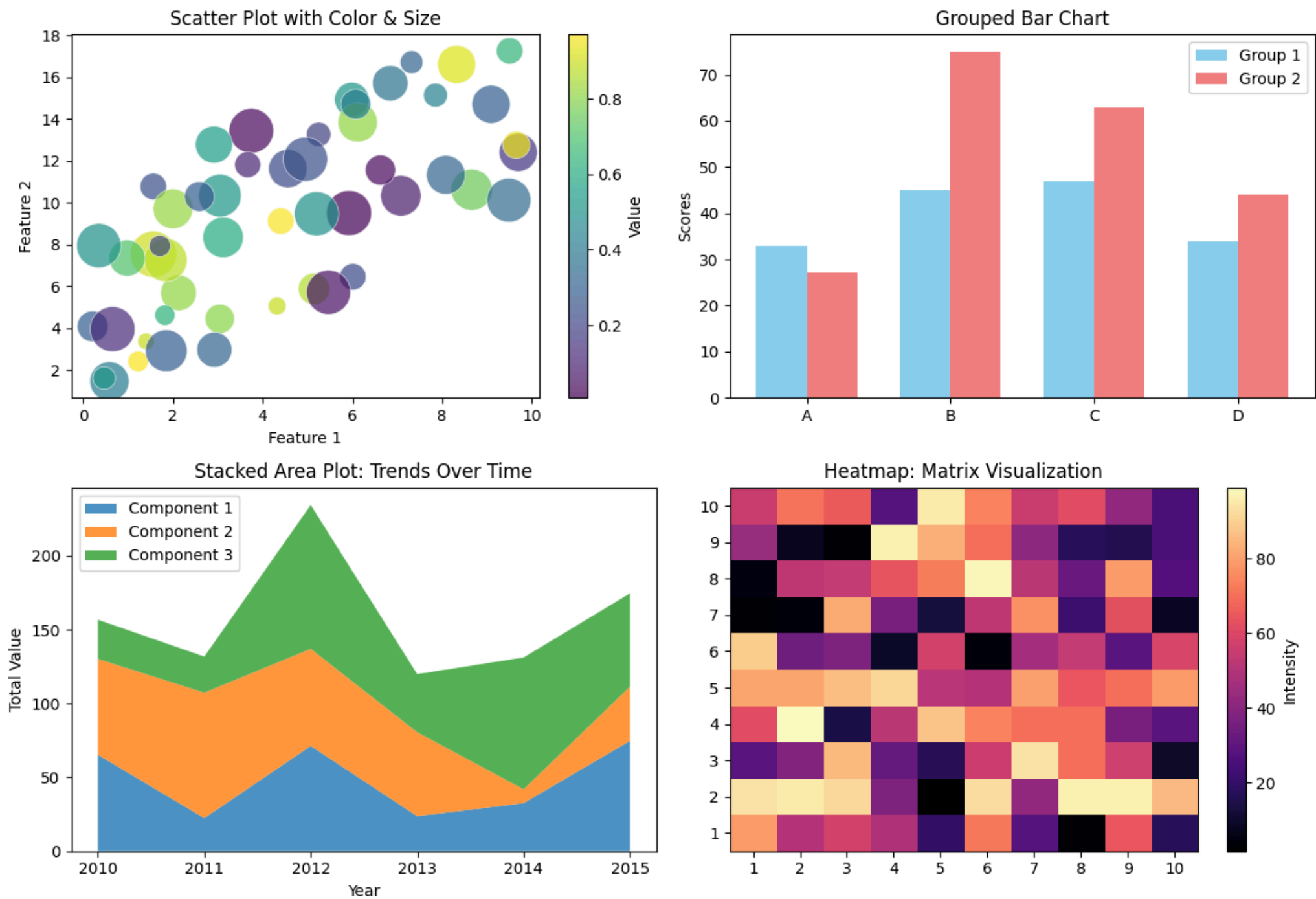
axes[1, 0].stackplot(years, data_comp1, data_comp2, data_comp3, labels=['Component 1', 'Component 2', 'Component 3'], alpha=0.8)
axes[1, 0].set_title('Stacked Area Plot: Trends Over Time')
axes[1, 0].set_xlabel('Year')
axes[1, 0].set_ylabel('Total Value')
axes[1, 0].legend(loc='upper left')

heatmap = axes[1, 1].imshow(matrix_data, cmap='magma', origin='lower', aspect='auto')
axes[1, 1].set_title('Heatmap: Matrix Visualization')
axes[1, 1].set_xticks(np.arange(matrix_data.shape[1]))
axes[1, 1].set_yticks(np.arange(matrix_data.shape[0]))
axes[1, 1].set_xticklabels(np.arange(1, matrix_data.shape[1] + 1))
axes[1, 1].set_yticklabels(np.arange(1, matrix_data.shape[0] + 1))
fig.colorbar(heatmap, ax=axes[1, 1], label='Intensity')

plt.tight_layout(rect=[0, 0.03, 1, 0.95])
fig.suptitle('Matplotlib\'s Multi-Panel Power: Diverse Visualizations', fontsize=18, fontweight='bold', color='darkblue')

plt.show()
```

Matplotlib's Multi-Panel Power: Diverse Visualizations



Slide 7

```
In [8]: import matplotlib.pyplot as plt
import numpy as np

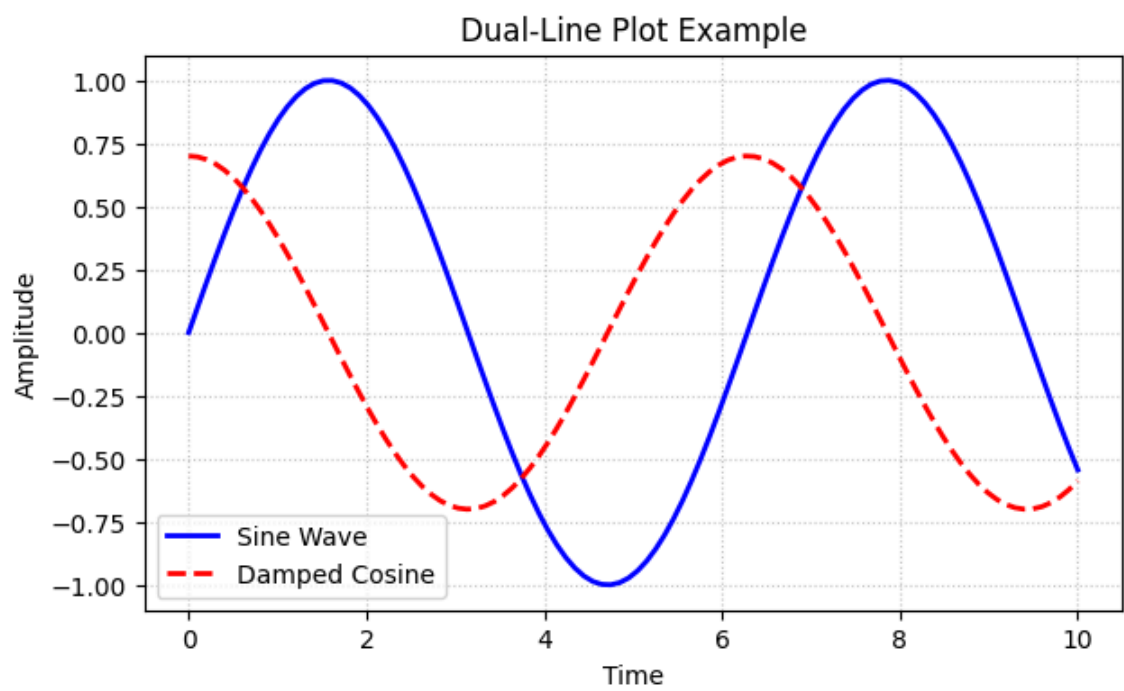
fig, ax = plt.subplots(figsize=(7, 4))

x = np.linspace(0, 10, 100)
y1 = np.sin(x)
y2 = np.cos(x) * 0.7

ax.plot(x, y1, label='Sine Wave', color='blue', linewidth=2)
ax.plot(x, y2, label='Damped Cosine', color='red', linestyle='--', linewidth=2)

ax.set_title('Dual-Line Plot Example')
ax.set_xlabel('Time')
ax.set_ylabel('Amplitude')
ax.legend()
ax.grid(True, linestyle=':', alpha=0.7)

plt.show()
```

```
In [9]: import matplotlib.pyplot as plt
import numpy as np

fig, axes = plt.subplots(1, 4, figsize=(16, 4))

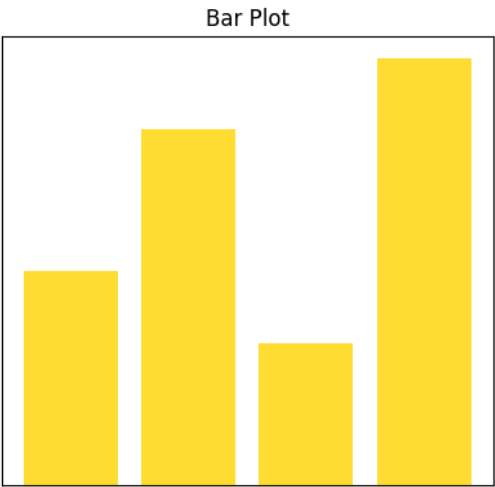
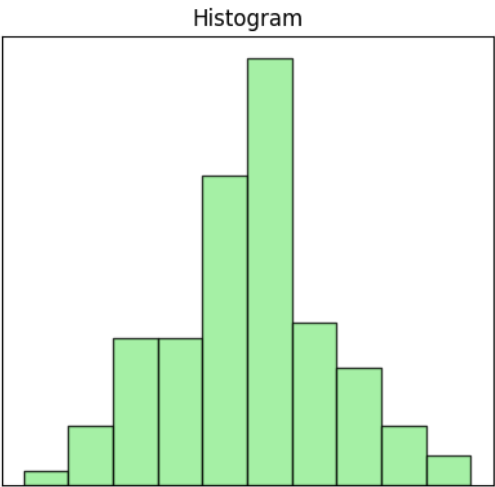
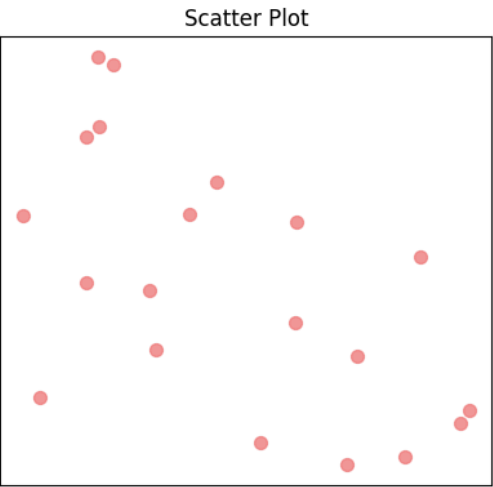
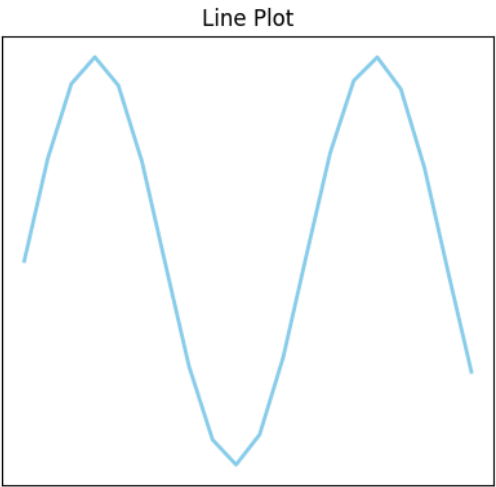
x_line = np.linspace(0, 10, 20)
y_line = np.sin(x_line)
axes[0].plot(x_line, y_line, color='skyblue', linewidth=2)
axes[0].set_title('Line Plot')
axes[0].set_xticks([])
axes[0].set_yticks([])

np.random.seed(42)
x_scatter = np.random.rand(20) * 10
y_scatter = np.random.rand(20) * 10
axes[1].scatter(x_scatter, y_scatter, color='lightcoral', s=50, alpha=0.8)
axes[1].set_title('Scatter Plot')
axes[1].set_xticks([])
axes[1].set_yticks([])

hist_data = np.random.normal(0, 1, 100)
axes[2].hist(hist_data, bins=10, color='lightgreen', edgecolor='black', alpha=0.8)
axes[2].set_title('Histogram')
axes[2].set_xticks([])
axes[2].set_yticks([])

categories = ['A', 'B', 'C', 'D']
values = [15, 25, 10, 30]
axes[3].bar(categories, values, color='gold', alpha=0.8)
axes[3].set_title('Bar Plot')
axes[3].set_xticks([])
axes[3].set_yticks([])

plt.tight_layout()
plt.show()
```



Matplotlib: Basic Plot Types 📊

In this section, we'll dive into creating the most fundamental and commonly used plot types in Matplotlib. Understanding these basic plots is crucial as they form the foundation for more complex visualizations. We'll use the object-oriented approach, focusing on the `Figure` and `Axes` objects for better control.

```
In [13]: import matplotlib.pyplot as plt
import numpy as np

# A good practice in Jupyter for plots to appear inline
%matplotlib inline
```

1. Line Plot 🌊

Line plots are ideal for visualizing trends over time or continuous relationships between two variables.

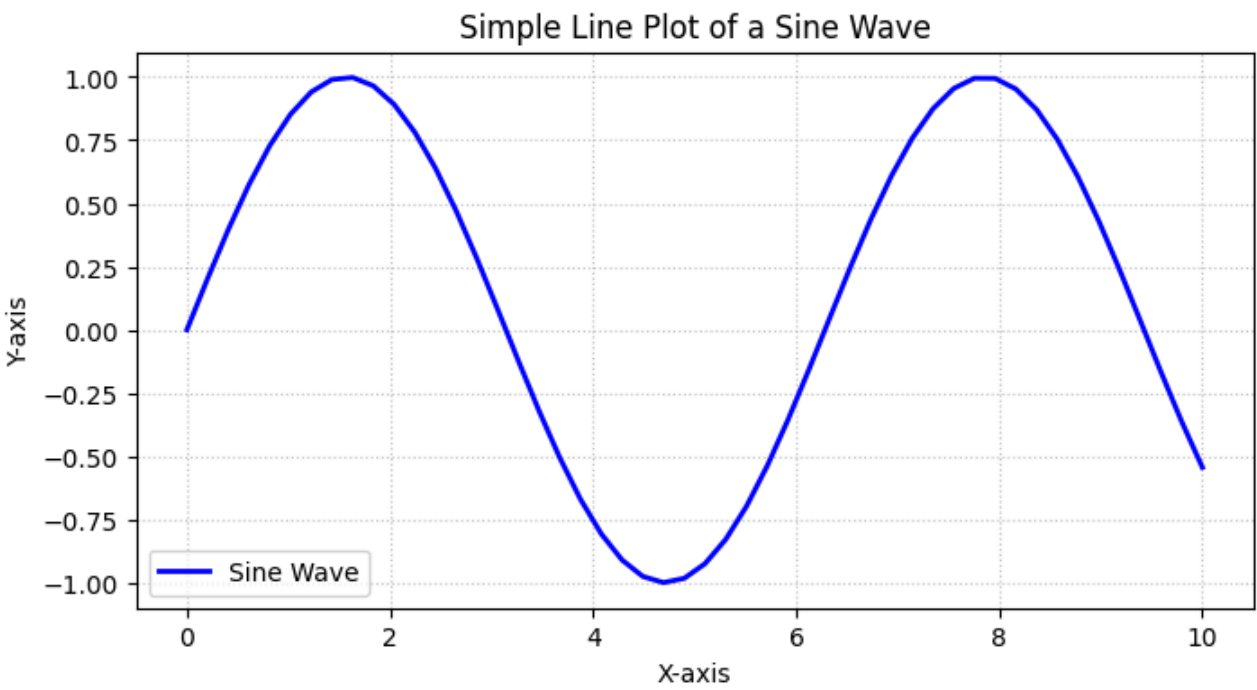
```
In [14]: # Data for Line Plot
x_line = np.linspace(0, 10, 50)
y_line = np.sin(x_line)

# Create a Figure and an Axes object
fig_line, ax_line = plt.subplots(figsize=(8, 4))

# Plot the data
ax_line.plot(x_line, y_line, color='blue', linestyle='-', linewidth=2, label='Sine Wave')

# Add titles and Labels
ax_line.set_title('Simple Line Plot of a Sine Wave')
ax_line.set_xlabel('X-axis')
ax_line.set_ylabel('Y-axis')
ax_line.legend()
ax_line.grid(True, linestyle=':', alpha=0.7)

plt.show()
```



2. Scatter Plot

Scatter plots are excellent for exploring the relationship or correlation between two numerical variables and identifying clusters or outliers.

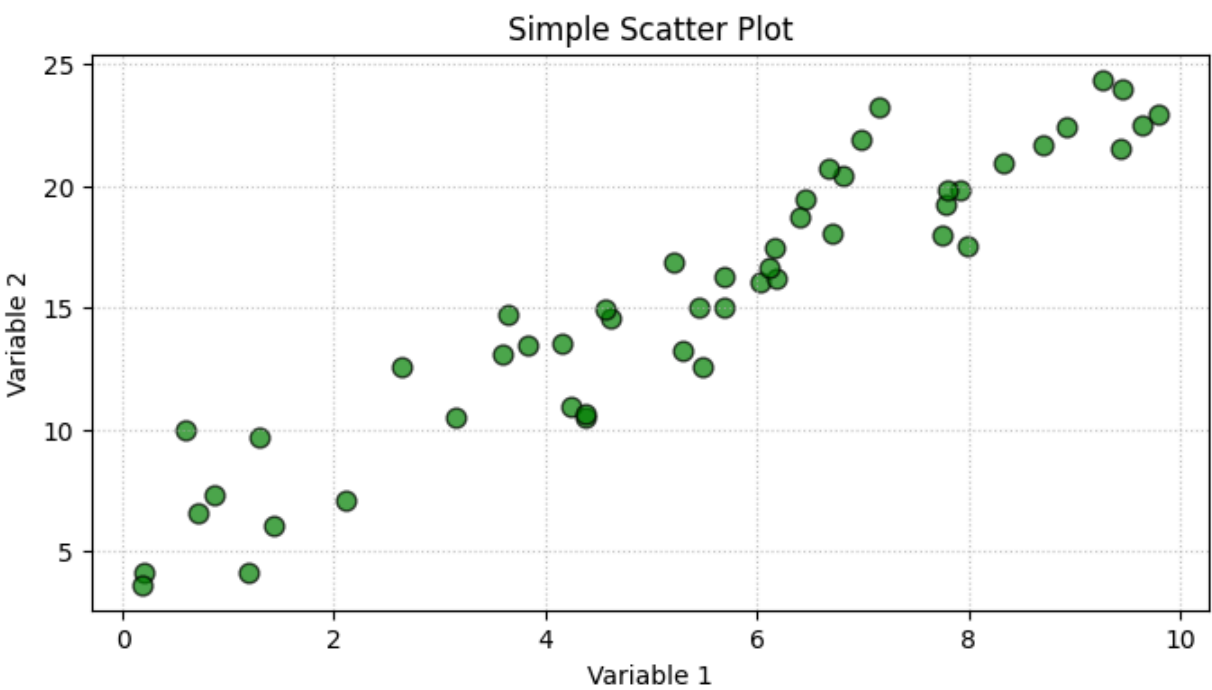
```
In [15]: # Data for Scatter Plot
np.random.seed(0) # For reproducibility
x_scatter = np.random.rand(50) * 10
y_scatter = 2 * x_scatter + np.random.randn(50) * 2 + 5

# Create a Figure and an Axes object
fig_scatter, ax_scatter = plt.subplots(figsize=(8, 4))

# Plot the data
ax_scatter.scatter(x_scatter, y_scatter, color='green', marker='o', s=60, alpha=0.7, edgecolors='black')

# Add titles and Labels
ax_scatter.set_title('Simple Scatter Plot')
ax_scatter.set_xlabel('Variable 1')
ax_scatter.set_ylabel('Variable 2')
ax_scatter.grid(True, linestyle=':', alpha=0.7)

plt.show()
```



3. Histogram

Histograms are used to visualize the **distribution** of a single numerical variable, showing the **frequency** of data points within different intervals (bins).

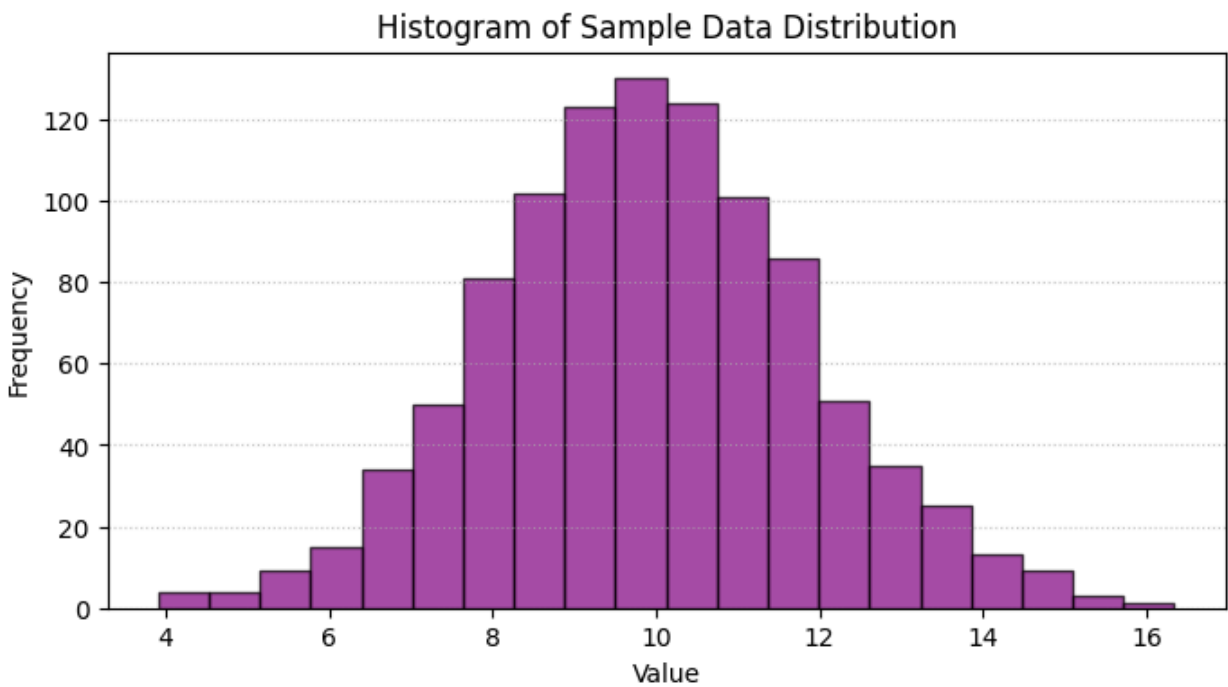
```
In [16]: # Data for Histogram
data_hist = np.random.normal(loc=10, scale=2, size=1000) # Mean 10, std dev 2, 1000 points

# Create a Figure and an Axes object
fig_hist, ax_hist = plt.subplots(figsize=(8, 4))

# Plot the histogram
ax_hist.hist(data_hist, bins=20, color='purple', edgecolor='black', alpha=0.7)

# Add titles and Labels
ax_hist.set_title('Histogram of Sample Data Distribution')
ax_hist.set_xlabel('Value')
ax_hist.set_ylabel('Frequency')
ax_hist.grid(axis='y', linestyle=':', alpha=0.7)

plt.show()
```



4. Bar Plot 📊

Bar plots are used to **compare** values across different categories or to show the **magnitude** of each category.

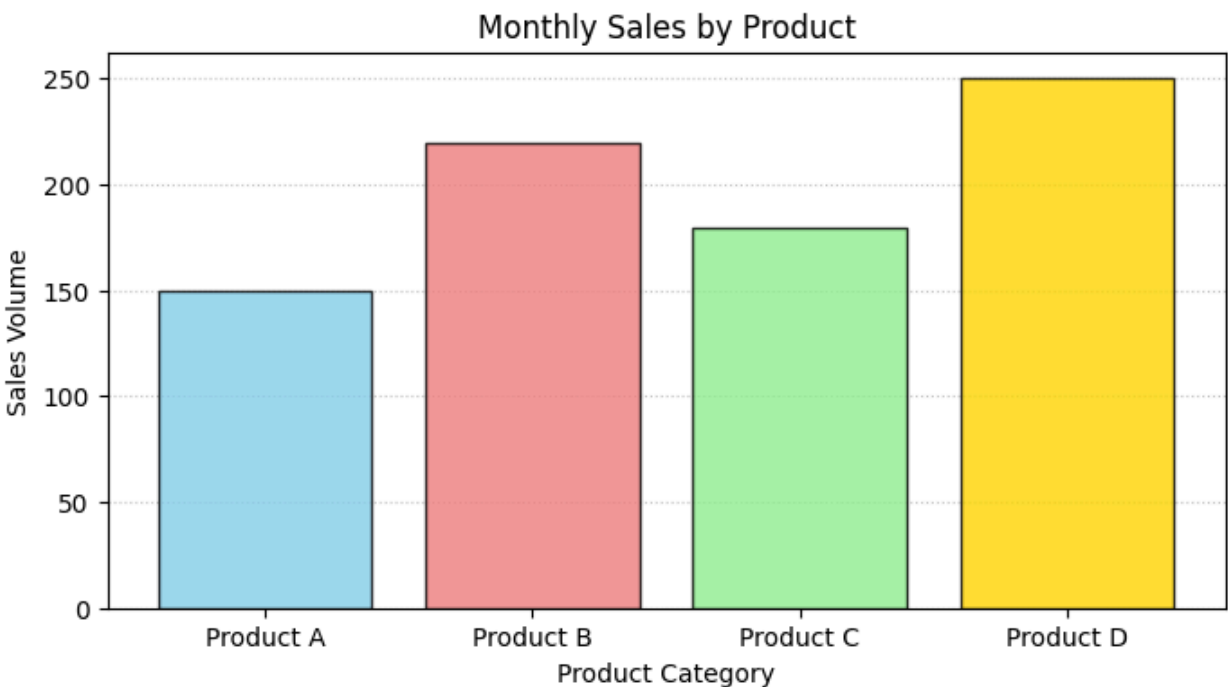
```
In [17]: # Data for Bar Plot
categories = ['Product A', 'Product B', 'Product C', 'Product D']
sales = [150, 220, 180, 250]

# Create a Figure and an Axes object
fig_bar, ax_bar = plt.subplots(figsize=(8, 4))

# Plot the bar chart
ax_bar.bar(categories, sales, color=['skyblue', 'lightcoral', 'lightgreen', 'gold'], edgecolor='black', alpha=0.8)

# Add titles and Labels
ax_bar.set_title('Monthly Sales by Product')
ax_bar.set_xlabel('Product Category')
ax_bar.set_ylabel('Sales Volume')
ax_bar.grid(axis='y', linestyle=':', alpha=0.7)
ax_bar.set_axisbelow(True) # Ensure grid lines are behind bars

plt.show()
```



Conclusion of Basic Plots

These four plot types are your **fundamental** building blocks in Matplotlib. By mastering them, you'll be able to **effectively** visualize a **wide** range of data. In the next section, we'll explore how to **customize** these plots to make them even more **informative** and **aesthetically pleasing**!

Slide 8

```
In [18]: import matplotlib.pyplot as plt
import numpy as np

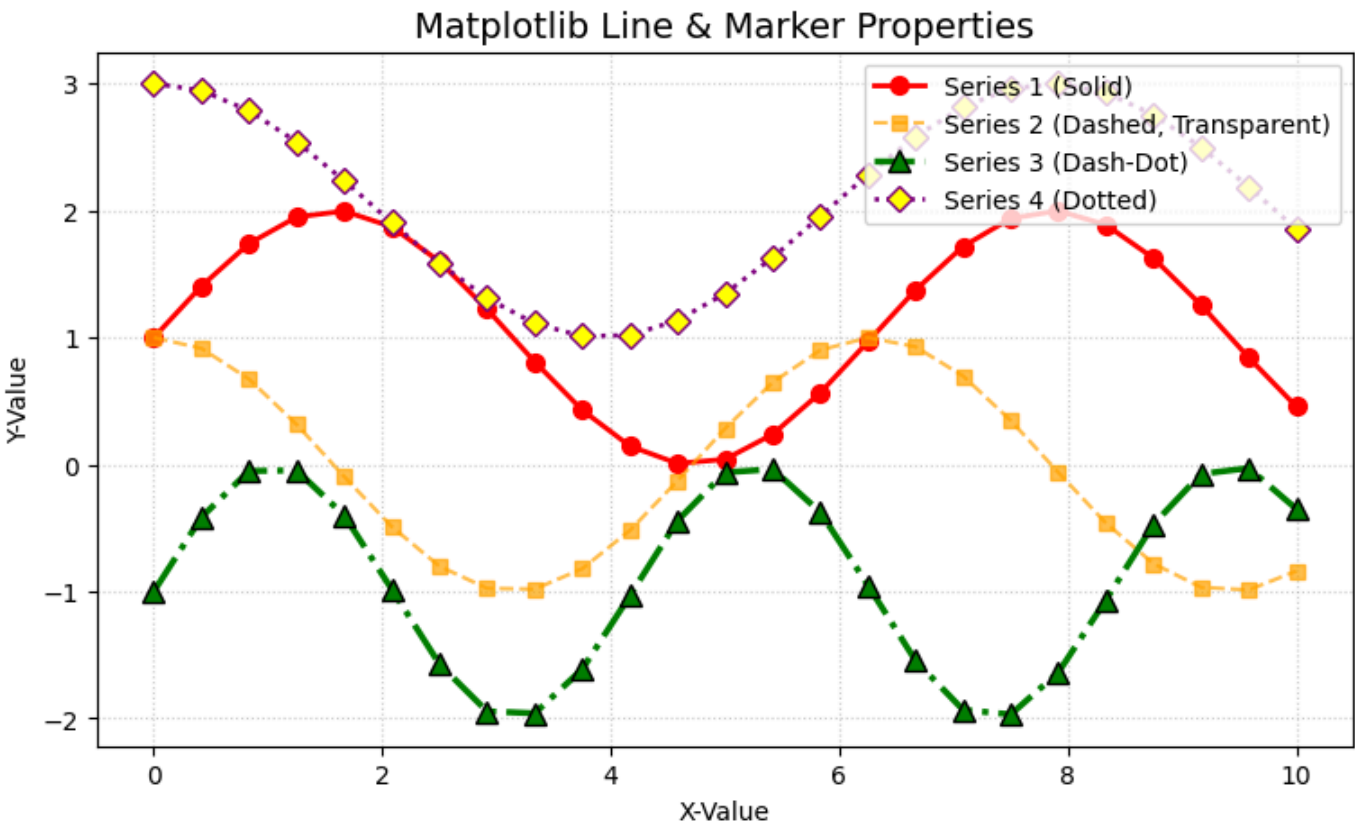
x_data = np.linspace(0, 10, 25)
y1 = np.sin(x_data) + 1
y2 = np.cos(x_data)
y3 = np.sin(x_data * 1.5) - 1
y4 = np.cos(x_data * 0.8) + 2

fig, ax = plt.subplots(figsize=(9, 5))

ax.plot(x_data, y1, color='red', linestyle='-', linewidth=2, marker='o', markersize=7, label='Series 1 (Solid)')
ax.plot(x_data, y2, color='orange', linestyle='--', linewidth=1.5, marker='s', markersize=6, alpha=0.7, label='Series 2 (Dashed, Transparent)')
ax.plot(x_data, y3, color='green', linestyle='-.', linewidth=2.5, marker='^', markersize=8, markeredgecolor='black', label='Series 3 (Dash-Dot)')
ax.plot(x_data, y4, color='purple', linestyle=':', linewidth=1.8, marker='D', markersize=7, markerfacecolor='yellow', label='Series 4 (Dotted)')

ax.set_title('Matplotlib Line & Marker Properties', fontsize=14)
ax.set_xlabel('X-Value')
ax.set_ylabel('Y-Value')
ax.legend(loc='upper right')
ax.grid(True, linestyle=':', alpha=0.6)

plt.show()
```



Matplotlib: Precision & Publication

This section explores how to exert fine-grained control over your Matplotlib plots, moving **beyond** basic aesthetics to create **highly customized** and **publication-quality figures**. We'll cover **line properties**, **advanced subplot arrangements**, and **saving plots for various uses**.

```
In [19]: import matplotlib.pyplot as plt
import numpy as np
%matplotlib inline
```

1. Line Properties: Customizing Your Visuals

Matplotlib offers extensive control over how lines and markers appear. This allows you to **differentiate** data series clearly and **emphasize specific points**.

```
In [20]: # Data for line properties demonstration
x_prop = np.linspace(0, 10, 20)
y_prop1 = np.sin(x_prop)
y_prop2 = np.cos(x_prop)
```

```
y_prop3 = np.sin(x_prop) * np.exp(-x_prop/5) # Damped sine wave

fig_props, ax_props = plt.subplots(figsize=(10, 6))

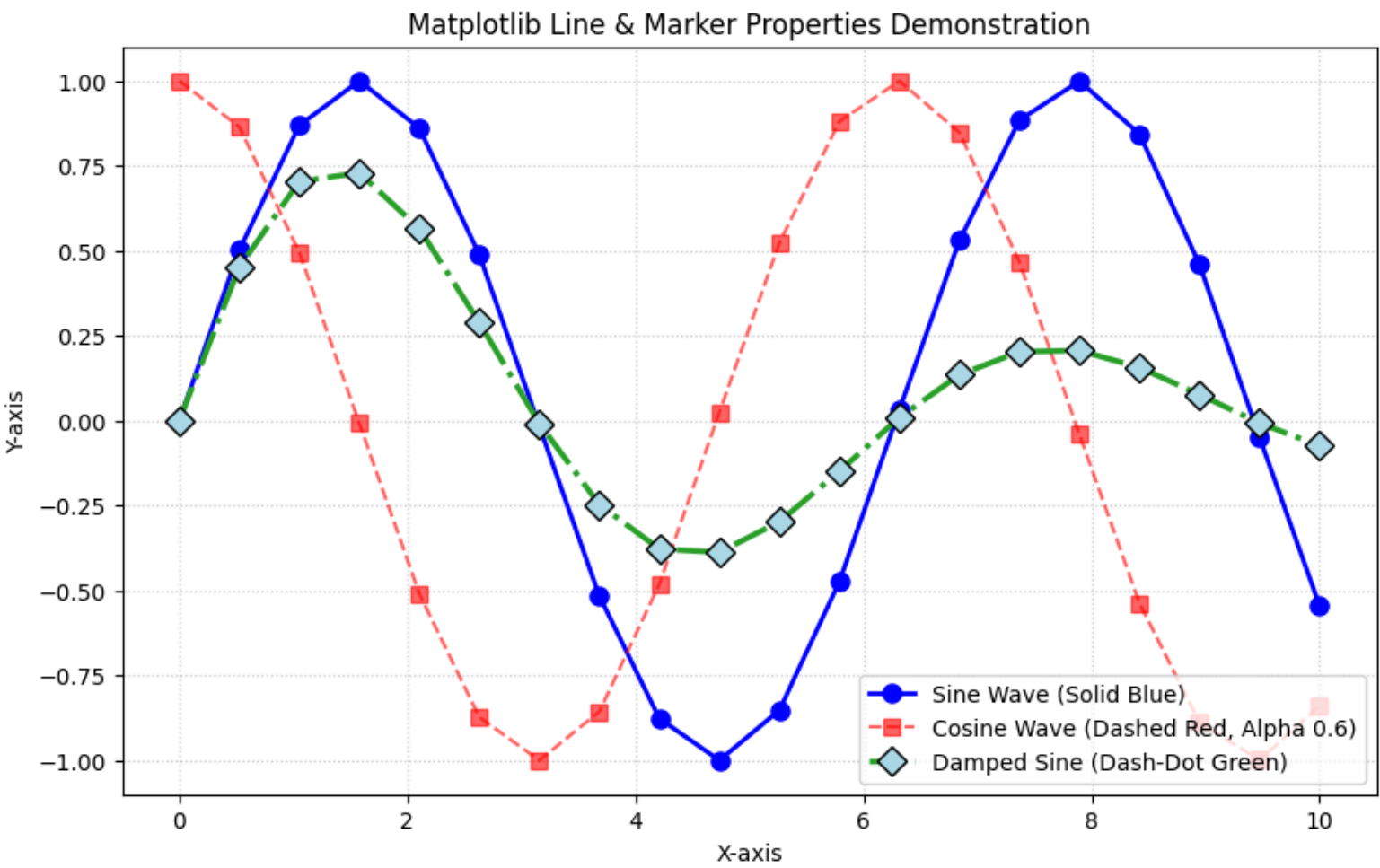
# Line 1: Basic solid blue line with circular markers
ax_props.plot(x_prop, y_prop1,
              color='blue',           # Line color
              linestyle='-',         # Solid line
              linewidth=2,           # Line thickness
              marker='o',            # Circle marker
              markersize=8,          # Marker size
              label='Sine Wave (Solid Blue)')

# Line 2: Dashed red line with square markers and transparency
ax_props.plot(x_prop, y_prop2,
              color='red',           # Line color
              linestyle='--',        # Dashed line
              linewidth=1.5,         # Line thickness
              marker='s',            # Square marker
              markersize=7,          # Marker size
              alpha=0.6,             # Transparency
              label='Cosine Wave (Dashed Red, Alpha 0.6)')

# Line 3: Dash-dot green line with diamond markers and larger size
ax_props.plot(x_prop, y_prop3,
              color='#2ca02c',       # Hex color code for green
              linestyle='-.',        # Dash-dot line
              linewidth=2.5,         # Line thickness
              marker='D',            # Diamond marker
              markersize=9,          # Marker size
              markeredgecolor='black', # Marker border color
              markerfacecolor='lightblue', # Marker fill color
              label='Damped Sine (Dash-Dot Green)')

ax_props.set_title('Matplotlib Line & Marker Properties Demonstration')
ax_props.set_xlabel('X-axis')
ax_props.set_ylabel('Y-axis')
ax_props.legend()
ax_props.grid(True, linestyle=':', alpha=0.6)

plt.show()
```



2.1 Subplots: Arranging Multiple Plots

When you need to display several related plots in a single figure, `subplots` is your go-to tool. It allows for clear comparisons and a comprehensive view of your data.

```
In [21]: # Data for subplots
x_sub = np.linspace(0, 5, 50)
y_sub1 = np.exp(x_sub)
y_sub2 = x_sub**2
y_sub3 = np.random.rand(50) * 10
y_sub4 = np.log(x_sub + 1)

# Create a 2x2 grid of subplots
fig_sub, axes_sub = plt.subplots(2, 2, figsize=(12, 10))

# Plot 1: Exponential Growth
axes_sub[0, 0].plot(x_sub, y_sub1, color='darkorange')
axes_sub[0, 0].set_title('Exponential Growth')
axes_sub[0, 0].set_xlabel('Time')
axes_sub[0, 0].set_ylabel('Value')
axes_sub[0, 0].grid(True, linestyle=':', alpha=0.6)
```



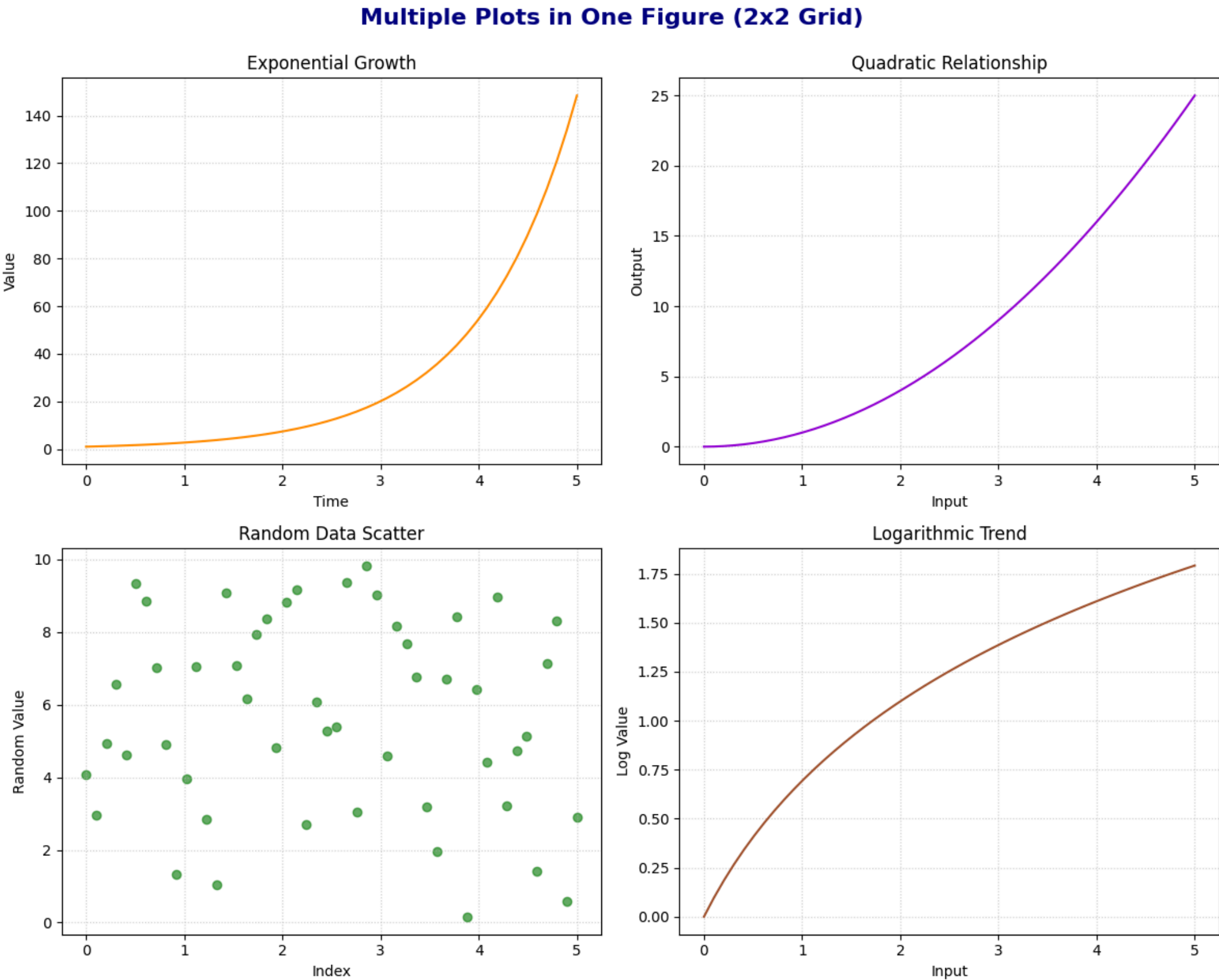
```
# Plot 2: Quadratic Relationship
axes_sub[0, 1].plot(x_sub, y_sub2, color='darkviolet')
axes_sub[0, 1].set_title('Quadratic Relationship')
axes_sub[0, 1].set_xlabel('Input')
axes_sub[0, 1].set_ylabel('Output')
axes_sub[0, 1].grid(True, linestyle=':', alpha=0.6)

# Plot 3: Random Data Distribution (Scatter)
axes_sub[1, 0].scatter(x_sub, y_sub3, color='forestgreen', alpha=0.7)
axes_sub[1, 0].set_title('Random Data Scatter')
axes_sub[1, 0].set_xlabel('Index')
axes_sub[1, 0].set_ylabel('Random Value')
axes_sub[1, 0].grid(True, linestyle=':', alpha=0.6)

# Plot 4: Logarithmic Trend
axes_sub[1, 1].plot(x_sub, y_sub4, color='sienna')
axes_sub[1, 1].set_title('Logarithmic Trend')
axes_sub[1, 1].set_xlabel('Input')
axes_sub[1, 1].set_ylabel('Log Value')
axes_sub[1, 1].grid(True, linestyle=':', alpha=0.6)

# Adjust layout to prevent overlapping
plt.tight_layout(rect=[0, 0.03, 1, 0.95])
fig_sub.suptitle('Multiple Plots in One Figure (2x2 Grid)', fontsize=16, fontweight='bold', color='navy')

plt.show()
```



2.2 Advanced Subplot Layouts with `fig.add_subplot()`

For more complex or non-uniform subplot arrangements, `fig.add_subplot()` offers granular control over subplot placement. It uses a grid specification (rows, columns, plot_number).

```
In [22]: # Example of a more complex subplot layout (e.g., one large plot and two smaller ones)
fig_custom = plt.figure(figsize=(12, 6))

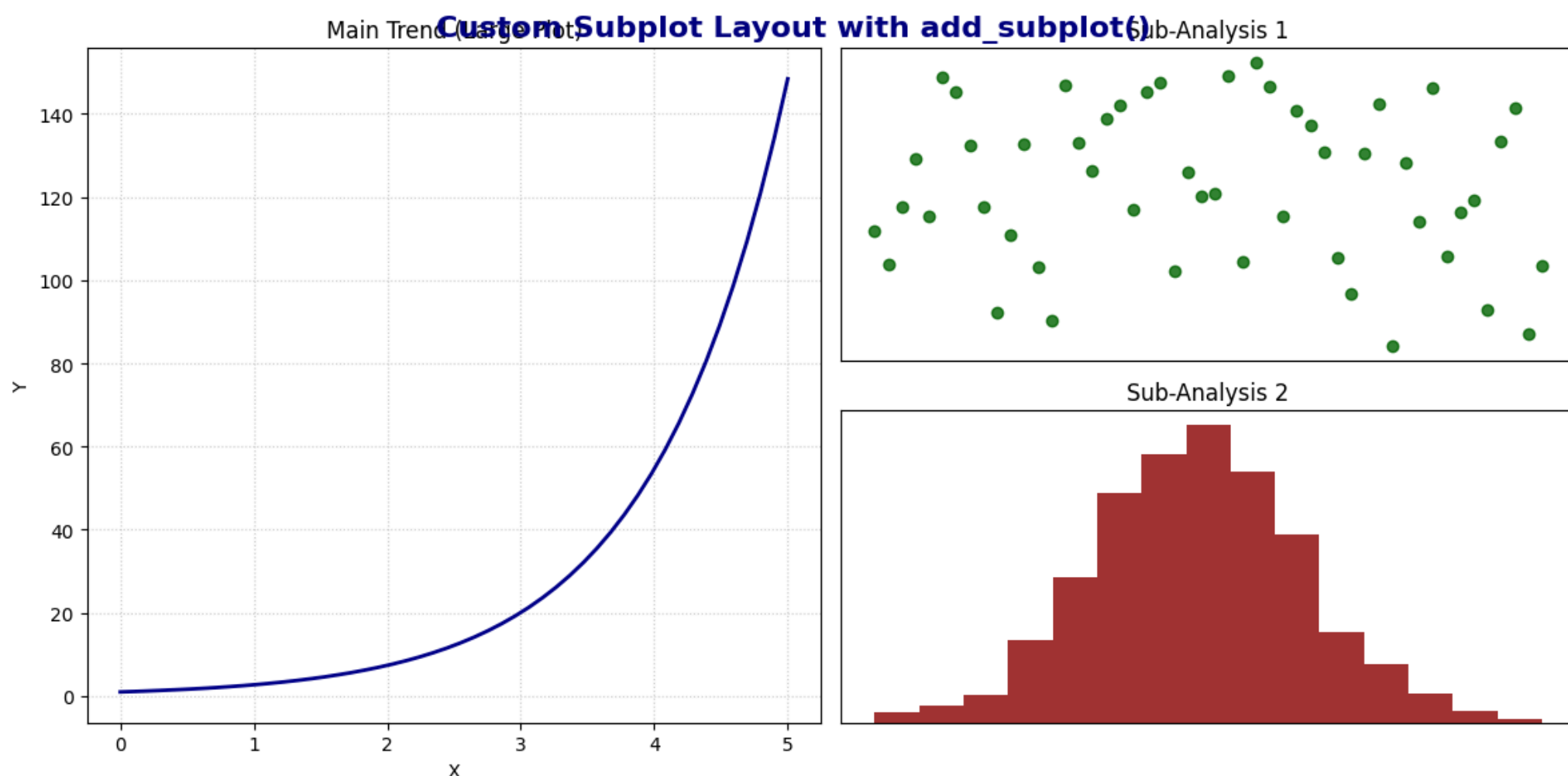
# Large plot on the Left (occupies 1st and 2nd rows, 1st column of a 2x2 conceptual grid)
ax1 = fig_custom.add_subplot(1, 2, 1) # 1 row, 2 columns, 1st plot
ax1.plot(x_sub, y_sub1, color='darkblue', linewidth=2)
ax1.set_title('Main Trend (Large Plot)')
ax1.set_xlabel('X')
ax1.set_ylabel('Y')
ax1.grid(True, linestyle=':', alpha=0.6)
```

```
# Small plots on the right (occupy 2nd column of a 2x2 conceptual grid, split vertically)
ax2 = fig_custom.add_subplot(2, 2, 2) # 2 rows, 2 columns, 2nd plot (top right)
ax2.scatter(x_sub, y_sub3, color='darkgreen', alpha=0.8)
ax2.set_title('Sub-Analysis 1')
ax2.set_xticks([]) # Remove ticks for cleaner look
ax2.set_yticks([])

ax3 = fig_custom.add_subplot(2, 2, 4) # 2 rows, 2 columns, 4th plot (bottom right)
ax3.hist(data_hist, bins=15, color='darkred', alpha=0.8)
ax3.set_title('Sub-Analysis 2')
ax3.set_xticks([])
ax3.set_yticks([])

plt.tight_layout()
fig_custom.suptitle('Custom Subplot Layout with add_subplot()', fontsize=16, fontweight='bold', color='navy')

plt.show()
```



3. Saving Plots for Publication

Once your plot is perfect, you'll often need to **save** it in a high-quality format for reports, papers, or presentations. `plt.savefig()` is your tool for this.

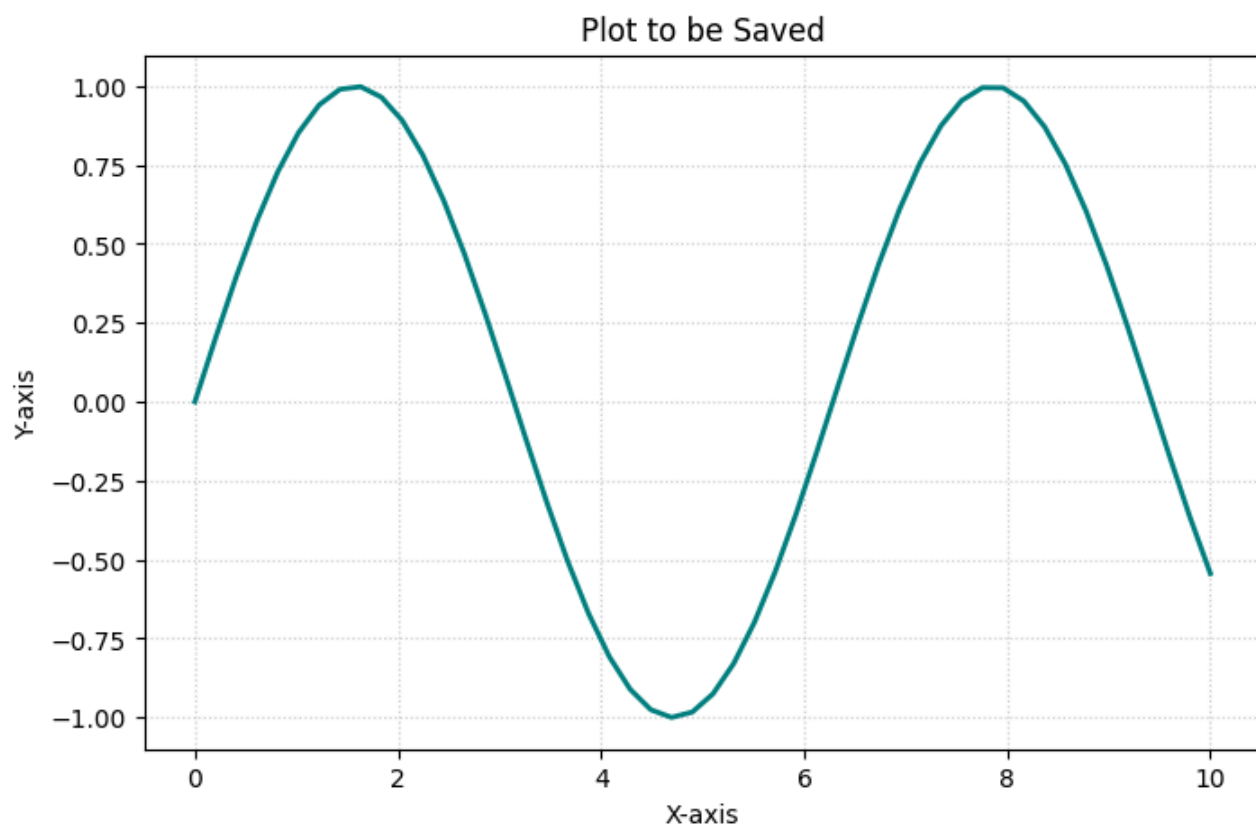
```
In [23]: # Create a simple plot to save

fig_save, ax_save = plt.subplots(figsize=(8, 5))
ax_save.plot(x_line, y_line, color='teal', linewidth=2)
ax_save.set_title('Plot to be Saved')
ax_save.set_xlabel('X-axis')
ax_save.set_ylabel('Y-axis')
ax_save.grid(True, linestyle=':', alpha=0.6)

# Save the figure in high resolution
# dpi (dots per inch) controls resolution. Use a high value for publication quality.
# bbox_inches='tight' ensures all elements (like labels) are included.

plt.savefig('my_publication_quality_plot.png', dpi=300, bbox_inches='tight')
plt.savefig('my_publication_quality_plot.pdf', bbox_inches='tight') # Also save as PDF for vector graphics

plt.show() # Display the plot in the notebook as well
```

You can now find `my_publication_quality_plot.png` and `my_publication_quality_plot.pdf` in **your current working directory**.

This concludes our dive into Matplotlib's fundamental and advanced controls. You now have the tools to create precise, informative, and visually stunning plots for any engineering or research task!

Slide 9

```
In [24]: import matplotlib.pyplot as plt
import numpy as np
import seaborn as sns

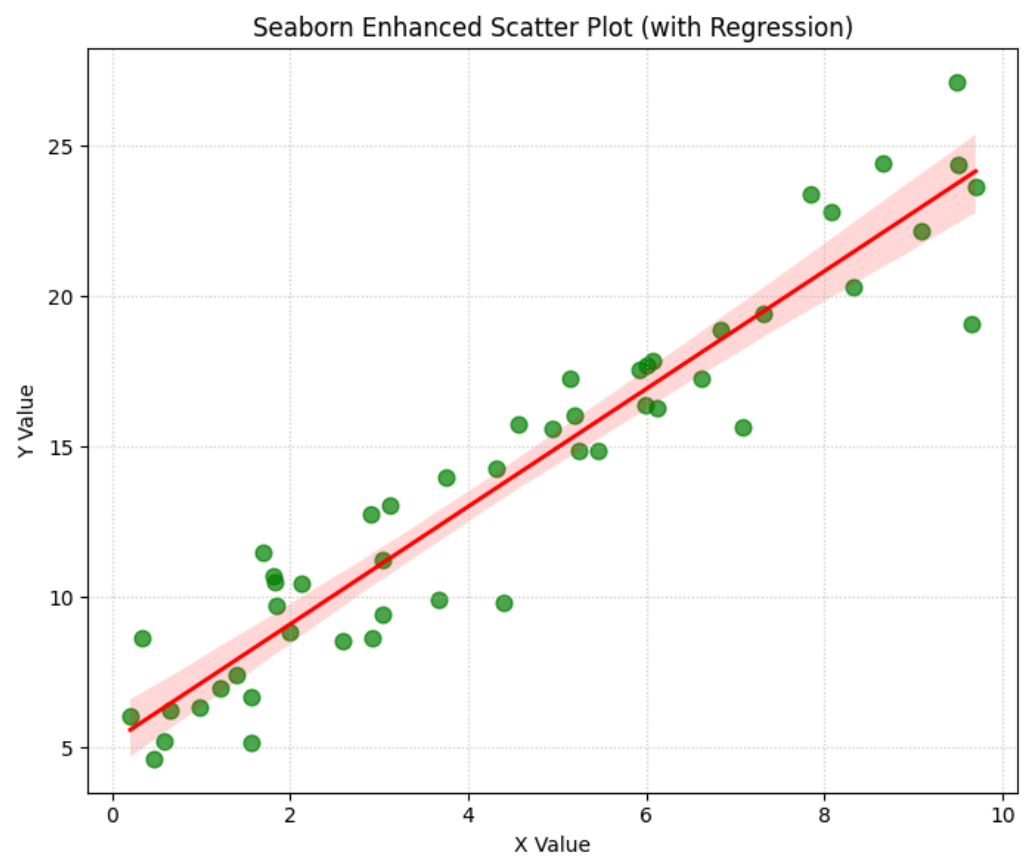
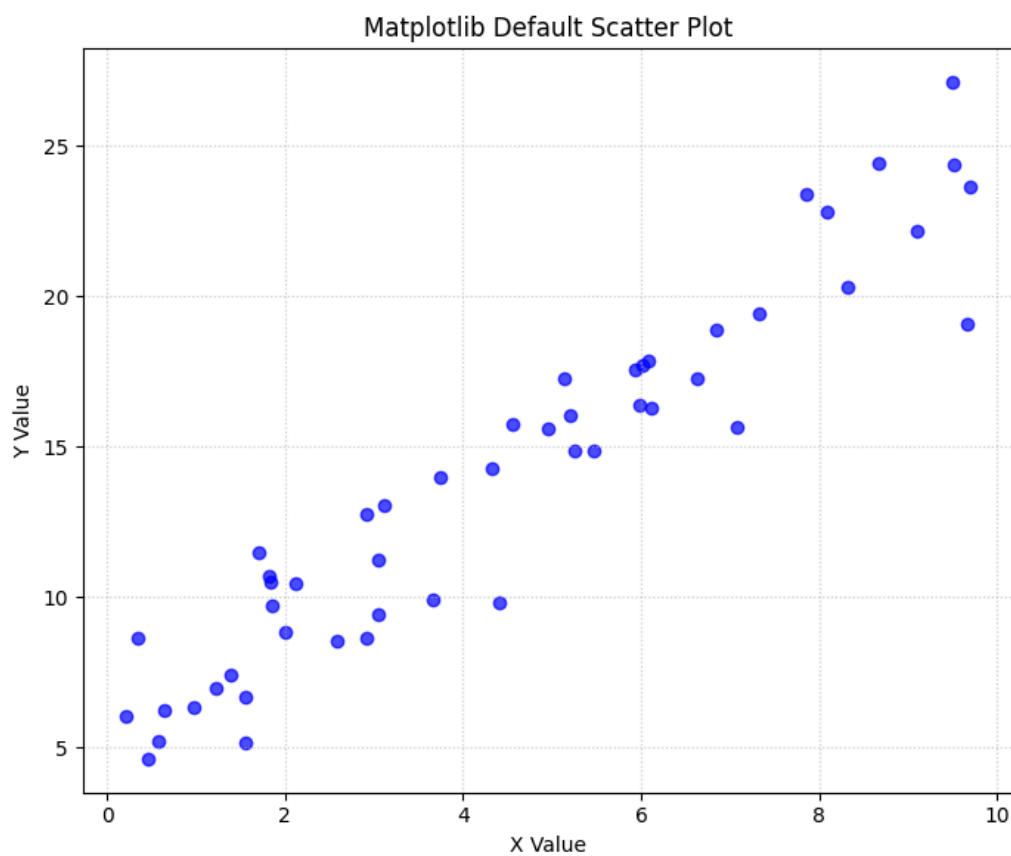
np.random.seed(42)
n_points = 50
x_data = np.random.rand(n_points) * 10
y_data = 2 * x_data + 5 + np.random.randn(n_points) * 2

fig, axes = plt.subplots(1, 2, figsize=(14, 6))

# Matplotlib Default Scatter Plot
axes[0].scatter(x_data, y_data, color='blue', alpha=0.7)
axes[0].set_title('Matplotlib Default Scatter Plot')
axes[0].set_xlabel('X Value')
axes[0].set_ylabel('Y Value')
axes[0].grid(True, linestyle=':', alpha=0.6)

# Seaborn Enhanced Scatter Plot with Regression Line
sns.regplot(x=x_data, y=y_data, ax=axes[1], color='green', scatter_kws={'alpha':0.7, 's':60}, line_kws={'color':'red', 'linewidth':2})
axes[1].set_title('Seaborn Enhanced Scatter Plot (with Regression)')
axes[1].set_xlabel('X Value')
axes[1].set_ylabel('Y Value')
axes[1].grid(True, linestyle=':', alpha=0.6)

plt.tight_layout()
plt.show()
```



Seaborn: Effortless Statistical Insights ⚡☀️

Seaborn is a Python data visualization library **based on Matplotlib**. It provides a **high-level interface** for drawing **attractive and informative statistical graphics**. While Matplotlib gives you **granular control**, Seaborn makes common statistical plots **easy and beautiful** right out of the box.

```
In [25]: import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
%matplotlib inline
```

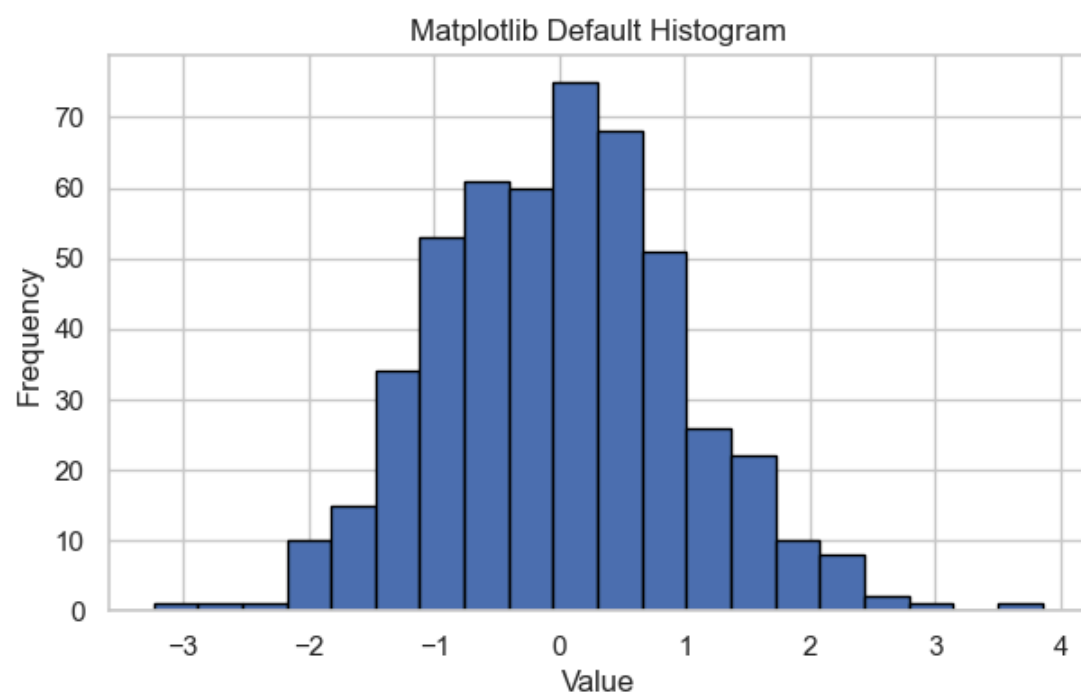
```
In [26]: # Set a style for better aesthetics (optional, but good for consistency)
sns.set_theme(style="whitegrid")
```

1. Why Seaborn? A Visual Comparison ✨

Let's compare a simple plot generated with default Matplotlib settings versus the same plot generated with Seaborn. Notice the difference in **aesthetics and the simplicity of the code**.

```
In [27]: # Generate sample data for comparison
np.random.seed(42)
data_compare = np.random.normal(loc=0, scale=1, size=500) # Normally distributed data
```

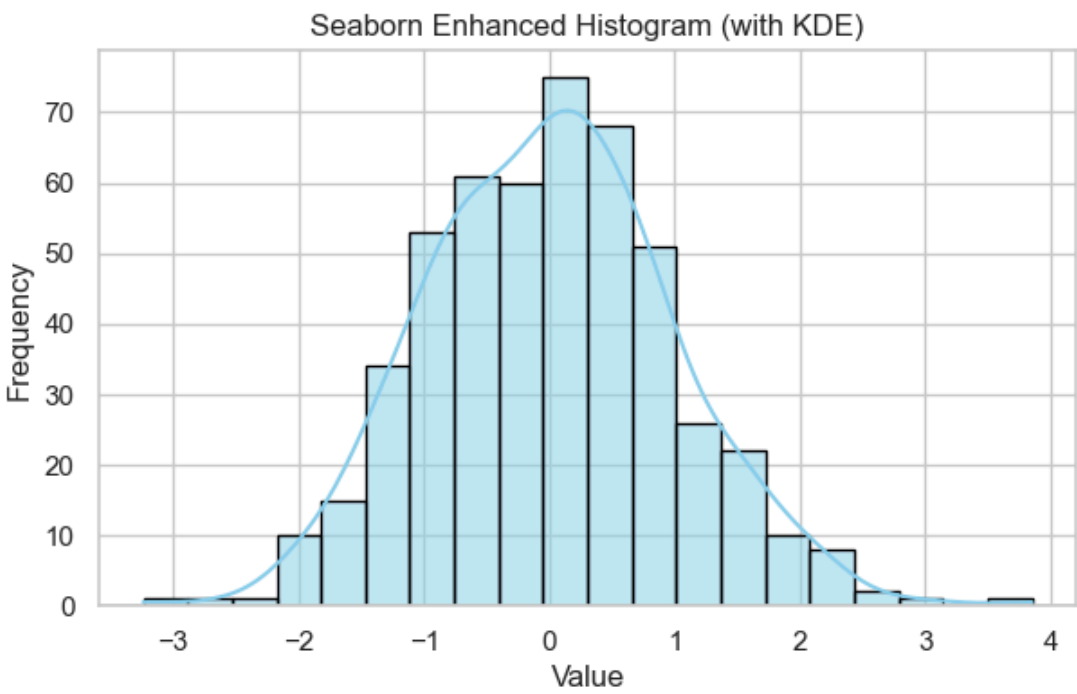
```
In [28]: # --- Matplotlib Default Histogram ---
fig_mpl, ax_mpl = plt.subplots(figsize=(7, 4))
ax_mpl.hist(data_compare, bins=20, edgecolor='black')
ax_mpl.set_title('Matplotlib Default Histogram')
ax_mpl.set_xlabel('Value')
ax_mpl.set_ylabel('Frequency')
plt.show()
```



Now, let's create a similar plot using Seaborn. ✨

Notice how Seaborn **automatically** adds a **Kernel Density Estimate (KDE)** and provides a **more polished look** with less explicit styling.

```
In [29]: fig_sns, ax_sns = plt.subplots(figsize=(7, 4))
sns.histplot(data_compare, bins=20, kde=True, color='skyblue', edgecolor='black', ax=ax_sns)
ax_sns.set_title('Seaborn Enhanced Histogram (with KDE)')
ax_sns.set_xlabel('Value')
ax_sns.set_ylabel('Frequency')
plt.show()
```



Key Takeaways from the Comparison:

Aesthetics: Seaborn plots often look more modern and professional by default, with better color choices and typography.

Functionality: Seaborn functions often include statistical elements (like KDE in `histplot`) that would require more manual coding in Matplotlib.

Code Simplicity: For common statistical plots, Seaborn typically requires fewer lines of code.

This ease of use and improved visual appeal makes Seaborn an invaluable tool for quickly understanding your data and get insightful meaning from your data. 🔥

Slide 10

```
In [36]: import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns

sns.set_theme(style="whitegrid", palette="Spectral")

iris = sns.load_dataset("iris")

fig, axes = plt.subplots(2, 2, figsize=(14, 10))

sns.scatterplot(data=iris, x="sepal_length", y="sepal_width", hue="species", style="species", s=100, ax=axes[0, 0], edgecolor='w', linewidth=0.5, legend=True)
axes[0, 0].set_title('Sepal Length vs. Width by Species')
axes[0, 0].set_xlabel('Sepal Length (cm)')
axes[0, 0].set_ylabel('Sepal Width (cm)')
#axes[0, 0].legend(title='Species')

sns.kdeplot(data=iris, x="petal_length", y="petal_width", hue="species", fill=True, ax=axes[0, 1], alpha=0.6, legend=True)
axes[0, 1].set_title('Petal Length & Width Density by Species')
axes[0, 1].set_xlabel('Petal Length (cm)')
axes[0, 1].set_ylabel('Petal Width (cm)')
#axes[0, 1].legend(title='Species')

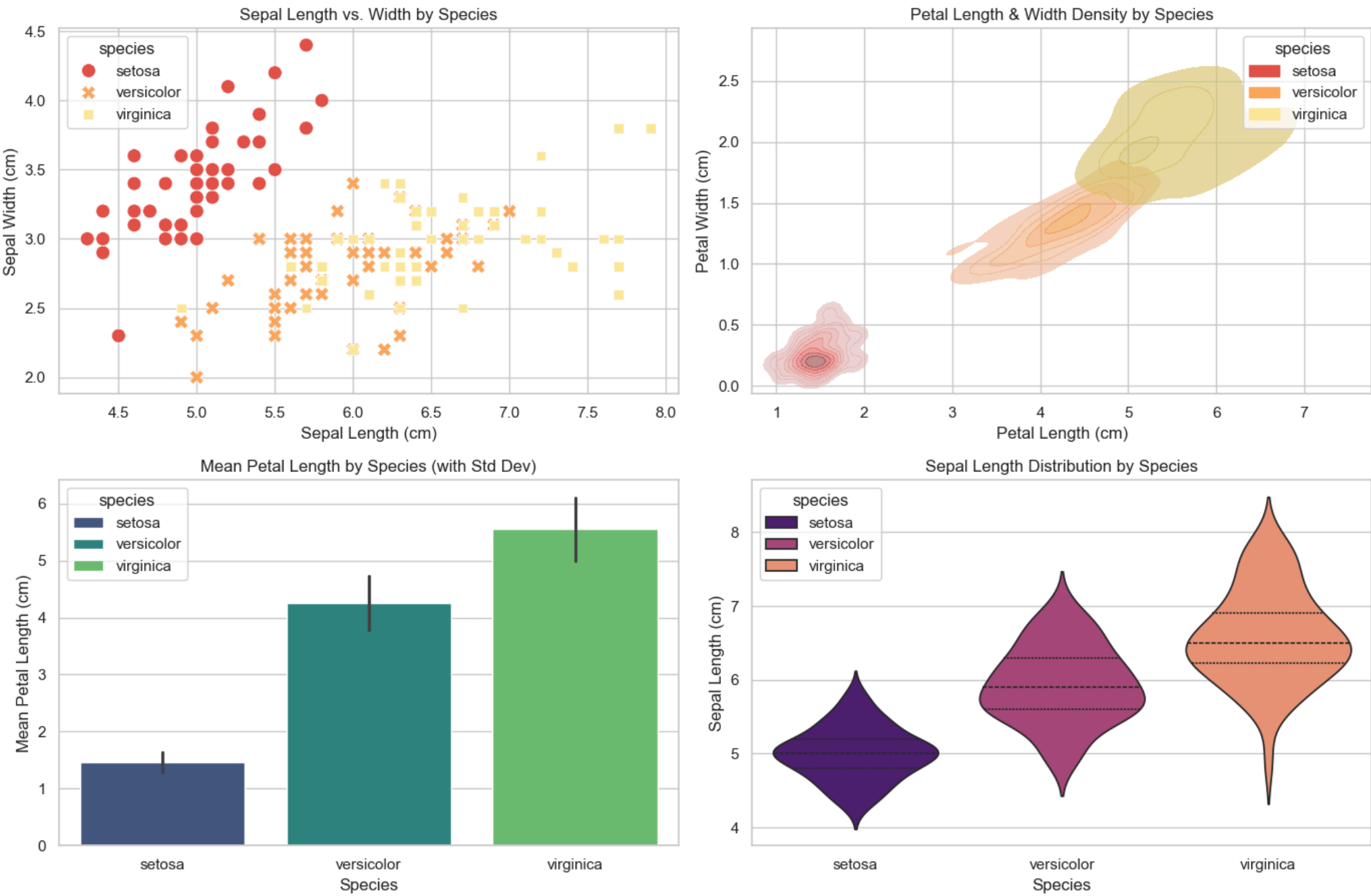
sns.barplot(data=iris, x="species", y="petal_length", hue="species", estimator=np.mean, errorbar='sd', ax=axes[1, 0], palette='viridis', legend=True)
axes[1, 0].set_title('Mean Petal Length by Species (with Std Dev)')
axes[1, 0].set_xlabel('Species')
axes[1, 0].set_ylabel('Mean Petal Length (cm)')
#axes[1, 0].legend().set_visible(False)

sns.violinplot(data=iris, x="species", y="sepal_length", hue="species", inner="quartile", ax=axes[1, 1], palette='magma', legend=True)
axes[1, 1].set_title('Sepal Length Distribution by Species')
axes[1, 1].set_xlabel('Species')
axes[1, 1].set_ylabel('Sepal Length (cm)')
#axes[1, 1].legend().set_visible(False)

plt.tight_layout(rect=[0, 0.03, 1, 0.95])
fig.suptitle('Seaborn\'s Statistical & Aesthetic Power: A Multi-View Analysis', fontsize=20, fontweight='bold', color='darkslategray')
```

```
plt.show()
```

Seaborn's Statistical & Aesthetic Power: A Multi-View Analysis



2. Common Seaborn Statistical Plots 🎨

Seaborn provides specialized functions for **common statistical visualization tasks**. Let's explore some of the **most frequently used ones**.

```
In [37]: # Load a built-in Seaborn dataset for richer examples
# The 'tips' dataset is commonly used for demonstrating various plot types.
tips_df = sns.load_dataset('tips')
print("First 5 rows of the 'tips' dataset:")
print(tips_df.head())
print("\nDataset Info:")
tips_df.info()
```

First 5 rows of the 'tips' dataset:

	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3
3	23.68	3.31	Male	No	Sun	Dinner	2
4	24.59	3.61	Female	No	Sun	Dinner	4

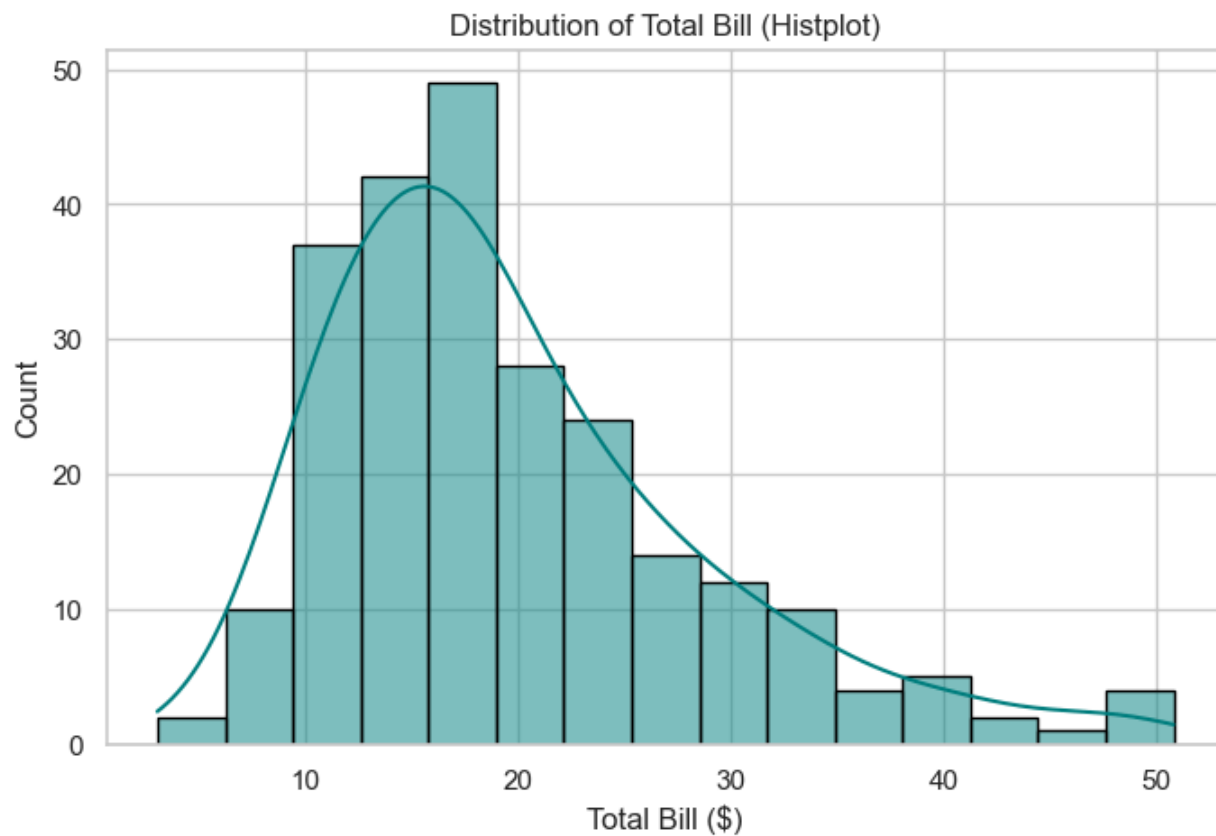
Dataset Info:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 244 entries, 0 to 243
Data columns (total 7 columns):
#   Column      Non-Null Count  Dtype
---  -
0   total_bill  244 non-null    float64
1   tip         244 non-null    float64
2   sex        244 non-null    category
3   smoker     244 non-null    category
4   day        244 non-null    category
5   time       244 non-null    category
6   size       244 non-null    int64
dtypes: category(4), float64(2), int64(1)
memory usage: 7.4 KB
```

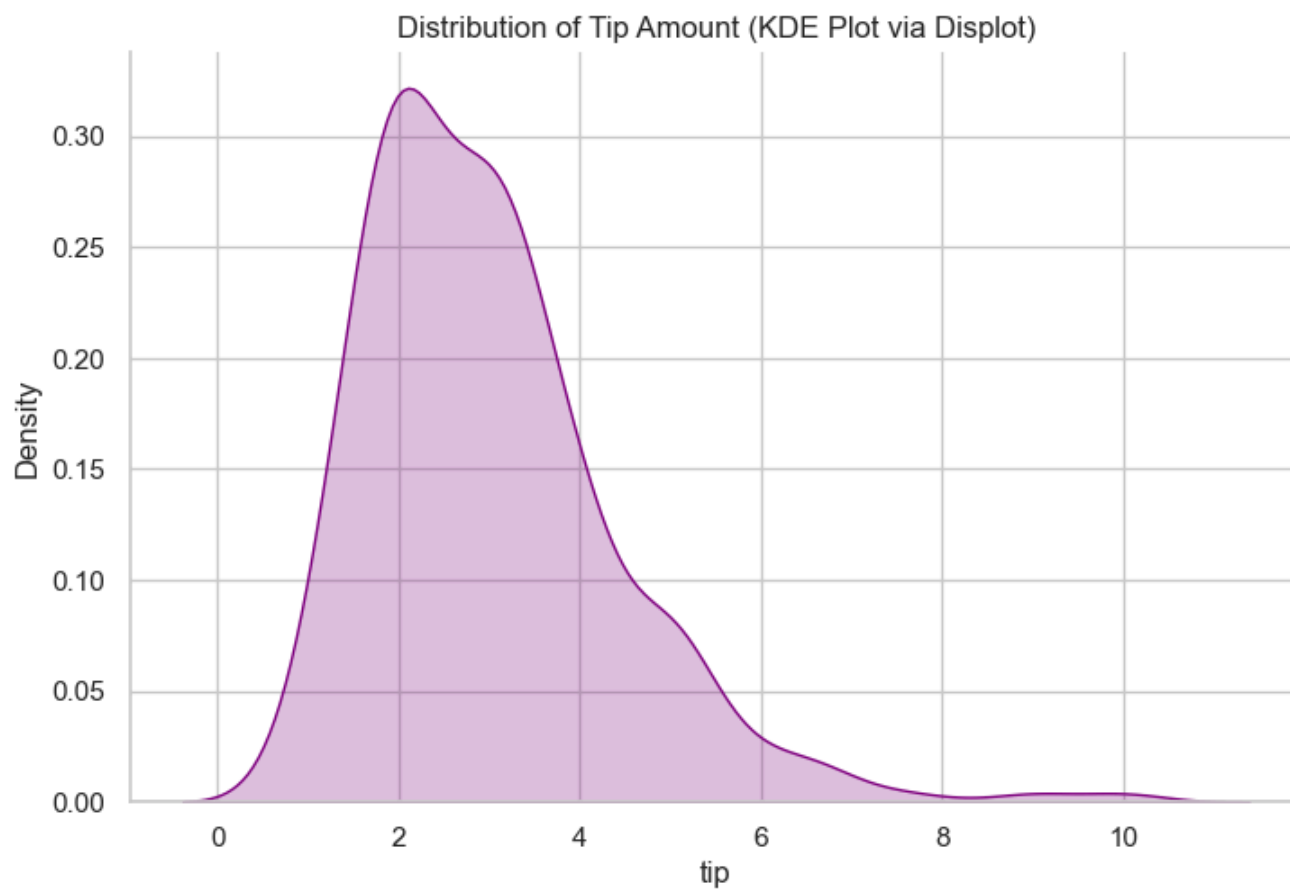
2.1. Distribution Plots (histplot, kdeplot, displot) ✨

These plots help visualize the **distribution of a single numerical variable**. `displot` is a **figure-level function** that can combine `histplot`, `kdeplot`, and `rugplot`.

```
In [38]: # Histogram with KDE using histplot
plt.figure(figsize=(8, 5))
sns.histplot(data=tips_df, x='total_bill', kde=True, bins=15, color='teal', edgecolor='black')
plt.title('Distribution of Total Bill (Histplot)')
plt.xlabel('Total Bill ($)')
plt.ylabel('Count')
plt.show()
```



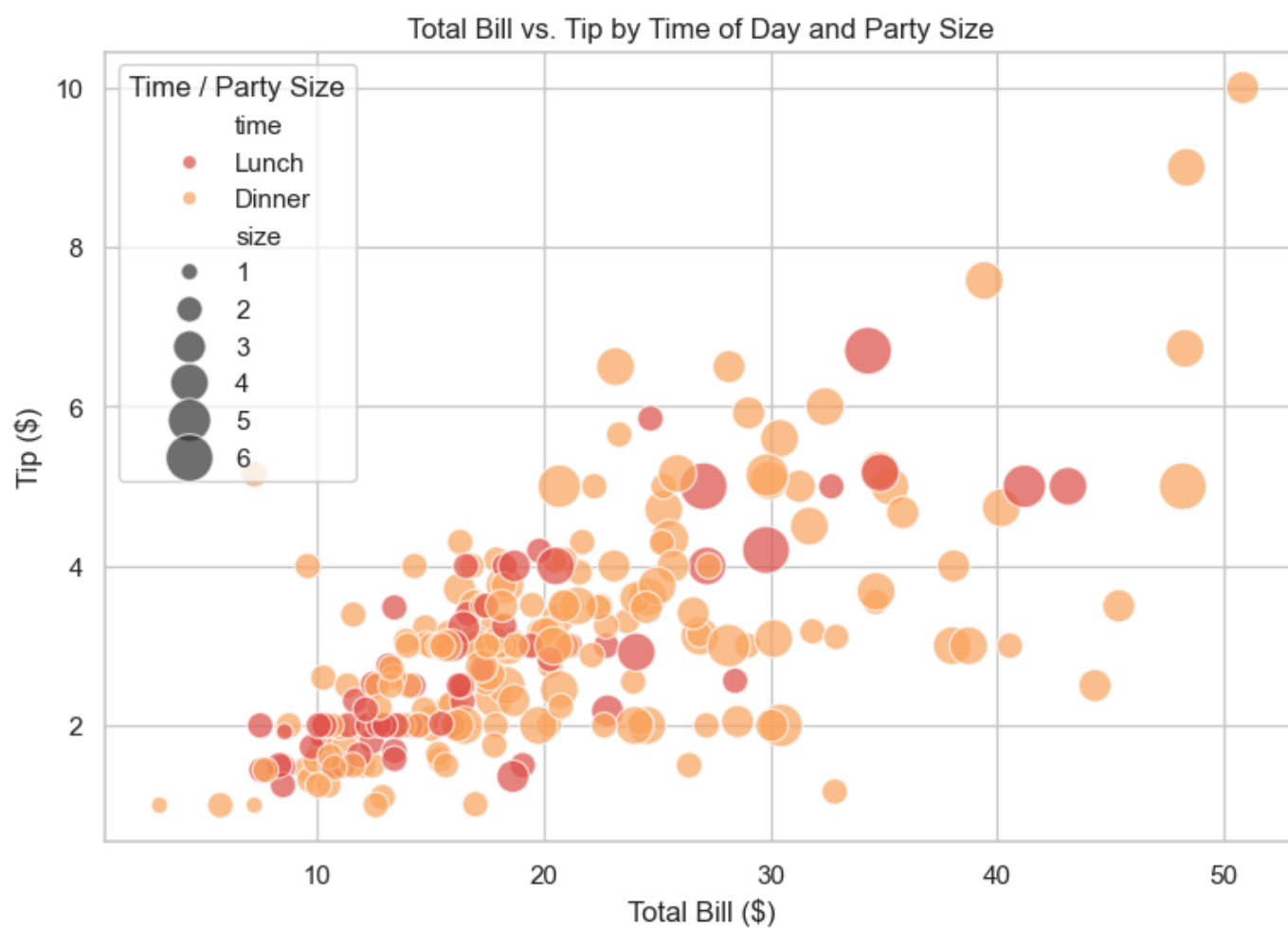
```
In [39]: # Combined distribution plot using displot
# displot is a figure-level function, so it creates its own Figure and Axes.
sns.displot(data=tips_df, x='tip', kind='kde', fill=True, color='purple', height=5, aspect=1.5)
plt.title('Distribution of Tip Amount (KDE Plot via Displot)') # Title for the figure
plt.show()
```



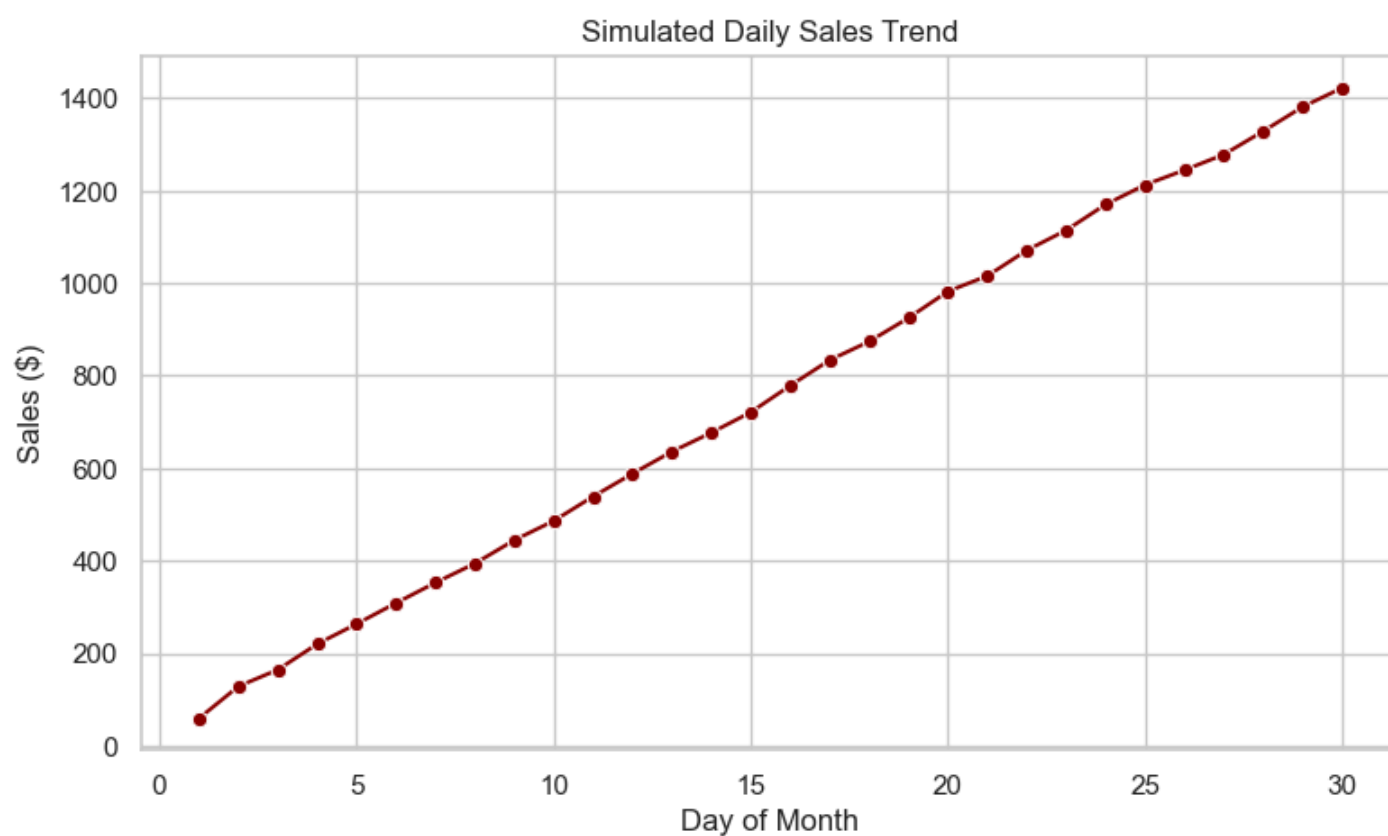
2.2. Relational Plots (scatterplot, lineplot, relplot) 🕶

These plots visualize the relationship between two or more variables. `relplot` is a figure-level function that can create `scatterplot` or `lineplot` and allows for easy faceting.

```
In [40]: # Scatter plot with hue and size mapping
plt.figure(figsize=(9, 6))
sns.scatterplot(data=tips_df, x='total_bill', y='tip', hue='time', size='size', sizes=(50, 400), alpha=0.7, edgecolor='w')
plt.title('Total Bill vs. Tip by Time of Day and Party Size')
plt.xlabel('Total Bill ($)')
plt.ylabel('Tip ($)')
plt.legend(title='Time / Party Size')
plt.show()
```



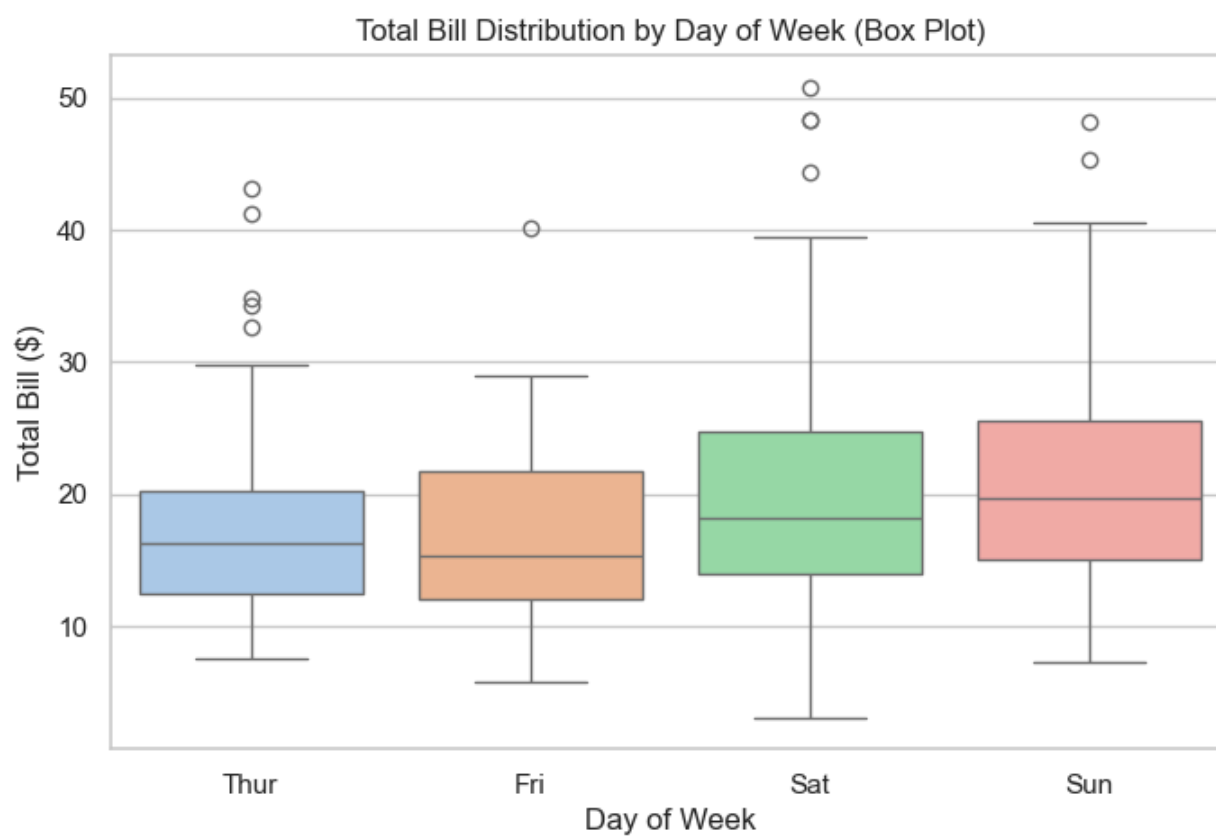
```
In [41]: # Line plot (for ordered data, though tips_df is not time-series, this demonstrates basic usage)
# Let's create some dummy time-series like data for demonstration 🤖
time_data = pd.DataFrame({
    'Day': np.arange(1, 31),
    'Sales': np.cumsum(np.random.randn(30) * 10 + 50)
})
plt.figure(figsize=(9, 5))
sns.lineplot(data=time_data, x='Day', y='Sales', color='darkred', marker='o')
plt.title('Simulated Daily Sales Trend')
plt.xlabel('Day of Month')
plt.ylabel('Sales ($)')
plt.show()
```



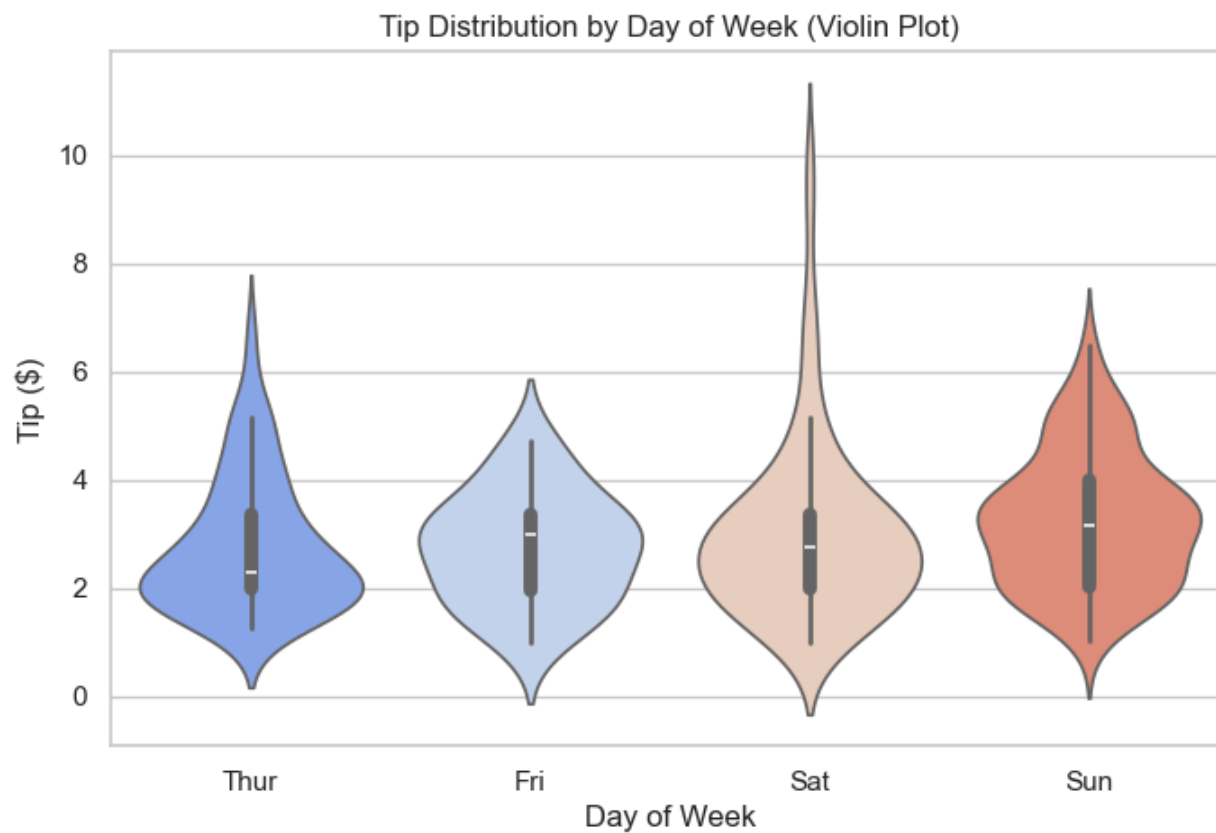
2.3. Categorical Plots (boxplot, violinplot, catplot) 🧩

These plots visualize the relationship between a numerical variable and one or more categorical variables. `catplot` is a figure-level function for categorical plots.

```
In [43]: # Box Plot: Distribution of total bill by day
plt.figure(figsize=(8, 5))
sns.boxplot(data=tips_df, x='day', y='total_bill', hue='day', palette='pastel', legend=False)
plt.title('Total Bill Distribution by Day of Week (Box Plot)')
plt.xlabel('Day of Week')
plt.ylabel('Total Bill ($)')
plt.show()
```

```
In [45]: # Violin Plot: Distribution of tip by day
plt.figure(figsize=(8, 5))
sns.violinplot(data=tips_df, x='day', y='tip', hue='day', palette='coolwarm', legend=False)
plt.title('Tip Distribution by Day of Week (Violin Plot)')
plt.xlabel('Day of Week')
plt.ylabel('Tip ($)')
plt.show()
```

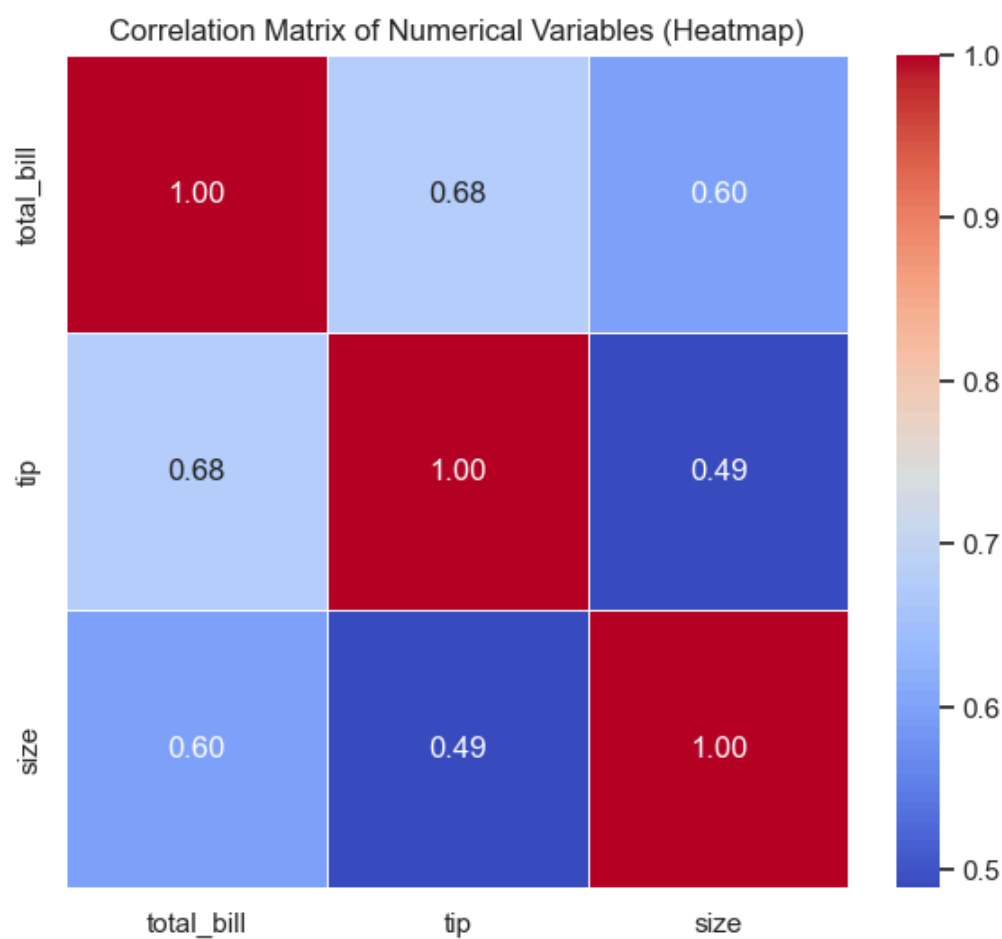


2.4. Matrix Plots (heatmap)

Heatmaps are excellent for visualizing **matrices**, such as **correlation matrices**, where the **intensity of color represents the value**.

```
In [46]: # Calculate correlation matrix
correlation_matrix = tips_df[['total_bill', 'tip', 'size']].corr()

# Create a heatmap
plt.figure(figsize=(7, 6))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', fmt=".2f", linewidths=.5)
plt.title('Correlation Matrix of Numerical Variables (Heatmap)')
plt.show()
```



Conclusion: Seaborn's Power for Statistical Analysis 🪄 ✨ 🔥

Seaborn streamlines the process of creating visually **appealing** and **statistically informative** plots. By **abstracting** much of the Matplotlib **boilerplate**, it allows you to **focus more on the data and less on the plotting mechanics**. This makes it an indispensable tool for **Exploratory Data Analysis** and communicating **complex** statistical relationships **effectively**.

Slide 11

```
In [47]: import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns

sns.set_theme(style="whitegrid", palette="deep")

tips = sns.load_dataset("tips")

fig, axes = plt.subplots(1, 2, figsize=(15, 7), sharey=True)

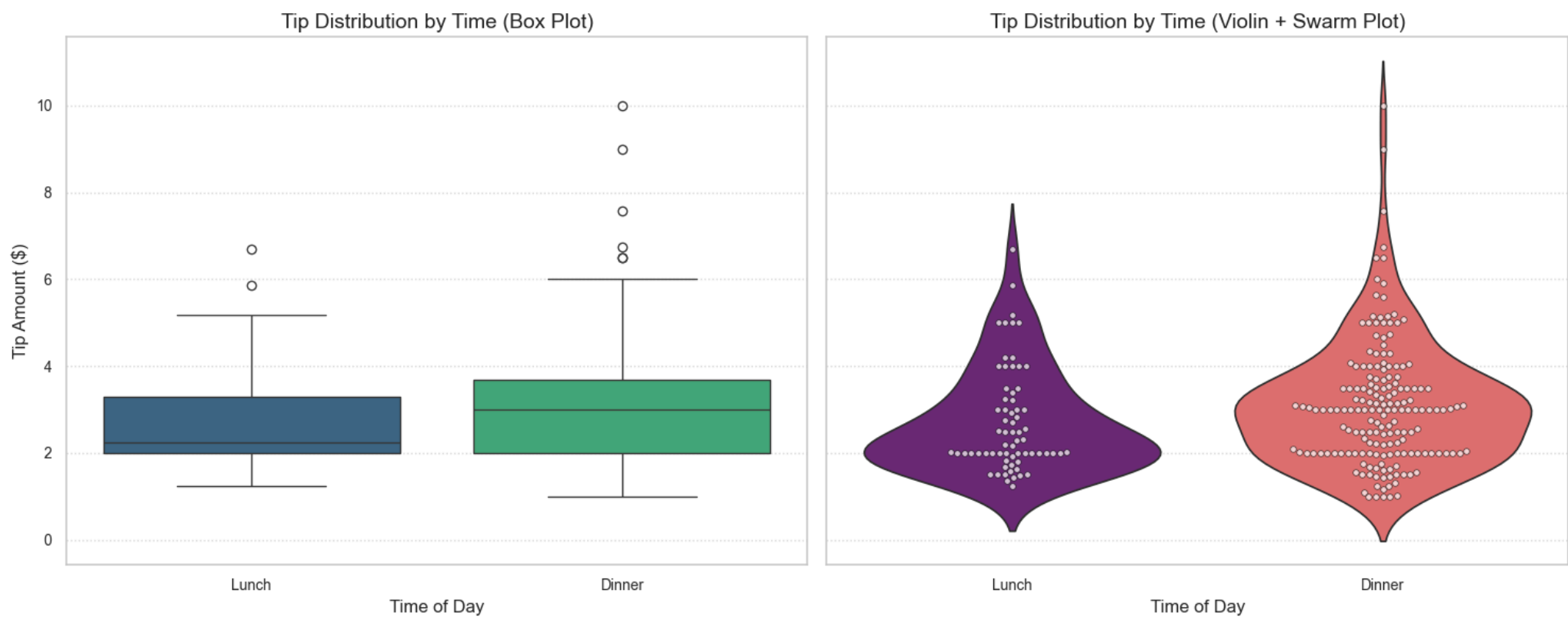
sns.boxplot(data=tips, x="time", y="tip", ax=axes[0], hue="time", legend=False, palette="viridis")
axes[0].set_title('Tip Distribution by Time (Box Plot)', fontsize=14)
axes[0].set_xlabel('Time of Day', fontsize=12)
axes[0].set_ylabel('Tip Amount ($)', fontsize=12)
axes[0].tick_params(labelsize=10)
axes[0].grid(axis='y', linestyle=':', alpha=0.7)

sns.violinplot(data=tips, x="time", y="tip", ax=axes[1], hue="time", legend=False, palette="magma", inner=None)
sns.swarmplot(data=tips, x="time", y="tip", ax=axes[1], color="white", edgecolor="black", linewidth=0.5, size=4, alpha=0.7)
axes[1].set_title('Tip Distribution by Time (Violin + Swarm Plot)', fontsize=14)
axes[1].set_xlabel('Time of Day', fontsize=12)
axes[1].set_ylabel('')
axes[1].tick_params(labelsize=10)
axes[1].grid(axis='y', linestyle=':', alpha=0.7)

fig.suptitle('Comparing Tip Distributions: Box Plot vs. Violin Plot with Data Points', fontsize=18, fontweight='bold', color='darkslategray')

plt.tight_layout(rect=[0, 0.03, 1, 0.95])
plt.show()
```

Comparing Tip Distributions: Box Plot vs. Violin Plot with Data Points



3. Comparing Distributions Across Categories

Understanding how a numerical variable's distribution **differs** across various categories is a fundamental task in data analysis. Seaborn provides powerful tools like `boxplot` and `violinplot` to **quickly reveal these insights**.

```
In [48]: # Load the Iris 🌸 dataset for a classic comparison example
# This dataset contains measurements of iris flowers, categorized by species.
iris_df = sns.load_dataset('iris')
print("\nFirst 5 rows of the 'iris' dataset:")
print(iris_df.head())
print("\nDataset Info:")
iris_df.info()
```

```
First 5 rows of the 'iris' dataset:
   sepal_length  sepal_width  petal_length  petal_width  species
0           5.1           3.5           1.4           0.2   setosa
1           4.9           3.0           1.4           0.2   setosa
2           4.7           3.2           1.3           0.2   setosa
3           4.6           3.1           1.5           0.2   setosa
4           5.0           3.6           1.4           0.2   setosa
```

```
Dataset Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
 #   Column          Non-Null Count  Dtype
---  -
 0   sepal_length    150 non-null   float64
 1   sepal_width     150 non-null   float64
 2   petal_length    150 non-null   float64
 3   petal_width     150 non-null   float64
 4   species         150 non-null   object
dtypes: float64(4), object(1)
memory usage: 6.0+ KB
```

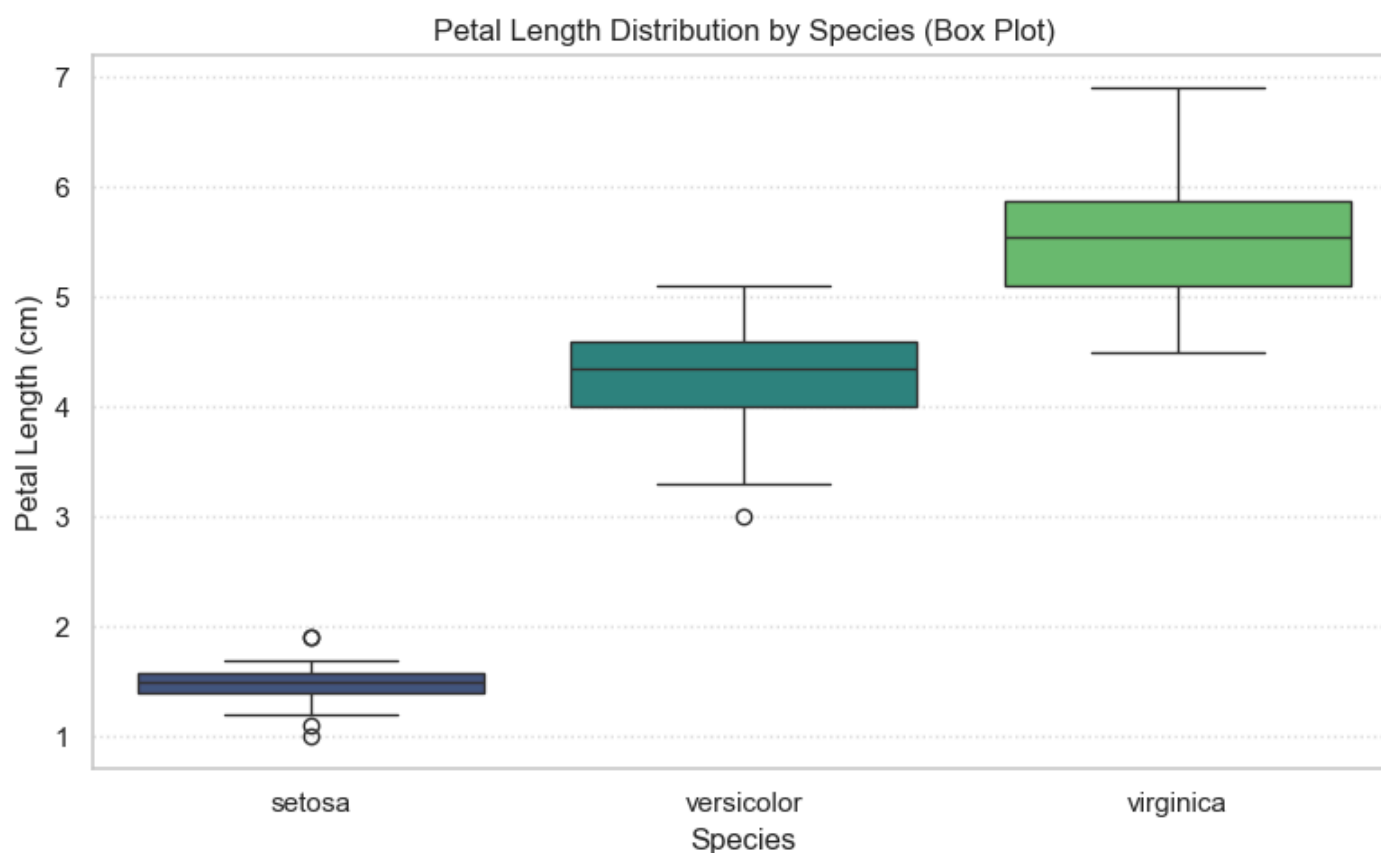
3.1. `sns.boxplot()`: Quick Summaries and Outlier Detection

The box plot summarizes the data's distribution using **quartiles (25th, 50th/median, 75th percentiles)** and **highlights potential outliers**. It's excellent for a **concise comparison of central tendency and spread across groups**.

```
In [49]: fig, ax = plt.subplots(figsize=(8, 5))

# Box plot: Petal Length distribution by Species from 'iris' dataset
sns.boxplot(data=iris_df, x='species', y='petal_length', palette='viridis', ax=ax, hue='species', legend=False)
ax.set_title('Petal Length Distribution by Species (Box Plot)')
ax.set_xlabel('Species')
ax.set_ylabel('Petal Length (cm)')
ax.grid(axis='y', linestyle=':', alpha=0.6)

plt.tight_layout()
plt.show()
```



3.2. `sns.violinplot()`: Full Distribution Shape and Density 🌸

The violin plot extends the box plot by showing the **full probability density** of the data at different **values**. It's particularly useful for revealing **multi-modal distributions** or **subtle shape differences between categories**.

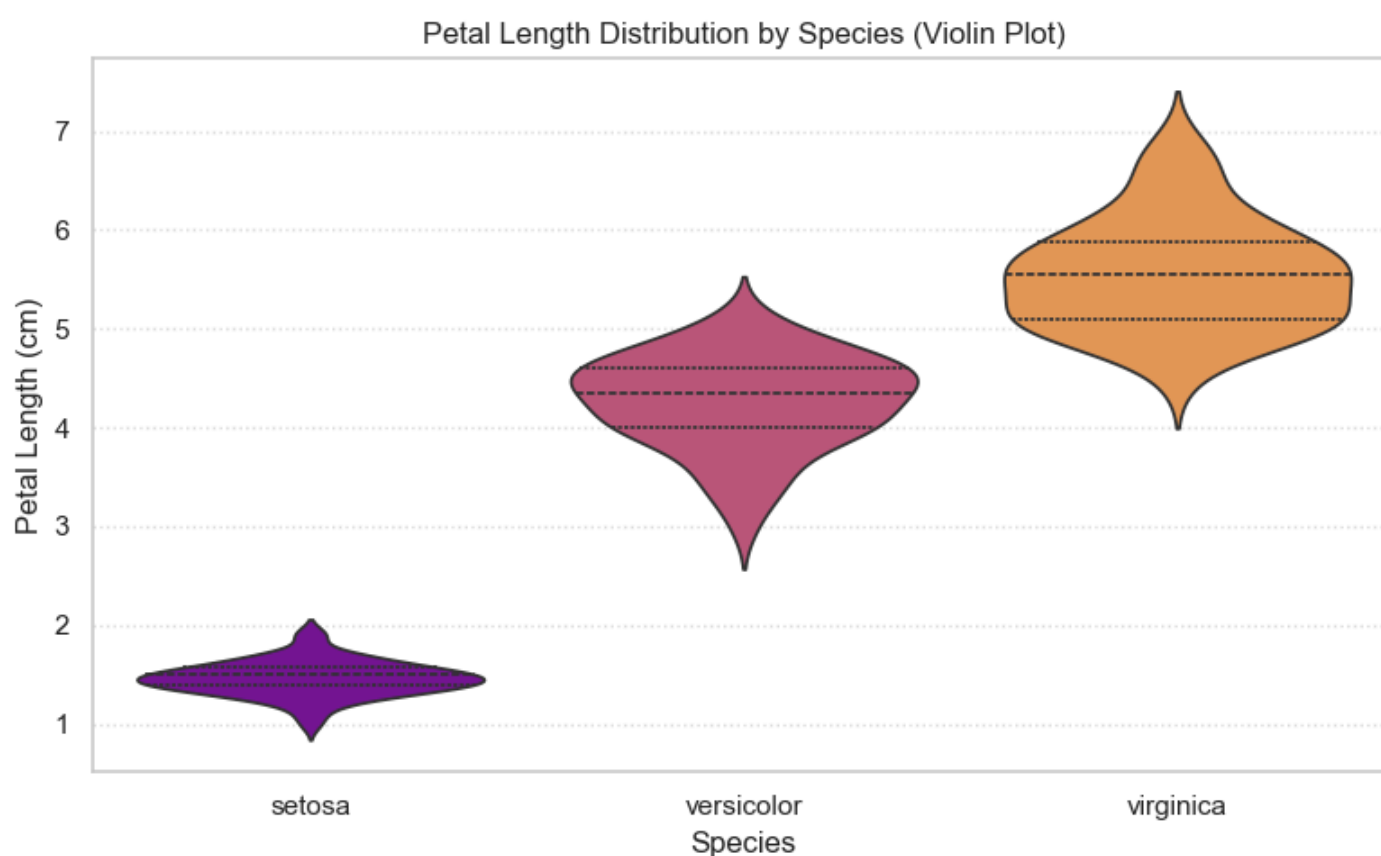
```
In [50]: sns.set_theme(style="whitegrid", palette="viridis")

iris_df = sns.load_dataset('iris')

# Create a Figure and a single Axes specifically for this violin plot
fig, ax = plt.subplots(figsize=(8, 5))

# Violin Plot: Petal Length distribution by Species from 'iris' dataset
# Fix applied: Added hue='species' and Legend=False to address the FutureWarning.
sns.violinplot(data=iris_df, x='species', y='petal_length', palette='plasma', inner='quartile', ax=ax, hue='species', legend=False)
ax.set_title('Petal Length Distribution by Species (Violin Plot)')
ax.set_xlabel('Species')
ax.set_ylabel('Petal Length (cm)')
ax.grid(axis='y', linestyle=':', alpha=0.6)

plt.tight_layout()
plt.show()
```



When to Use Which? 🤔

boxplot: Choose `boxplot` for a **quick, concise summary** of central tendency, spread, and outliers. It's great when you have **many categories** or want to focus on key statistical measures.

violinplot: Opt for `violinplot` when you need to understand the *full shape of the distribution* within each category, including whether it's **unimodal, bimodal, or skewed**. It provides **more detail about the data's density**.

Both plots are invaluable for identifying differences and similarities in data spread between groups, helping you make informed decisions in your engineering and data analysis tasks.

Slide 12

```
In [51]: import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns

sns.set_theme(style="whitegrid", palette="deep")

flights = sns.load_dataset("flights")

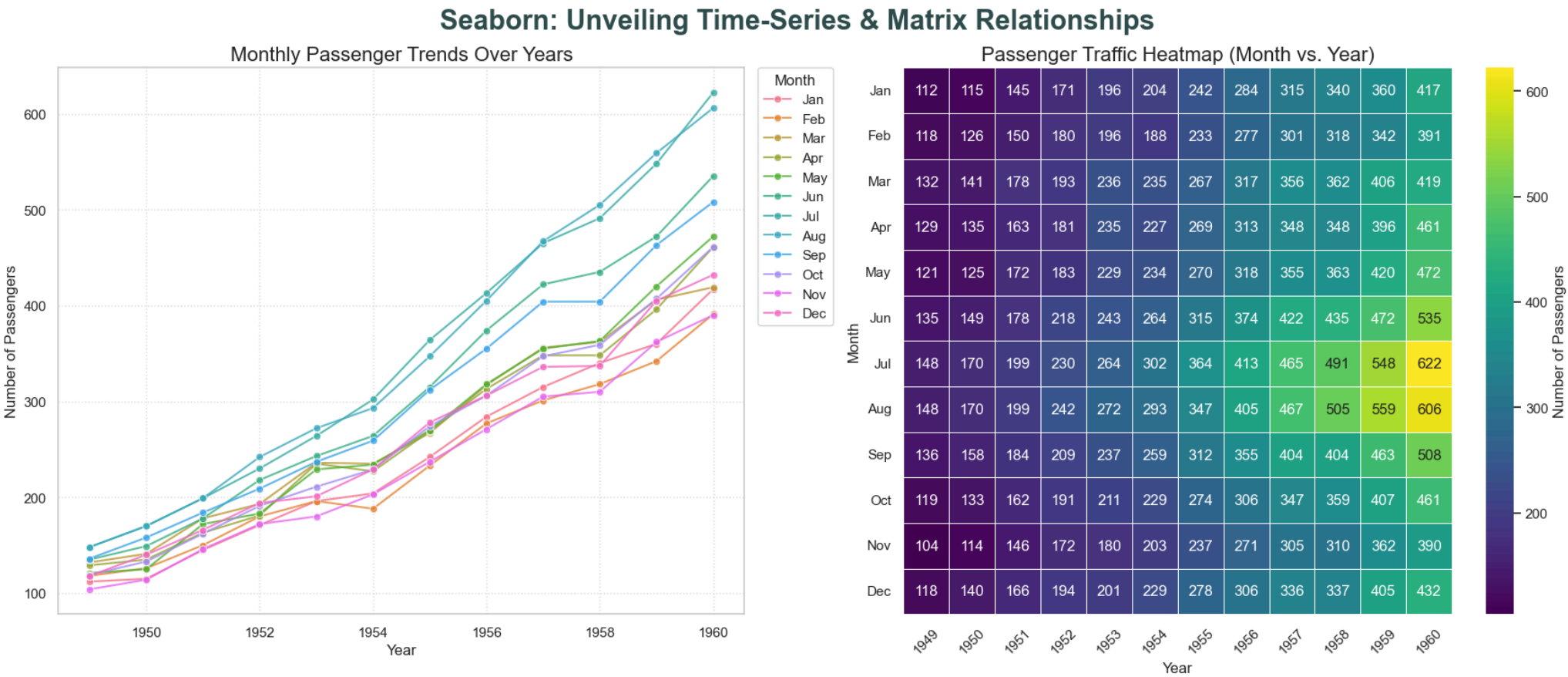
fig, axes = plt.subplots(1, 2, figsize=(18, 8))

sns.lineplot(data=flights, x="year", y="passengers", hue="month",
             marker="o", linewidth=1.5, alpha=0.8,
             ax=axes[0])
axes[0].set_title('Monthly Passenger Trends Over Years', fontsize=16)
axes[0].set_xlabel('Year', fontsize=12)
axes[0].set_ylabel('Number of Passengers', fontsize=12)
axes[0].legend(title='Month', bbox_to_anchor=(1.02, 1), loc='upper left', borderaxespad=0)
axes[0].grid(True, linestyle=':', alpha=0.6)

flights_pivot = flights.pivot_table(index='month', columns='year', values='passengers', observed=False)
sns.heatmap(flights_pivot, annot=True, fmt=".0f", cmap='viridis', linewidths=.5, linecolor='white',
           cbar_kws={'label': 'Number of Passengers'},
           ax=axes[1])
axes[1].set_title('Passenger Traffic Heatmap (Month vs. Year)', fontsize=16)
axes[1].set_xlabel('Year', fontsize=12)
axes[1].set_ylabel('Month', fontsize=12)
axes[1].tick_params(axis='x', rotation=45)
axes[1].tick_params(axis='y', rotation=0)

plt.tight_layout(rect=[0, 0.03, 1, 0.95])
fig.suptitle('Seaborn: Unveiling Time-Series & Matrix Relationships', fontsize=22, fontweight='bold', color='darkslategray')

plt.show()
```



4. Uncovering Relationships !?

Understanding the relationships between variables is paramount in data analysis, especially in machine learning. Seaborn provides powerful and intuitive tools to visualize these connections, helping us identify correlations, clusters, and multi-dimensional patterns that might be hidden in raw data.

```
In [52]: # Load the Iris dataset again for relationship plots

iris_df_rel = sns.load_dataset('iris')
print("\nFirst 5 rows of the 'iris' dataset for relationship plots:")
print(iris_df_rel.head())
```


First 5 rows of the 'iris' dataset for relationship plots:

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa

4.1. sns.scatterplot() with hue and size: Multi-Dimensional Insights 🌟

The scatter plot is a fundamental tool for showing the relationship between two numerical variables. Seaborn enhances this by allowing you to map additional variables to visual properties like color (hue) and point size (size), effectively visualizing three or even four dimensions of data 🤖 .

```
In [53]: import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns

# Set a vibrant and contrasting Seaborn theme for a "WOW" effect
sns.set_theme(style="whitegrid", palette="deep")

# Load the Iris dataset (assuming iris_df_rel is meant to be iris)
iris_df_rel = sns.load_dataset("iris")

# Create a Figure with 1 row and 2 columns of Axes (subplots)
fig_rel, axes_rel = plt.subplots(1, 2, figsize=(16, 7))

# Scatter Plot with hue (species) and size (petal_width)
sns.scatterplot(data=iris_df_rel, x='sepal_length', y='sepal_width',
                hue='species', # Map 'species' to color
                size='petal_length', # Map 'petal_length' to marker size
                sizes=(50, 800), # Range of marker sizes
                alpha=0.8, # Transparency for overlapping points
                edgecolor='w', # White edge for markers
                linewidth=0.5, # Line width for marker edge
                ax=axes_rel[0], # Plot on the first Axes (left)
                palette='tab10') # Use a distinct categorical palette

axes_rel[0].set_title('Sepal Dimensions by Species & Petal Length', fontsize=14)
axes_rel[0].set_xlabel('Sepal Length (cm)', fontsize=12)
axes_rel[0].set_ylabel('Sepal Width (cm) (±)', fontsize=12) # Added (±) as seen in user's image
axes_rel[0].legend(title='Species / Petal Length', bbox_to_anchor=(1.05, 1), loc='upper left') # Place Legend outside
axes_rel[0].grid(True, linestyle=':', alpha=0.6)

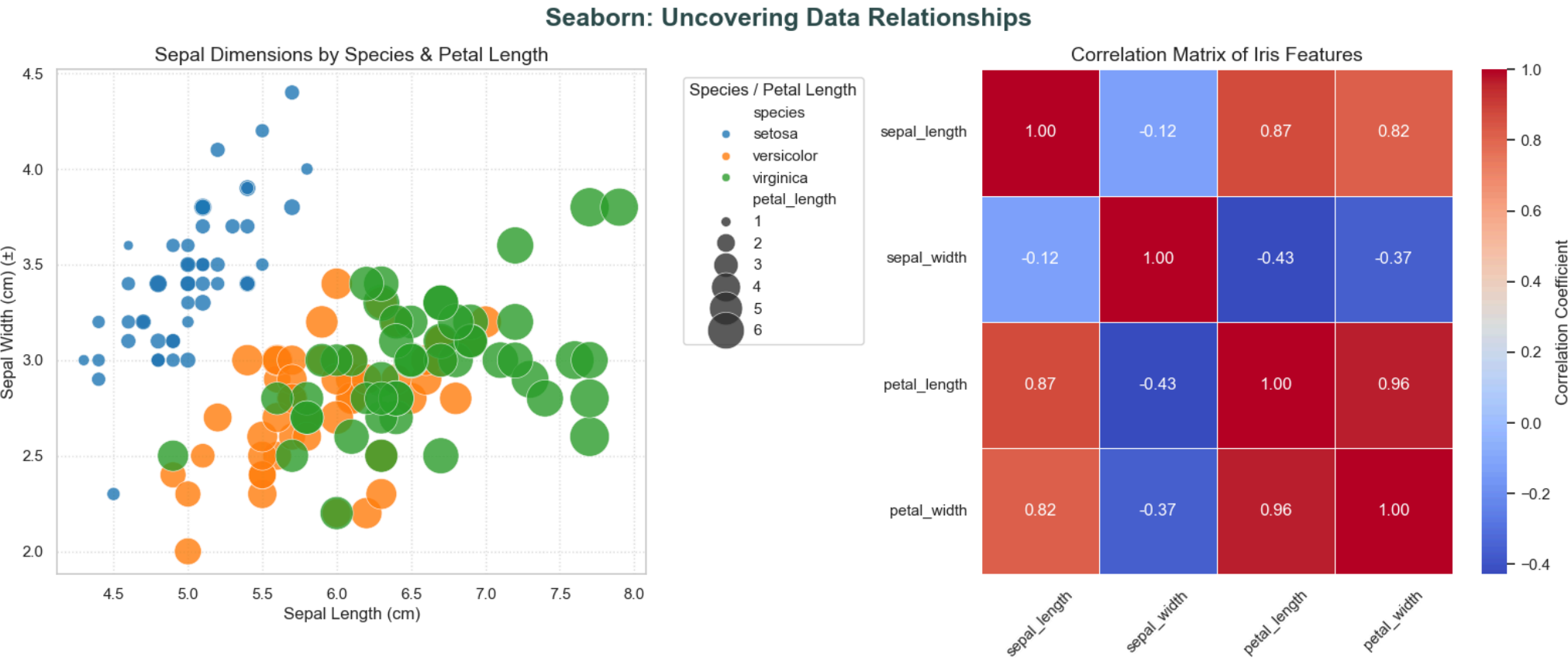
# Calculate the correlation matrix for numerical features in the Iris dataset
correlation_matrix_iris = iris_df_rel.drop(columns=['species']).corr()

# Create a heatmap of the correlation matrix
sns.heatmap(correlation_matrix_iris,
            annot=True, # Show the correlation values on the heatmap
            cmap='coolwarm', # Colormap (e.g., 'coolwarm', 'viridis', 'magma')
            fmt=".2f", # Format annotation values to 2 decimal places
            linewidths=.5, # Add lines between cells
            cbar_kws={'label': 'Correlation Coefficient'}, # Label for the color bar
            ax=axes_rel[1]) # Plot on the second Axes

axes_rel[1].set_title('Correlation Matrix of Iris Features', fontsize=14)
axes_rel[1].tick_params(axis='x', rotation=45) # Rotate x-axis labels for better readability
axes_rel[1].tick_params(axis='y', rotation=0) # Ensure y-axis labels are horizontal

plt.tight_layout(rect=[0, 0.03, 1, 0.95]) # Adjust Layout to make space for supitle
fig_rel.suptitle('Seaborn: Uncovering Data Relationships', fontsize=18, fontweight='bold', color='darkslategray')

plt.show()
```



4.2. sns.heatmap() : Visualizing Correlation Matrices 📊 🙌

Heatmaps are incredibly useful for visualizing matrices, especially **correlation matrices**, which are crucial in machine learning for understanding **feature interactions and multicollinearity**. The **intensity of color** directly represents the **value**.

Conclusion: Unlocking Deeper Insights ✨

`scatterplot` with `hue` and `size` allows us to visualize complex, multi-dimensional relationships, while `heatmap` provides a clear, immediate understanding of correlations within your dataset. These powerful tools are essential for:

Feature Engineering: Identifying which features are most relevant or redundant.

Model Interpretation: Understanding how different input variables might influence your model's predictions.

Data Storytelling: Communicating complex interactions in an intuitive and impactful way.

Mastering these plots will significantly enhance your ability to perform comprehensive data analysis and build more robust machine learning models. 😊 🙌

Slide 13

```
In [56]: import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns

# 'viridis', 'plasma', 'magma', 'cividis' are great sequential palettes.
# For categorical data, 'tab10', 'Paired', 'Set1', 'Dark2' are good.
sns.set_theme(style="whitegrid", palette="Dark2") # Using 'Dark2' for distinct species colors

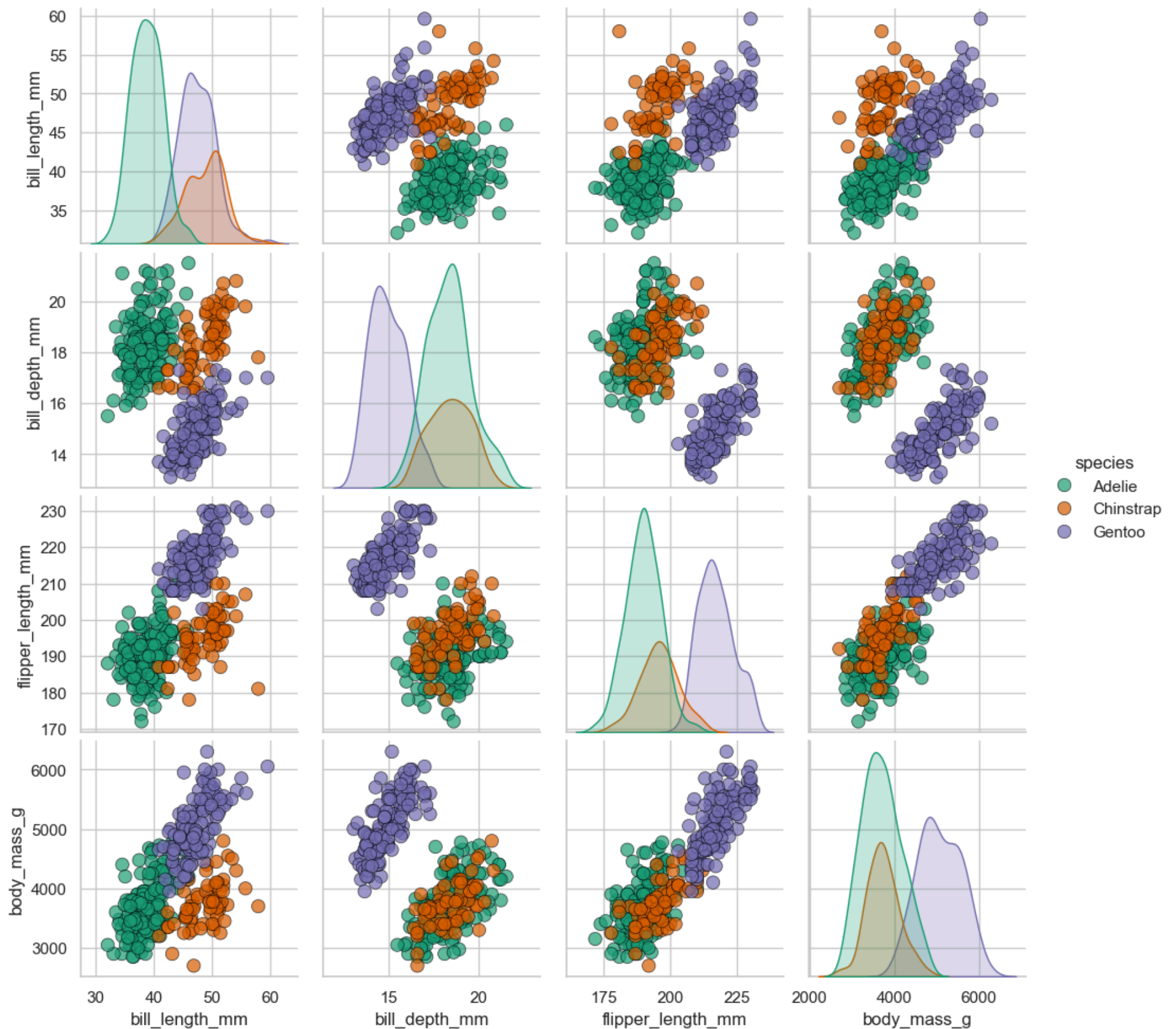
# Load the built-in 'penguins' dataset
# This dataset contains measurements for different penguin species
penguins = sns.load_dataset("penguins")

# Create a pairplot for the penguins dataset
# 'hue' maps the 'species' column to different colors, clearly separating groups.
# 'diag_kind="kde"' shows Kernel Density Estimates on the diagonal, revealing distribution shapes.
# 'height' controls the size of each subplot in the grid.
g = sns.pairplot(data=penguins, hue="species", diag_kind="kde", height=2.5,
                 plot_kws={'alpha': 0.7, 's': 80, 'edgecolor': 'black', 'linewidth': 0.5}) # Customizing scatter points

# Add a super title for the entire Figure
g.fig.suptitle('Penguins: Unveiling Relationships Across Species (sns.pairplot)', y=1.02, # Adjust y to prevent overlap
              fontsize=20, fontweight='bold', color='darkslategray')

plt.show()
```

Penguins: Unveiling Relationships Across Species (sns.pairplot)



5. The "OMG!!!" Moment: `sns.pairplot()` 🤖

This is Seaborn's **ultimate tool** for **quick, holistic data exploration**. With just one line of code, `pairplot()` generates a **grid of pairwise relationships** for all numerical variables in your dataset, **providing endless insights**. 🤖 🤖

In [57]: `# Load the Iris dataset for pairplot`

```
iris_df_pair = sns.load_dataset('iris')
print("\nFirst 5 rows of the 'iris' dataset for pairplot:")
print(iris_df_pair.head())
```

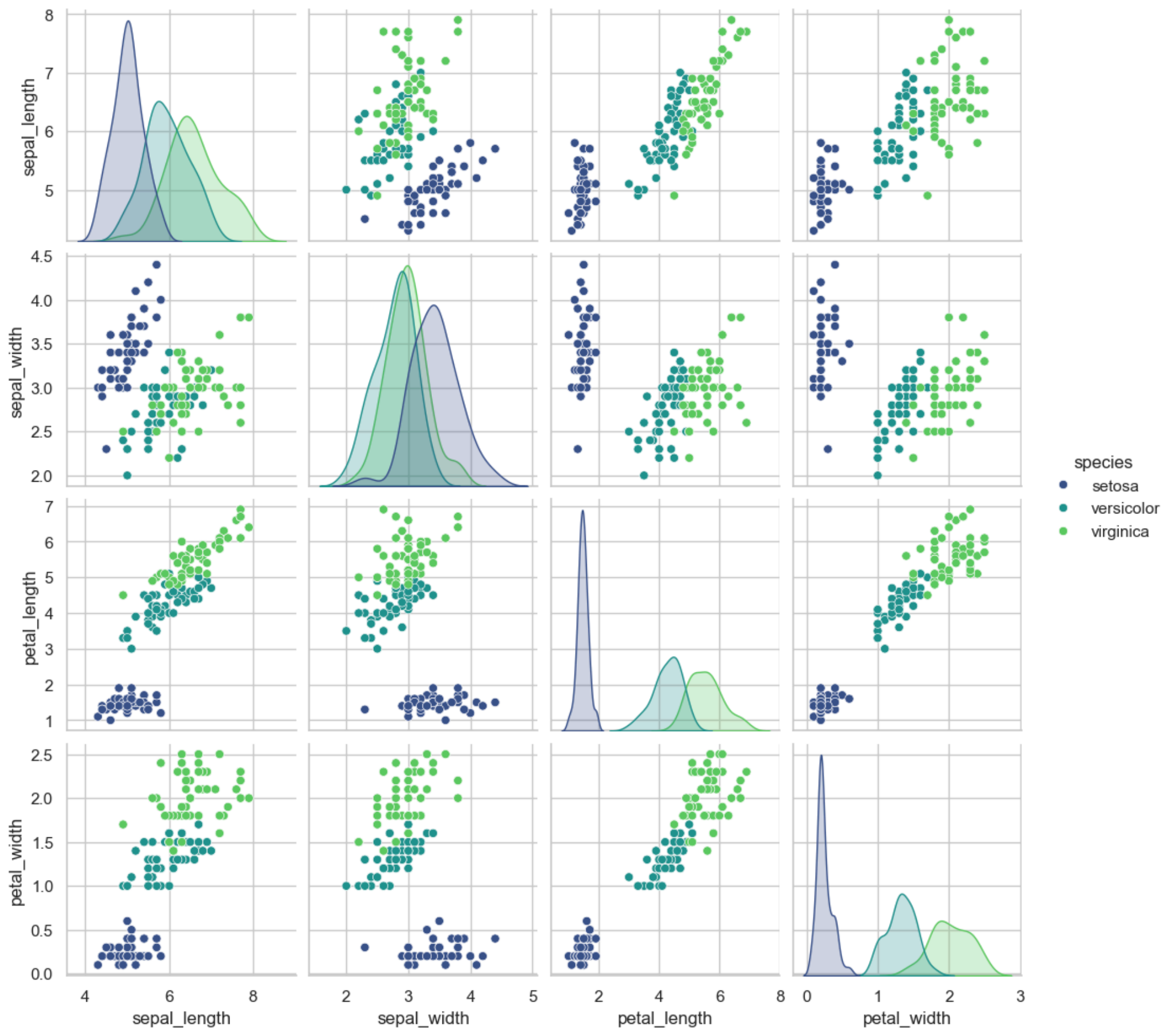
First 5 rows of the 'iris' dataset for pairplot:

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa

In [58]: `# Create a pairplot`
`# The diagonal plots show univariate distributions (histograms by default).`
`# The off-diagonal plots display scatter plots showing relationships between each pair.`
`# 'hue' differentiates categories by color.`

```
sns.pairplot(data=iris_df_pair, hue='species', diag_kind='kde', palette='viridis', height=2.5)
plt.suptitle('Pairwise Relationships of Iris Features by Species', y=1.02, fontsize=16, fontweight='bold', color='darkslategray') # Add a super title
plt.show()
```

Pairwise Relationships of Iris Features by Species



Why `pairplot` is Invaluable: 🤖

Holistic Overview: Get a comprehensive view of your entire numerical dataset at a glance.

Feature Selection: Quickly identify highly correlated features (for multicollinearity) or features that clearly separate classes.

Cluster Identification: Visually spot natural groupings in your data.

Rapid EDA: Accelerate your initial data exploration phase.

`pairplot` is an indispensable tool for any data scientist or engineer looking to quickly understand complex datasets.

Slide 14

```
In [ ]: pip install plotly
```

```
In [61]: import plotly.express as px
import plotly.graph_objects as go
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt # Needed for the static plot comparison
import seaborn as sns # For loading tips dataset for static plot
```

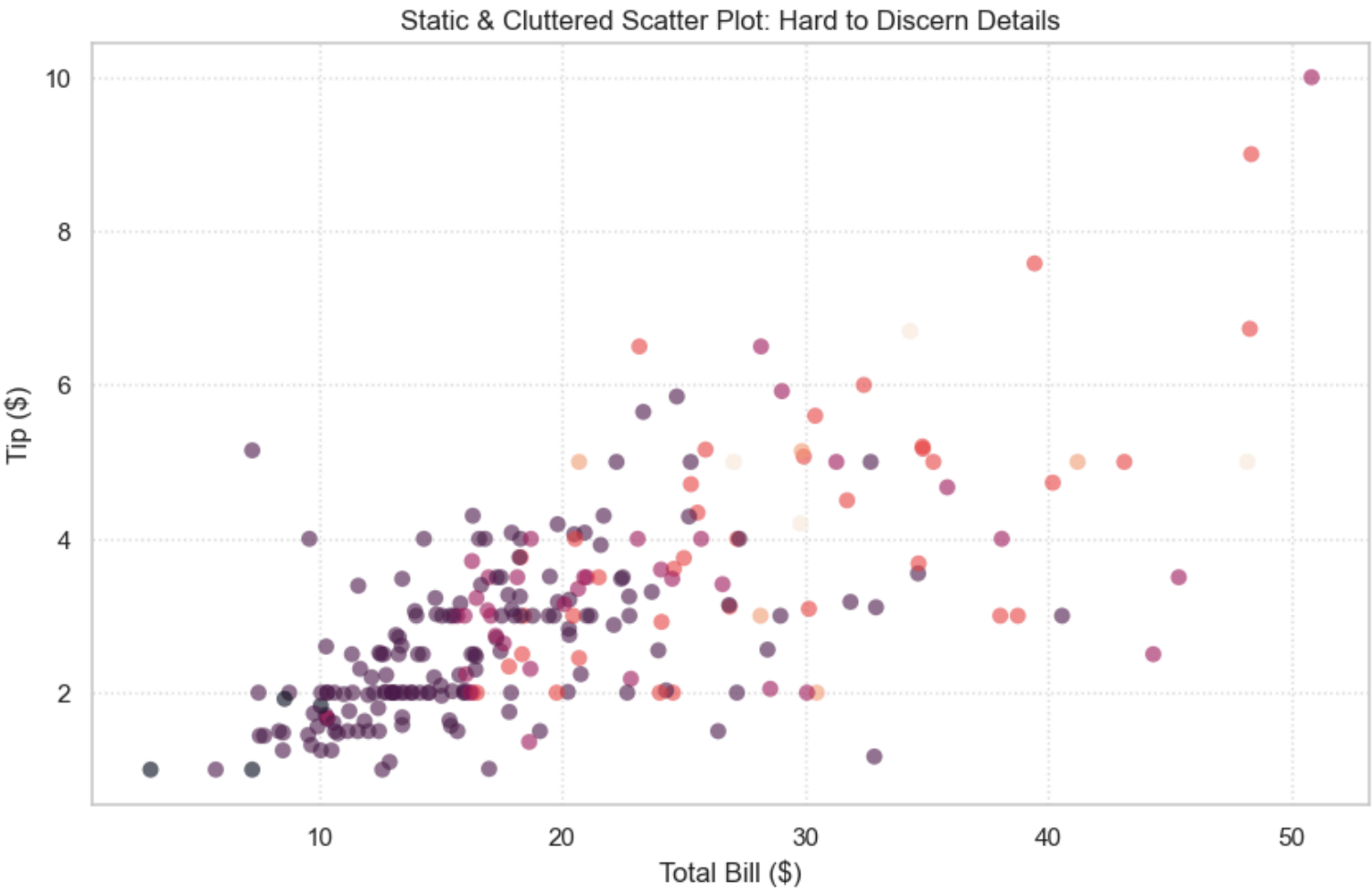


```
tips_df_static = sns.load_dataset('tips')

plt.figure(figsize=(10, 6))
# Plotting total_bill vs tip, but with multiple overlapping categories and default markers/colors
# This makes it hard to distinguish patterns or individual points.
plt.scatter(tips_df_static['total_bill'], tips_df_static['tip'],
            c=tips_df_static['size'], # Map 'size' to color, but with default cmap, it can be messy
            alpha=0.6, s=50, edgecolor='none') # Some transparency, but still dense

plt.title('Static & Cluttered Scatter Plot: Hard to Discern Details')
plt.xlabel('Total Bill ($)')
plt.ylabel('Tip ($)')
plt.grid(True, linestyle=':', alpha=0.6)

plt.show()
```



Plotly: Your Data Comes Alive! 🌈☀️

Plotly is a powerful open-source **graphing library** that allows you to create **interactive, web-based visualizations**. Unlike **static plots from Matplotlib or Seaborn**, Plotly charts can be **zoomed, panned, and hovered over, revealing detailed information**. This **interactivity** is crucial for **exploring large datasets and building dynamic dashboards**.

```
In [62]: import plotly.express as px
import plotly.graph_objects as go
import pandas as pd
import numpy as np
```

1. Why Interactivity? 🤔

Static charts, while useful, have **limitations** when dealing with complex or large datasets. Interactive visualizations overcome these limitations by allowing **dynamic exploration**.

The Limitations of Static Plots ⚠️:

Information Overload: Too many data points or dimensions can make static charts cluttered and unreadable.

Lack of Detail: You can't zoom in on specific areas or get precise values for individual data points.

Limited Exploration: The viewer is passive; they can only see what the creator chose to highlight.

Poor for Large Datasets: Static images struggle to convey patterns and outliers effectively in big data.

The Power of Interactive Visuals 🚀:

Interactive plots, especially those created with Plotly, offer significant advantages:

1 Dynamic Exploration:

Zoom & Pan: Focus on specific regions of interest.

Hover for Details: Get precise values and contextual information by simply moving your mouse over data points.

Filter & Select: Interactively filter data subsets (more advanced, often with Dash).

2 Uncovering Granular Insights:

Easily identify outliers, clusters, and subtle trends that might be hidden in static representations of large datasets.

3 Enhanced Collaborative Analysis:

Empower stakeholders and team members to explore the data themselves, fostering deeper understanding and more informed discussions.

4 Foundation for Web Applications & Dashboards:

Plotly charts are web-native (HTML/JavaScript), making them perfect for embedding in online reports, websites, and building interactive data dashboards with frameworks like Dash.

Let's see some simple examples of an interactive plot to feel the difference 🤖

```
In [ ]: # just for assurance for local run.
pip install ipywidgets

In [68]: import plotly.io as pio

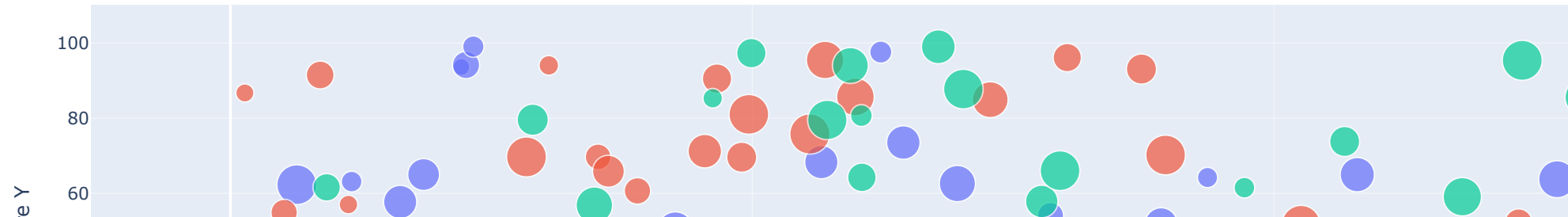
# Generate some sample data for an interactive scatter plot
np.random.seed(42)
num_points = 200
x_interactive = np.random.rand(num_points) * 100
y_interactive = np.random.rand(num_points) * 100
z_interactive = np.random.rand(num_points) * 50 + 10 # For size/color
categories_interactive = np.random.choice(['Group A', 'Group B', 'Group C'], num_points)

interactive_df = pd.DataFrame({
    'X_Value': x_interactive,
    'Y_Value': y_interactive,
    'Size_Metric': z_interactive,
    'Category': categories_interactive,
    'Detail_Info': [f'Point {i}: X={x_interactive[i]:.2f}, Y={y_interactive[i]:.2f}' for i in range(num_points)]
})

# Create an interactive scatter plot using Plotly Express
fig_interactive = px.scatter(interactive_df, x="X_Value", y="Y_Value",
                             size="Size_Metric", color="Category",
                             hover_name="Detail_Info", # Show this info on hover
                             title="Interactive Scatter Plot: Explore Data by Hovering, Zooming, Panning",
                             labels={"X_Value": "Feature X", "Y_Value": "Feature Y", "Size_Metric": "Importance"},
                             height=500)

pio.renderers.default = "notebook"
fig_interactive.show()
```

Interactive Scatter Plot: Explore Data by Hovering, Zooming, Panning



🔔 Notice how you can:

Hover over each point to see its `Detail_Info`.

Zoom in on specific clusters using your mouse scroll or by dragging a box.

Pan around the plot by clicking and dragging.

Toggle categories in the legend to show/hide them.

This immediate feedback and ability to explore is what makes interactive visualization so powerful!

```
In [72]: import plotly.express as px
import pandas as pd
import numpy as np

# --- Simulate data for Shiraz, Iran with realistic busy spots ---

# Approximate central coordinates for Shiraz
shiraz_lat_center = 29.60
shiraz_lon_center = 52.54

# Define known busy/popular locations in Shiraz with approximate coordinates
# These are chosen to represent diverse types of busy areas (historical, commercial, recreational)
busy_spots = {
    "Vakil Bazaar": {"lat": 29.617, "lon": 52.538},
    "Hafezieh": {"lat": 29.605, "lon": 52.558},
    "Eram Garden": {"lat": 29.630, "lon": 52.528},
    "Chamran Blvd": {"lat": 29.620, "lon": 52.490},
    "KhaliJ Fars Complex": {"lat": 29.690, "lon": 52.420},
    "Zeytoon Shopping Center": {"lat": 29.620, "lon": 52.510}
}

num_locations = 200 # More locations for a denser map
np.random.seed(42) # for reproducibility

simulated_data = []

# Generate data points, concentrating some around busy spots
for i in range(num_locations):
    location_name = f"Location {i+1}"

    # Randomly pick a busy spot to cluster around for a subset of points
    if np.random.rand() < 0.6: # 60% chance to be near a busy spot
        chosen_spot_name = np.random.choice(list(busy_spots.keys()))
        spot_coords = busy_spots[chosen_spot_name]
        lat = spot_coords["lat"] + np.random.normal(0, 0.01) # Small deviation
        lon = spot_coords["lon"] + np.random.normal(0, 0.01) # Small deviation

        # Higher car hours and more likely to be peak hour for busy spots
        car_hours = np.random.uniform(30, 80) # Higher range
        peak_hour = np.random.choice(['Morning Peak', 'Evening Peak'], p=[0.5, 0.5]) # More likely peak
        location_name = f"{chosen_spot_name} Area {np.random.randint(1,5)}" # Give a name related to busy spot
    else: # Remaining 40% are more spread out and less busy
        lat = shiraz_lat_center + (np.random.rand() - 0.5) * 0.2 # Wider spread
        lon = shiraz_lon_center + (np.random.rand() - 0.5) * 0.2 # Wider spread
        car_hours = np.random.uniform(5, 35) # Lower range
        peak_hour = np.random.choice(['Morning Peak', 'Evening Peak', 'Off-Peak'], p=[0.2, 0.2, 0.6]) # More likely off-peak

    simulated_data.append([lat, lon, car_hours, peak_hour, location_name])

shiraz_df = pd.DataFrame(simulated_data, columns=[
    'centroid_lat', 'centroid_lon', 'car_hours', 'peak_hour', 'location_name'
])

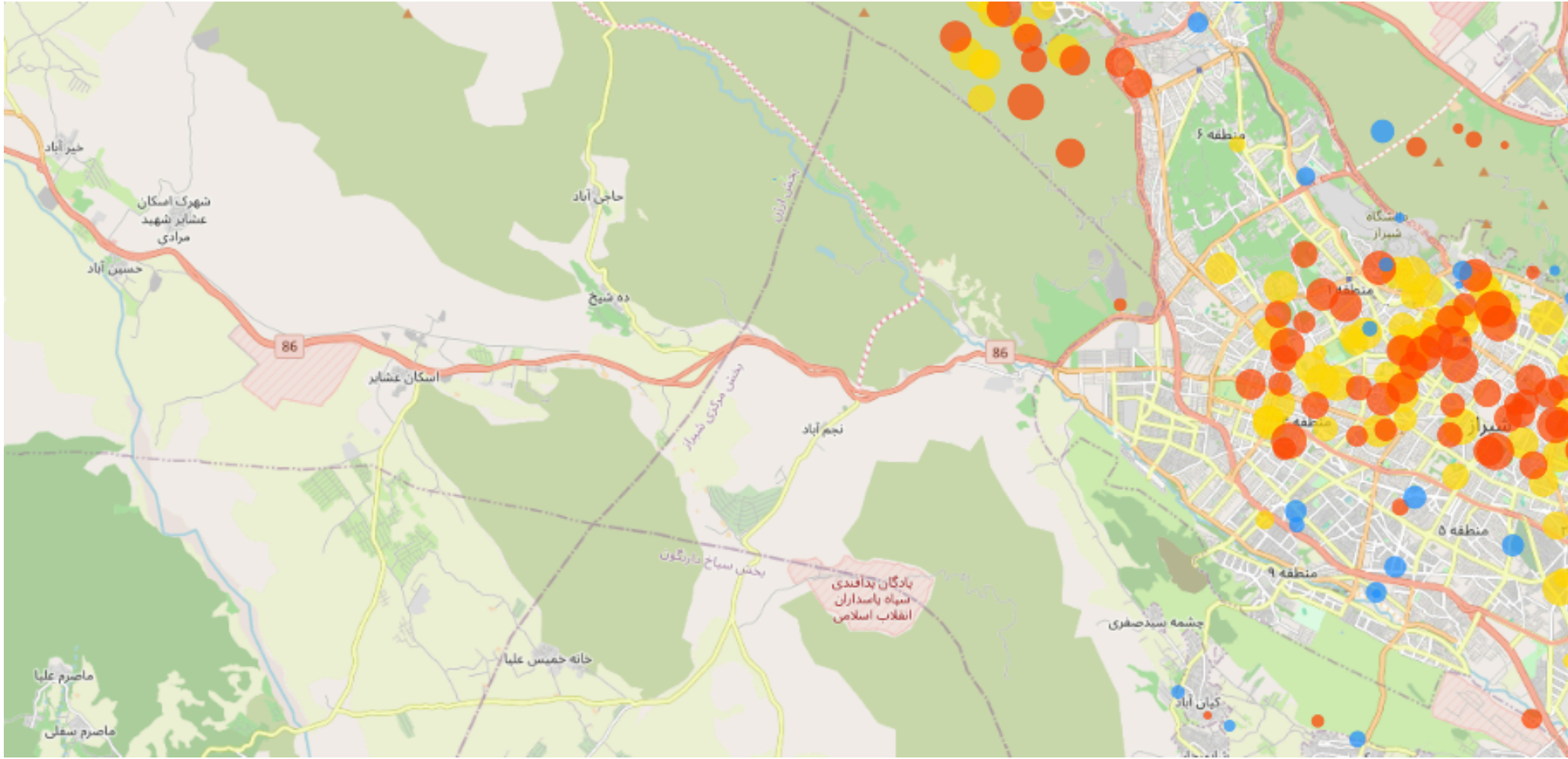
# Define a custom color sequence for peak hours
color_map = {
    'Morning Peak': '#FFD700', # Gold
    'Evening Peak': '#FF4500', # OrangeRed
    'Off-Peak': '#1E90FF'     # DodgerBlue
}

# Create an interactive scatter plot on a map using Plotly Express
fig = px.scatter_map(shiraz_df,
    lat="centroid_lat",
    lon="centroid_lon",
    color="peak_hour", # Color by peak hour usage
    size="car_hours", # Size by car hours
    zoom=11, # Initial zoom level, adjust as needed
    map_style="open-street-map", # Open Street Map style
    hover_name="location_name", # Show location name on hover
    hover_data=["peak_hour", "car_hours"], # Additional data on hover
    title="Interactive Carshare Usage Map: Shiraz, Iran (Simulated Peak Areas)", # title
    labels={"car_hours": "Car Hours (per day)", "peak_hour": "Peak Hour Usage"},
    color_discrete_map=color_map, # Apply custom colors
    height=600)

# set layout to center the map on Shiraz and set margins
fig.update_layout(
    margin={"r":0,"t":50,"l":0,"b":0},
    mapbox_center={"lat": shiraz_lat_center, "lon": shiraz_lon_center},
    mapbox_zoom=11)

fig.show()
```


Interactive Carshare Usage Map: Shiraz, Iran (Simulated Peak Areas)



Another instance with GDP but in 3D 🤔🕶️👉

```
In [73]: import plotly.express as px
import pandas as pd
import numpy as np

# Load the built-in 'gapminder' dataset (original data up to 2007)
# This dataset contains historical data for life expectancy, GDP per capita, and population for various countries.
gapminder_original = px.data.gapminder()

# --- Data Extrapolation to 2025 for a more realistic and extended trend ---
# We aim to extend the time series data from 2007 up to 2025 to visualize more recent trends.
# The extrapolation is done in 2-year steps for smoother animation transitions.

# Get the last available year's data (2007) from the original dataset.
data_2007 = gapminder_original[gapminder_original['year'] == 2007].copy()

# Define the years for which we will extrapolate data.
# This creates a sequence of years from 2007 up to and including 2025, with a step of 2 years.
extrapolation_years = np.arange(2007, 2026, 2).tolist()
# Ensure that 2025 is explicitly included in case the step size doesn't land exactly on it.
if 2025 not in extrapolation_years:
    extrapolation_years.append(2025)
# Remove any duplicate years and sort the list to maintain order.
extrapolation_years = sorted(list(set(extrapolation_years)))

# Initialize an empty list to store all extrapolated dataframes.
all_extrapolated_data = []

# Loop through each extrapolation year to generate new data points.
for year_extrap in extrapolation_years:
    # Skip the year 2007 as its data is already present in the original dataset.
    if year_extrap == 2007:
        continue

    # Create a copy of the 2007 data to modify for the current extrapolation year.
    current_year_data = data_2007.copy()
    # Assign the current extrapolation year to the 'year' column.
    current_year_data['year'] = year_extrap

    # Calculate the number of years passed since 2007 for growth calculation.
    years_diff = year_extrap - 2007

    # Apply a simple growth/increase logic to GDP per capita, life expectancy, and population.
    # These are random uniform factors to simulate varied growth across countries, adjusted for the time difference.
    # GDP per Capita: Assumes an average annual growth rate (e.g., 1-2.5% per 2 years).
    current_year_data['gdpPercap'] = current_year_data['gdpPercap'] * (1 + np.random.uniform(0.01, 0.025, len(data_2007)) * (years_diff / 2))
    # Life Expectancy: Assumes a slower, additive increase (e.g., 0.05-0.15 years per 2 years).
    current_year_data['lifeExp'] = current_year_data['lifeExp'] + np.random.uniform(0.05, 0.15, len(data_2007)) * (years_diff / 2)
    # Population: Assumes an average annual growth rate (e.g., 0.25-0.75% per 2 years).
    current_year_data['pop'] = current_year_data['pop'] * (1 + np.random.uniform(0.0025, 0.0075, len(data_2007)) * (years_diff / 2))

    # Add the newly generated data for the current extrapolation year to the list.
    all_extrapolated_data.append(current_year_data)

# Concatenate the original Gapminder data with all the newly extrapolated dataframes.
# `ignore_index=True` resets the index of the combined DataFrame.
gapminder_extended = pd.concat([gapminder_original] + all_extrapolated_data, ignore_index=True)

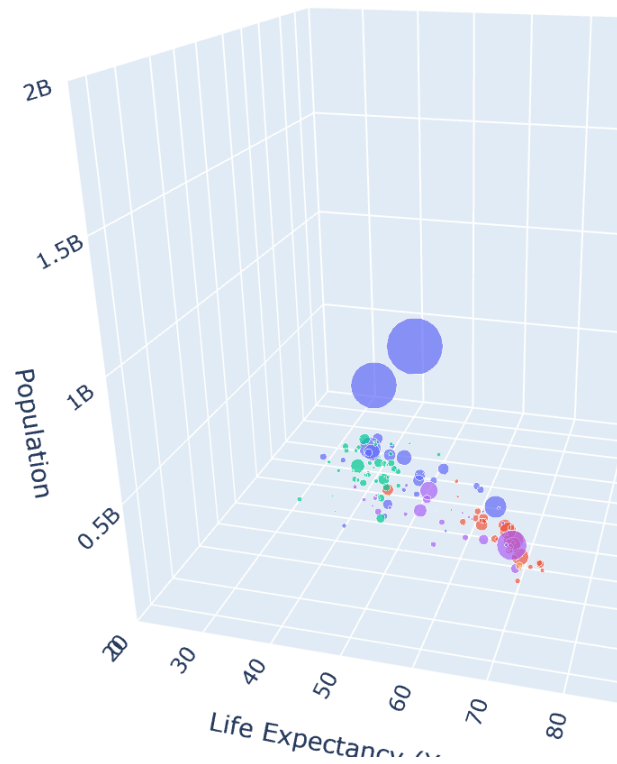
# Ensure the 'year' column is of integer type, which is crucial for `animation_frame` in Plotly Express.
gapminder_extended['year'] = gapminder_extended['year'].astype(int)
```

```
# Create an interactive, animated 3D Scatter Plot using Plotly Express.
# This plot visualizes the evolution of countries across three key development indicators.
fig = px.scatter_3d(gapminder_extended,
                    x="gdpPercap",          # X-axis represents GDP per Capita.
                    y="lifeExp",           # Y-axis represents Life Expectancy.
                    z="pop",               # Z-axis represents Population.
                    color="continent",    # Points are colored based on their respective continent.
                    size="pop",           # The size of each point is scaled by its population, providing visual emphasis.
                    animation_frame="year", # The plot animates over the 'year' column, creating a time-series visualization.
                    animation_group="country", # Ensures smooth transitions for each country across animation frames.
                    hover_name="country",  # Displays the country name when hovering over a point.
                    # Formats additional data shown on hover: GDP (no decimals), LifeExp (1 decimal), Pop (shortened format).
                    hover_data={"gdpPercap": ':.0f', "lifeExp": ':.1f', "pop": ':.0s'},
                    log_x=True,           # Uses a logarithmic scale for the X-axis (GDP per Capita) for better distribution visibility.
                    size_max=80,          # Sets the maximum size for the markers to prevent them from becoming too large.
                    # Defines the fixed range for the X-axis to ensure consistent view across animation frames,
                    # adjusted to better show Lower GDP values.
                    range_x=[50, 150000],
                    # Defines the fixed range for the Y-axis (Life Expectancy).
                    range_y=[20, 95],
                    # Defines the fixed range for the Z-axis (Population), adjusted to better show smaller countries.
                    range_z=[10000, 2000000000],
                    # Sets the main title of the plot.
                    title="Global Development Trends: Life Expectancy, GDP, and Population Over Time (3D)",
                    # Customizes axis labels for clarity.
                    labels={"gdpPercap": "GDP per Capita (Log Scale)",
                           "lifeExp": "Life Expectancy (Years)",
                           "pop": "Population"},
                    height=700) # Sets the height of the plot in pixels for a better 3D view.

# Update the layout of the figure for better presentation and initial camera angle.
fig.update_layout(
    # Configures the 3D scene camera's initial orientation.
    scene_camera=dict(
        up=dict(x=0, y=0, z=1), # Defines the 'up' direction in the 3D scene.
        center=dict(x=0, y=0, z=0), # Defines the center point around which the camera orbits.
        # Sets the initial position of the camera's eye, adjusted to provide a better view
        # of countries with Lower GDP and Life Expectancy.
        eye=dict(x=2.0, y=0.5, z=0.8)
    ),
    # Sets the margins around the plot (right, top, left, bottom) to zero for maximum plot area.
    margin={"r":0,"t":50,"l":0,"b":0},
    # Ensures the Legend is displayed.
    showlegend=True,
    # Refines the animation speed for smoother observation of changes over time.
    transition=dict(duration=1500, easing="linear")
)

# Displays the interactive Plotly figure.
fig.show()
```

Global Development Trends: Life Expectancy, GDP, and Population Over Time (3D)



```
In [74]: import plotly.express as px
import pandas as pd

# Load the built-in 'tips' dataset
tips = px.data.tips()

# --- 1. Interactive Scatter Plot (with hover tooltips) ---
# This plot visualizes the relationship between total bill and tip amount.
# You can hover over each point to see details about the day, time, and party size.
fig_scatter = px.scatter(tips,
                        x="total_bill", y="tip",
                        color="day",
                        size="size",
                        title="Total Bill vs Tip Amount by Day and Party Size",
                        labels={"total_bill": "Total Bill ($)", "tip": "Tip Amount ($)"},
                        hover_data={"day": True, "time": True, "size": True})

fig_scatter.show()

# --- 2. Interactive Line Plot (Zoomable) ---
# This plot visualizes the average total bill over the days of the week.
# You can zoom in and pan to examine specific trends and details.
daily_avg = tips.groupby('day')['total_bill'].mean().reset_index()
fig_line = px.line(daily_avg,
                  x="day", y="total_bill",
                  markers=True,
                  title="Average Total Bill by Day of the Week (Interactive)",
                  labels={"total_bill": "Average Total Bill ($)", "day": "Day of the Week"})

fig_line.show()

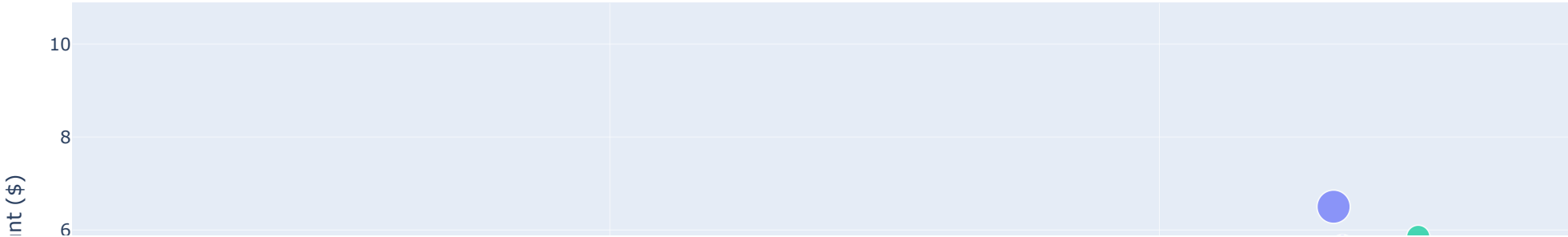
# --- 3. Interactive Bar Chart (with hover tooltips) ---
# This plot compares the total bill across different days of the week.
# Hovering over each bar reveals its exact value.
fig_bar = px.bar(tips,
                 x="day", y="total_bill",
                 color="time",
                 title="Total Bill by Day and Time (Interactive)",
                 labels={"total_bill": "Total Bill ($)", "day": "Day of the Week"})

fig_bar.show()

# --- 4. Interactive 3D Scatter Plot (Explore Multi-dimensional Data) ---
# This plot allows you to explore the relationship between three variables (total bill, tip, and size)
# in a rotatable 3D space.
fig_3d = px.scatter_3d(tips,
                      x="total_bill", y="tip", z="size",
                      color="sex",
                      title="3D View: Total Bill, Tip, and Party Size by Gender",
                      labels={"total_bill": "Total Bill", "tip": "Tip Amount", "size": "Party Size"})

fig_3d.show()
```

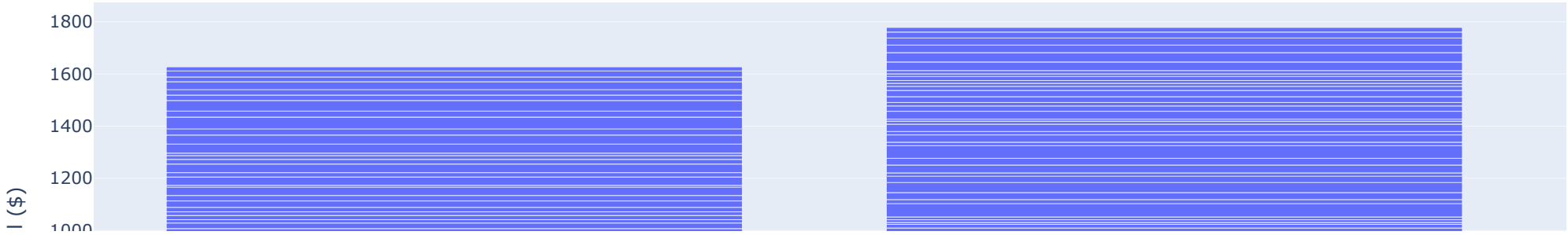
Total Bill vs Tip Amount by Day and Party Size



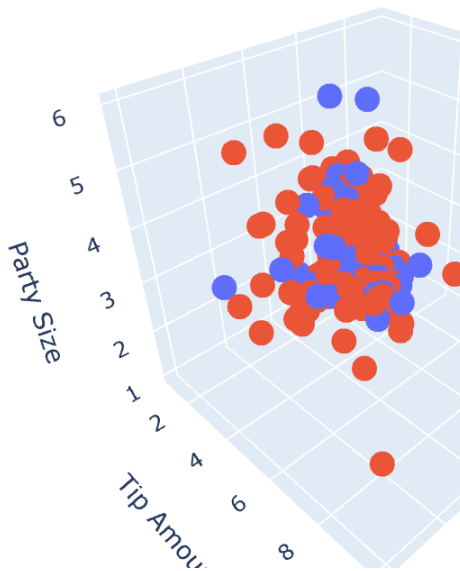
Average Total Bill by Day of the Week (Interactive)



Total Bill by Day and Time (Interactive)



3D View: Total Bill, Tip, and Party Size by Gender



2. Plotly Express: Your First Interactive Visuals 🧐 🎨

`plotly.express` (px) is a **high-level API** designed to make creating interactive plots **incredibly simple**. You can create a rich, interactive visualization in **just one line of code**.

```
In [75]: # Load a built-in dataset for demonstration
# The 'gapminder' dataset is perfect for showing global trends with multiple dimensions.
gapminder_px = px.data.gapminder()
print("First 5 rows of the 'gapminder' dataset:")
print(gapminder_px.head())
print("\nDataset Info:")
gapminder_px.info()
```

First 5 rows of the 'gapminder' dataset:

	country	continent	year	lifeExp	pop	gdpPercap	iso_alpha	\
0	Afghanistan	Asia	1952	28.801	8425333	779.445314	AFG	
1	Afghanistan	Asia	1957	30.332	9240934	820.853030	AFG	
2	Afghanistan	Asia	1962	31.997	10267083	853.100710	AFG	
3	Afghanistan	Asia	1967	34.020	11537966	836.197138	AFG	
4	Afghanistan	Asia	1972	36.088	13079460	739.981106	AFG	

	iso_num
0	4
1	4
2	4
3	4
4	4

Dataset Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1704 entries, 0 to 1703
Data columns (total 8 columns):
Column Non-Null Count Dtype
--- -
0 country 1704 non-null object
1 continent 1704 non-null object
2 year 1704 non-null int64
3 lifeExp 1704 non-null float64
4 pop 1704 non-null int64
5 gdpPercap 1704 non-null float64
6 iso_alpha 1704 non-null object
7 iso_num 1704 non-null int64
dtypes: float64(2), int64(3), object(3)
memory usage: 106.6+ KB

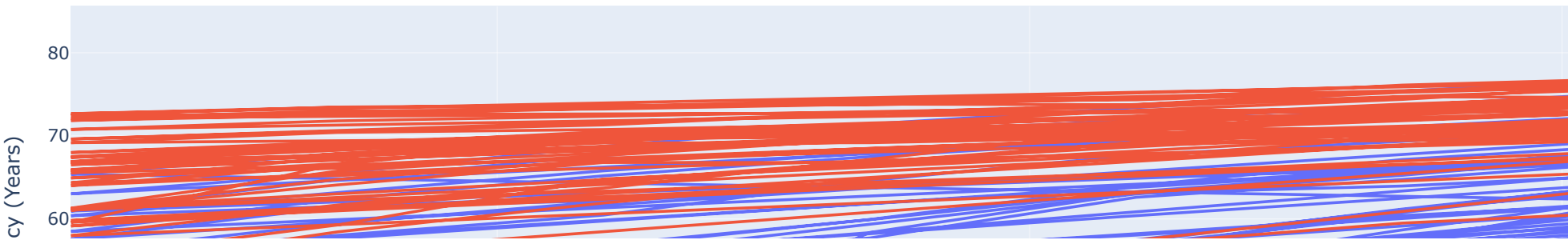
2.1. Interactive Line & Scatter Plots

These are the **bread-and-butter** of data visualization. Plotly Express makes them **interactive by default**.

The user can **zoom, pan, and hover over** data points to get detailed information.

```
In [76]: # Create an interactive line plot of Life expectancy over time for a few continents
fig_line_px = px.line(gapminder_px[gapminder_px['continent'].isin(['Asia', 'Europe'])],
                      x="year", y="lifeExp",
                      color="continent",
                      title="Life Expectancy Over Time by Continent (Interactive)",
                      labels={"lifeExp": "Life Expectancy (Years)", "year": "Year"},
                      hover_data={"country": True, "pop": ':.2s', "gdpPercap": ':.0f'})
fig_line_px.show()
```

Life Expectancy Over Time by Continent (Interactive)



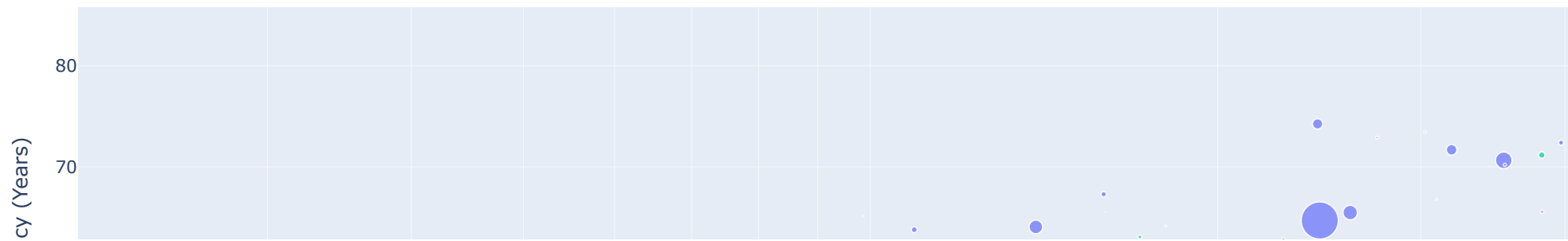
```
In [77]: # Create an interactive scatter plot of Life Expectancy vs GDP per Capita
fig_scatter_px = px.scatter(gapminder_px.query("year==2007"),
                             x="gdpPercap", y="lifeExp",
                             color="continent",
```



```
size="pop",
hover_name="country",
log_x=True,
title="Life Expectancy vs GDP per Capita (Interactive)",
labels={"gdpPercap": "GDP per Capita (Log Scale)", "lifeExp": "Life Expectancy (Years)"})

fig_scatter_px.show()
```

Life Expectancy vs GDP per Capita (Interactive)



2.2. Interactive Bar Charts

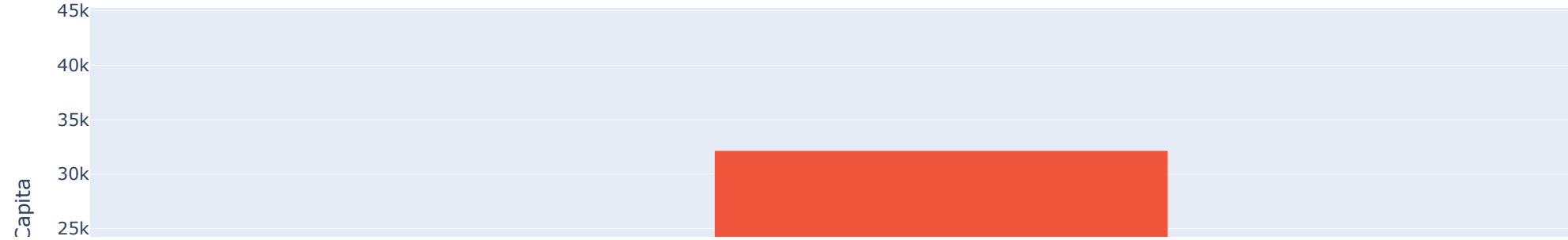
Bar charts are excellent for **comparing categorical data**. Plotly Express automatically adds interactivity to them as well.

```
In [78]: # Create an interactive bar chart of GDP per capita for a few countries in 2007
gdp_countries = ['Germany', 'United States', 'China', 'India', 'Japan']
gdp_data = gapminder_px.query("year==2007 and country.isin(@gdp_countries)")

fig_bar_px = px.bar(gdp_data,
                    x="country", y="gdpPercap",
                    color="country",
                    title="GDP per Capita by Country in 2007 (Interactive)",
                    labels={"country": "Country", "gdpPercap": "GDP per Capita"},
                    hover_data={"country": True, "gdpPercap": ':.0f', "pop": ':.2s'})

fig_bar_px.show()
```

GDP per Capita by Country in 2007 (Interactive)



2.3. 3D Plots

For data with **three or more dimensions**, `plotly.express` makes 3D plotting incredibly simple.

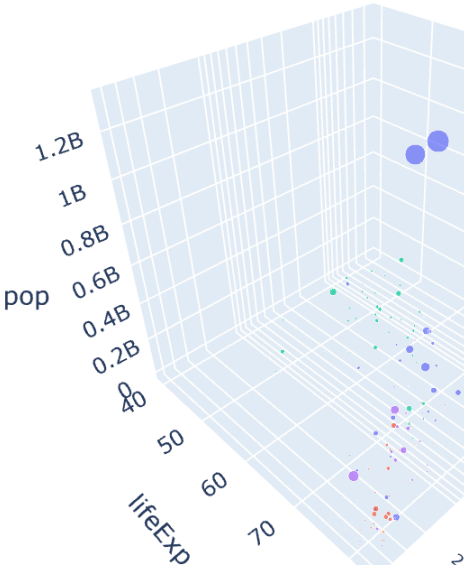
The plot below visualizes GDP, Life Expectancy, and Population, colored by continent.

You can rotate and zoom in this plot!

```
In [79]: # Existing plot :)
fig_scatter_3d_px = px.scatter_3d(gapminder_px.query("year==2007"),
                                  x="gdpPercap", y="lifeExp", z="pop",
                                  color="continent",
                                  hover_name="country",
                                  log_x=True,
                                  size="pop",
                                  title="Global Development: A 3D View (Interactive)")

fig_scatter_3d_px.show()
```

Global Development: A 3D View (Interactive)



Conclusion

Plotly Express allows you to create a wide range of interactive, aesthetically pleasing, and highly informative plots with minimal code, making it an excellent tool for both quick analysis and building **professional reports and dashboards**.

Slide 16

```
In [ ]: # Install Dash and its dependencies if you haven't already:
!pip install dash pandas plotly

In [81]: import dash
from dash import dcc
from dash import html
from dash.dependencies import Input, Output
import plotly.express as px
import pandas as pd

# Load a built-in dataset for demonstration
# The 'gapminder' dataset is excellent for demonstrating interactive filtering.
gapminder_dash = px.data.gapminder()
print("First 5 rows of the 'gapminder' dataset for Dash app:")
print(gapminder_dash.head())
```

First 5 rows of the 'gapminder' dataset for Dash app:

	country	continent	year	lifeExp	pop	gdpPercap	iso_alpha	\
0	Afghanistan	Asia	1952	28.801	8425333	779.445314	AFG	
1	Afghanistan	Asia	1957	30.332	9240934	820.853030	AFG	
2	Afghanistan	Asia	1962	31.997	10267083	853.100710	AFG	
3	Afghanistan	Asia	1967	34.020	11537966	836.197138	AFG	
4	Afghanistan	Asia	1972	36.088	13079460	739.981106	AFG	

	iso_num
0	4
1	4
2	4
3	4
4	4

```
In [82]: # Initialize the Dash app
app = dash.Dash(__name__)

# Define the layout of the app
app.layout = html.Div([
    html.H1("Life Expectancy vs GDP per Capita by Year 📈", style={'textAlign': 'center', 'color': '#333'}),
```

```
html.Div([
    dcc.Dropdown(
        id='year-dropdown',
        options=[{'label': str(year), 'value': year} for year in gapminder_dash['year'].unique()],
        value=2007, # Default selected year
        clearable=False,
        style={'width': '50%', 'margin': '0 auto', 'padding': '10px'}
    ),
], style={'display': 'flex', 'justifyContent': 'center'}), # Center the dropdown

dcc.Graph(id='life-gdp-graph', style={'height': '600px'}) # Graph component
])

# Define the callback function to update the graph based on dropdown selection
@app.callback(
    Output('life-gdp-graph', 'figure'),
    Input('year-dropdown', 'value')
)
def update_graph(selected_year):
    # Filter data for the selected year
    filtered_df = gapminder_dash[gapminder_dash.year == selected_year]

    # Create the scatter plot using Plotly Express
    fig = px.scatter(filtered_df,
                     x="gdpPercap",
                     y="lifeExp",
                     size="pop",
                     color="continent",
                     hover_name="country",
                     log_x=True,
                     title=f"Life Expectancy vs GDP per Capita in {selected_year}",
                     labels={"gdpPercap": "GDP per Capita (Log Scale)", "lifeExp": "Life Expectancy (Years)"}

    # Update layout for better aesthetics
    fig.update_layout(transition_duration=500) # Smooth transition when plot updates

    return fig

# Run the app
# In Jupyter/Colab, use app.run_server(mode='inline') for embedding directly.
# For running as a script, use app.run_server(debug=True).
if __name__ == '__main__':
    app.run(mode='inline', debug=False) # debug=False for presentation
```

INTERACTIVE MEDICAL IMAGE SLICER

Slice i Slice j Slice k Color Level Color Window

3. Building Web Apps with Dash

Dash is an open-source Python framework for **building analytical web applications**. It allows data scientists and engineers to create powerful, interactive **dashboards** and **deploy** them as web applications using only **Python**, abstracting away the complexities of web development (JavaScript, HTML, CSS). 😊

```
In [84]: import dash
from dash import dcc
from dash import html
from dash.dependencies import Input, Output
import plotly.express as px
```

```
import pandas as pd
import numpy as np # For simulating data if needed
import seaborn as sns

# Load the tips dataset for this Dash app example
tips_dash_app_df = sns.load_dataset('tips')
print("First 5 rows of the 'tips' dataset for Dash app:")
print(tips_dash_app_df.head())
```

First 5 rows of the 'tips' dataset for Dash app:

	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3
3	23.68	3.31	Male	No	Sun	Dinner	2
4	24.59	3.61	Female	No	Sun	Dinner	4

3.1. Interactive Dashboard Example: Tips Analysis 🧑🏻💻

This Dash app demonstrates how to create a multi-plot, interactive dashboard.

It will include a scatter plot and a bar chart, both reacting to user selections.

```
In [88]: # Initialize the Dash app
app_dash_example = dash.Dash(__name__)

# Define the layout of the app
app_dash_example.layout = html.Div([
    html.H1("Restaurant Tips Analysis Dashboard", style={'textAlign': 'center', 'color': '#2c3e50', 'fontFamily': 'Arial, sans-serif'}),

    html.Div([
        html.Label("Select Day of Week:", style={'fontWeight': 'bold', 'marginRight': '10px'}),
        dcc.Dropdown(
            id='day-dropdown',
            options=[{'label': day, 'value': day} for day in tips_dash_app_df['day'].unique()],
            value='Thur', # Default value
            clearable=False,
            style={'width': '200px'}
        ),
        html.Label("Smoker Status:", style={'fontWeight': 'bold', 'marginLeft': '20px', 'marginRight': '10px'}),
        dcc.RadioItems(
            id='smoker-radio',
            options=[
                {'label': 'All', 'value': 'All'},
                {'label': 'Yes', 'value': 'Yes'},
                {'label': 'No', 'value': 'No'}
            ],
            value='All', # Default value
            inline=True,
            style={'marginTop': '5px'}
        )
    ], style={'display': 'flex', 'justifyContent': 'center', 'alignItems': 'center', 'marginBottom': '30px', 'flexWrap': 'wrap'}),

    html.Div([
        dcc.Graph(id='scatter-plot-dash', style={'width': '50%', 'display': 'inline-block', 'padding': '10px'}),
        dcc.Graph(id='bar-chart-dash', style={'width': '50%', 'display': 'inline-block', 'padding': '10px'})
    ], style={'display': 'flex', 'flexWrap': 'wrap'}) # Use flex to arrange graphs side-by-side
])

# Define callbacks to update plots based on dropdown and radio button selections
@app_dash_example.callback(
    Output('scatter-plot-dash', 'figure'),
    Output('bar-chart-dash', 'figure'),
    Input('day-dropdown', 'value'),
    Input('smoker-radio', 'value')
)
def update_dashboard_plots(selected_day, selected_smoker):
    filtered_df = tips_dash_app_df[tips_dash_app_df.day == selected_day]

    if selected_smoker != 'All':
        filtered_df = filtered_df[filtered_df.smoker == selected_smoker]

    # Scatter plot: Total Bill vs Tip
    scatter_fig = px.scatter(filtered_df,
                             x="total_bill", y="tip",
                             color="time", size="size",
                             hover_name="sex",
                             title=f"Total Bill vs Tip on {selected_day} (Smoker: {selected_smoker})",
                             labels={"total_bill": "Total Bill ($)", "tip": "Tip ($)"})
    scatter_fig.update_layout(transition_duration=500)

    # Bar chart: Average Tip by Time (Lunch/Dinner)
    avg_tip_by_time = filtered_df.groupby('time', observed=False)['tip'].mean().reset_index()
    bar_fig = px.bar(avg_tip_by_time,
                     x="time", y="tip",
                     color="time",
                     title=f"Average Tip by Time on {selected_day} (Smoker: {selected_smoker})",
                     labels={"time": "Time of Day", "tip": "Average Tip ($)"})
    bar_fig.update_layout(transition_duration=500)

    return scatter_fig, bar_fig

# Run the app
if __name__ == '__main__':
    app_dash_example.run(mode='inline', debug=False) # Use app.run() for newer Dash versions:)
```

Interactive Medical Image Slicer



How to Interact with this Dash App ? :

Dropdown Menu: Select a `Day of Week` to filter the data for both plots.

Radio Buttons: Choose `Smoker Status` (All, Yes, No) to further filter the data.

Linked Plots: Observe how both the scatter plot and the bar chart update dynamically based on your selections.

Plotly Interactivity: All standard Plotly features (zoom, pan, hover) are available on both graphs.

This example showcases how Dash can be used to build interactive, multi-component dashboards purely in Python, allowing users to explore data insights in a dynamic web environment.

Now let's see some interesting instances 🤩👉

```
In [ ]: # Install necessary libraries if not already installed
!pip install dash dash-vtk itk dash-bootstrap-components
```

```
In [91]: # if you didn't have vtk, so clone it for first time by excuting this cell.

# !git clone https://github.com/plotly/dash-vtk.git
```

Isntead of running it here in the following cell (locally), run it through `mri_vtk.py`

```
In [ ]: import os
import dash
from dash import html # Correct import for html components
import dash_bootstrap_components as dbc # Imported for potential future use, or remove if not needed.
import itk # ITK is used for reading DICOM files

import dash_vtk
from dash_vtk.utils import to_volume_state

import vtk # VTK is used by ITK for visualization pipeline conversion

def dcm_to_volume(dir_path):
    """
    Converts a directory containing DICOM slices into a VTK volume state
    compatible with dash-vtk.
    """
    # Read the DICOM series from the specified directory using ITK
    # itk.imread automatically handles sorting slices and metadata.
    itk_image = itk.imread(dir_path)

    # Convert the ITK image to a VTK image data object.
```



```

# This is necessary because dash-vtk's to_volume_state expects a VTK image.
vtk_image = itk.vtk_image_from_image(itk_image)

# Convert the VTK image data to a volume state object for dash-vtk.
volume_state = to_volume_state(vtk_image)

return volume_state

# --- Data Loading and Volume State Creation ---
# The original code used __file__ which is not defined in Jupyter/Colab.

# Define the exact path to the MRI brain DICOM directory as provided by the user.
# This assumes the 'dash-vtk' repository has been cloned and is located at /content/dash-vtk.
mri_data_dir = "dash-vtk/demos/data/mri_brain"

# Check if the directory exists before attempting to load
if not os.path.exists(mri_data_dir):
    print(f"Error: MRI data directory not found at {mri_data_dir}")
    print("Please ensure you have cloned the 'dash-vtk' repository and the 'dicom-mri-brain' data is present.")
    print("Run '!git clone https://github.com/plotly/dash-vtk.git' in a previous cell.")
    # Create a dummy volume_state to prevent app crash if data is missing
    # For a real app, you might want to display an error message in the Dash layout.
    reader = vtk.vtkConeSource() # Use a simple cone as fallback
    reader.SetResolution(64)
    reader.Update()
    volume_state = to_volume_state(reader.GetOutput())
else:
    # Convert the DICOM series to a volume state
    volume_state = dcm_to_volume(mri_data_dir)

# --- Dash Application Setup ---

# Initialize the Dash application.
app = dash.Dash(__name__)
server = app.server # This line is typically used for Gunicorn deployment, but harmless in Jupyter.

# Define the Dash VTK view component.
# This sets up a 3D rendering view with a VolumeRepresentation and VolumeController.
vtk_view = dash_vtk.View(
    dash_vtk.VolumeRepresentation(
        children=[
            # VolumeController allows interactive adjustment of volume rendering properties (e.g., opacity, colors).
            dash_vtk.VolumeController(),
            # Volume component renders the 3D medical data.
            dash_vtk.Volume(state=volume_state),
        ]
    )
)

# Define the layout of the Dash application.
app.layout = html.Div(
    style={"height": "calc(100vh - 50px)", "width": "100%", "backgroundColor": "#f0f0f0"},
    children=[
        html.Div(
            vtk_view,
            style={"height": "100%", "width": "100%", "display": "flex", "justifyContent": "center", "alignItems": "center"}
        )
    ],
)

# Run the Dash application server.
# For Jupyter/Colab, 'mode='inline'' embeds the app output directly in the notebook.
# 'debug=True' enables hot-reloading and debug tools (set to False for production).
if __name__ == "__main__":
    app.run(debug=False, mode='inline') # Corrected to app.run() for newer Dash versions

```

Isntead of running it here in the following cell (locally), run it through
lung_ct_vtk.py

```

In [ ]: # Install necessary libraries if not already installed
# !pip install dash dash-vtk itk dash-bootstrap-components

import os
import dash
from dash import html # Correct import for html components
# dash_bootstrap_components is not directly used in this example, can be removed if not needed.
# import dash_bootstrap_components as dbc
import itk # ITK is used for reading DICOM files

import dash_vtk
from dash_vtk.utils import to_volume_state

import vtk # VTK is used by ITK for visualization pipeline conversion

def dcm_to_volume(dir_path):
    """
    Converts a directory containing DICOM slices into a VTK volume state
    compatible with dash-vtk.
    """
    # Read the DICOM series from the specified directory using ITK
    # itk.imread automatically handles sorting slices and spatial metadata.
    itk_image = itk.imread(dir_path)

    # Convert the ITK image to a VTK image data object.

```

```

# This is necessary because dash-vtk's to_volume_state expects a VTK image.
vtk_image = itk.vtk_image_from_image(itk_image)

# Convert the VTK image data to a volume state object for dash-vtk.
volume_state = to_volume_state(vtk_image)

return volume_state

# --- Data Loading and Volume State Creation ---
# The original code used __file__ which is not defined in Jupyter/Colab.
# Define the exact path to the CT Lung DICOM directory.
# This assumes the 'dash-vtk' repository has been cloned and is located at /content/dash-vtk.
ct_lung_data_dir = "/dash-vtk/demos/data/ct_lung"

# Check if the directory exists before attempting to load
if not os.path.exists(ct_lung_data_dir):
    print(f"Error: CT lung data directory not found at {ct_lung_data_dir}")
    print("Please ensure you have cloned the 'dash-vtk' repository and the 'ct_lung' data is present.")
    print("Run '!git clone https://github.com/plotly/dash-vtk.git' in a previous cell.")
    # Create a dummy volume_state to prevent app crash if data is missing
    # For a real app, you might want to display an error message in the Dash layout.
    reader = vtk.vtkConeSource() # Use a simple cone as fallback
    reader.SetResolution(64)
    reader.Update()
    volume_state = to_volume_state(reader.GetOutput())
else:
    # Convert the DICOM series to a volume state
    volume_state = dcm_to_volume(ct_lung_data_dir)

# --- Dash Application Setup ---

# Initialize the Dash application.
app = dash.Dash(__name__)
server = app.server # This line is typically used for Gunicorn deployment, but harmless in Jupyter.

# Define the Dash VTK view component.
# This sets up a 3D rendering view with a VolumeRepresentation and VolumeController.
vtk_view = dash_vtk.View(
    dash_vtk.VolumeRepresentation(
        children=[
            # VolumeController allows interactive adjustment of volume rendering properties (e.g., opacity, colors).
            dash_vtk.VolumeController(),
            # Volume component renders the 3D medical data.
            dash_vtk.Volume(state=volume_state),
        ]
    )
)

# Define the layout of the Dash application.
app.layout = html.Div(
    style={"height": "calc(100vh - 50px)", "width": "100%", "backgroundColor": "#f0f0f0"},
    children=[
        html.Div(
            vtk_view,
            style={"height": "100%", "width": "100%", "display": "flex", "justifyContent": "center", "alignItems": "center"}
        )
    ],
)

# Run the Dash application server.
# For Jupyter/Colab, 'mode='inline' embeds the app output directly in the notebook.
# 'debug=True' enables hot-reloading and debug tools (set to False for production).
if __name__ == "__main__":
    app.run(debug=False, mode='inline') # Corrected to app.run() for newer Dash versions

```

Isntead of running it here in the following cell (locally), run it through `mri_pancreas_vtk.py`

```

In [ ]: # Install necessary libraries if not already installed
# !pip install dash dash-vtk itk dash-bootstrap-components

import os
import dash
from dash import html # Correct import for html components
# dash_bootstrap_components is not directly used in this example, can be removed if not needed.
# import dash_bootstrap_components as dbc
import itk # ITK is used for reading DICOM files

import dash_vtk
from dash_vtk.utils import to_volume_state

import vtk # VTK is used by ITK for visualization pipeline conversion

def dcm_to_volume(dir_path):
    """
    Converts a directory containing DICOM slices into a VTK volume state
    compatible with dash-vtk.
    """
    # Read the DICOM series from the specified directory using ITK
    # itk.imread automatically handles sorting slices and spatial metadata.
    itk_image = itk.imread(dir_path)

    # Convert the ITK image to a VTK image data object.
    # This is necessary because dash-vtk's to_volume_state expects a VTK image.

```

```

    vtk_image = itk.vtk_image_from_image(itk_image)

    # Convert the VTK image data to a volume state object for dash-vtk.
    volume_state = to_volume_state(vtk_image)

    return volume_state

# --- Data Loading and Volume State Creation ---
# The original code used __file__ which is not defined in Jupyter/Colab.
# Define the exact path to the MRI pancreas DICOM directory.
# This assumes the 'dash-vtk' repository has been cloned and is located at /content/dash-vtk.
volume_data_dir = "dash-vtk/demos/data/mri_pancreas" # Updated path to MRI pancreas data

# Check if the directory exists before attempting to load
if not os.path.exists(volume_data_dir):
    print(f"Error: Volume data directory not found at {volume_data_dir}")
    print("Please ensure you have cloned the 'dash-vtk' repository and the 'mri_pancreas' data is present.")
    print("Run '!git clone https://github.com/plotly/dash-vtk.git' in a previous cell.")
    # Create a dummy volume_state to prevent app crash if data is missing
    # For a real app, you might want to display an error message in the Dash layout.
    reader = vtk.vtkCubeSource() # Use a simple cube as fallback for variety
    reader.SetResolution(64)
    reader.Update()
    volume_state = to_volume_state(reader.GetOutput())
else:
    # Convert the DICOM series to a volume state
    volume_state = dcm_to_volume(volume_data_dir)

# --- Dash Application Setup ---

# Initialize the Dash application.
app = dash.Dash(__name__)
server = app.server # This line is typically used for Gunicorn deployment, but harmless in Jupyter.

# Define the Dash VTK view component.
# This sets up a 3D rendering view with a VolumeRepresentation and VolumeController.
vtk_view = dash_vtk.View(
    dash_vtk.VolumeRepresentation(
        children=[
            # VolumeController allows interactive adjustment of volume rendering properties (e.g., opacity, colors).
            dash_vtk.VolumeController(),
            # Volume component renders the 3D medical data.
            dash_vtk.Volume(state=volume_state),
        ]
    )
)

# Define the layout of the Dash application.
app.layout = html.Div(
    style={"height": "calc(100vh - 50px)", "width": "100%", "backgroundColor": "#f0f0f0"},
    children=[
        html.Div(
            vtk_view,
            style={"height": "100%", "width": "100%", "display": "flex", "justifyContent": "center", "alignItems": "center"}
        ),
    ],
)

# Run the Dash application server.
# For Jupyter/Colab, 'mode='inline' embeds the app output directly in the notebook.
# 'debug=True' enables hot-reloading and debug tools (set to False for production).
if __name__ == "__main__":
    app.run(debug=False, mode='inline') # Corrected to app.run() for newer Dash versions

```

```
In [ ]: !pip install pyvista
```

Isntead of running it here in the following cell (locally), run it through `Lidar_vtk.py`

```

In [ ]: # Install necessary libraries if not already installed
# !pip install dash dash-vtk pyvista

import os
import dash
from dash import html # Correct import for html components
from dash import dcc # Correct import for core components (though not strictly used in this layout, good practice)
# dash_bootstrap_components is imported in the original snippet but not used in the layout.
# If you intend to use Bootstrap components, ensure it's installed and used.
# import dash_bootstrap_components as dbc
from dash.dependencies import Input, Output, State # Standard Dash imports for callbacks

import numpy as np
import pyvista as pv # PyVista for 3D data handling
from pyvista import examples # PyVista examples for data download

import dash_vtk # Dash VTK components
from dash_vtk.utils import to_volume_state # Not used in this specific point cloud example, but kept if part of larger context

np.random.seed(42) # Set seed for reproducibility of random selection

# Get point cloud data from PyVista examples
# pyvista.examples.download_lidar() downloads a sample LiDAR point cloud.
dataset = examples.download_lidar()
subset_ratio = 0.2 # Define the ratio for subsetting the points
selection_indices = np.random.randint(

```

```

        low=0, high=dataset.n_points - 1, size=int(dataset.n_points * subset_ratio)
    )
    # Select a random subset of points from the dataset
    points = dataset.points[selection_indices]
    xyz = points.ravel() # Flatten the array of points (x, y, z coordinates)
    elevation = points[:, -1].ravel() # Get the Z-coordinate (elevation) for scalar mapping
    min_elevation = np.amin(elevation) # Calculate min elevation for color mapping
    max_elevation = np.amax(elevation) # Calculate max elevation for color mapping
    print(f"Number of points: {points.shape[0]}") # Print number of selected points
    print(f"Elevation range: [{min_elevation:.2f}, {max_elevation:.2f}]") # Print elevation range

# --- Dash Application Setup ---

# Initialize the Dash application.
app = dash.Dash(__name__)
server = app.server # This line is typically used for Gunicorn deployment, but harmless in Jupyter.

# Define the Dash VTK view component.
# This sets up a 3D rendering view specifically for a PointCloudRepresentation.
vtk_view = dash_vtk.View(
    [
        dash_vtk.PointCloudRepresentation(
            xyz=xyz, # Pass the flattened XYZ coordinates
            scalars=elevation, # Pass the elevation values as scalars for coloring
            colorDataRange=[min_elevation, max_elevation], # Define the color range for scalars
            property={"pointSize": 2}, # Set the size of the rendered points
        )
    ]
)

# Define the layout of the Dash application.
app.layout = html.Div(
    style={"height": "calc(100vh - 16px)", "width": "100%", "backgroundColor": "#f0f0f0"}, # Added background for clarity
    children=[html.Div(vtk_view, style={"height": "100%", "width": "100%"})], # Main div containing the VTK view
)

# Run the Dash application server.
# For Jupyter/Colab, 'mode='inline' embeds the app output directly in the notebook.
# 'debug=True' enables hot-reloading and debug tools (set to False for production).
if __name__ == "__main__":
    app.run(debug=False, mode='inline') # Corrected to app.run() for newer Dash versions

```

Isntead of running it here in the following cell (locally), run it through
crater_topo.py

```

In [ ]: import random
import json
import numpy as np
import pyvista as pv
from pyvista import examples
from vtk.util.numpy_support import import vtk_to_numpy

import dash
from dash import html # Correct import for html components
from dash import dcc # Correct import for core components
import dash_bootstrap_components as dbc # Dash Bootstrap Components for Layout
from dash.dependencies import Input, Output, State

import dash_vtk
from dash_vtk.utils import presets # Presets for colormaps

random.seed(42) # Set random seed for reproducibility

def toDropOption(name):
    """Helper function to format options for Dash Dropdown."""
    return {"label": name, "value": name}

# Get point cloud data from PyVista
# examples.download_crater_topo() downloads a topographical dataset of a crater.
uniformGrid = examples.download_crater_topo()
# Extract a subset of the grid for more manageable visualization.
subset = uniformGrid.extract_subset((500, 900, 400, 800, 0, 0), (5, 5, 1))

def updateWarp(factor=1):
    """
    Warps the terrain by a scalar factor (elevation) and extracts PolyData.
    This simulates terrain deformation based on a scale factor.
    """
    # Warp the terrain geometry based on its scalar (elevation) data.
    terrain = subset.warp_by_scalar(factor=factor)
    # Extract the polygonal data (surface mesh) from the warped terrain.
    polydata = terrain.extract_geometry()
    # Flatten the points array (x, y, z coordinates) for dash-vtk.
    points = polydata.points.ravel()
    # Get the polygonal connectivity (faces) for dash-vtk.
    polys = vtk_to_numpy(polydata.GetPolys().GetData())
    # Get the elevation data (scalars) associated with the points.
    elevation = polydata["scalar1of1"]
    # Calculate the min and max elevation for color mapping.
    min_elevation = np.amin(elevation)
    max_elevation = np.amax(elevation)
    return [points, polys, elevation, [min_elevation, max_elevation]]

```

```

# Initial warp of the terrain with a factor of 1.
points, polys, elevation, color_range = updateWarp(1)

# --- Dash Application Setup ---

# Initialize the Dash application with Bootstrap external stylesheets for styling.
app = dash.Dash(__name__, external_stylesheets=[dbc.themes.BOOTSTRAP])
server = app.server # Standard way to expose the Flask server for deployment.

# Define the initial Dash VTK view component.
# This sets up a 3D rendering view for a GeometryRepresentation (our terrain mesh).
vtk_view = dash_vtk.View(
    id="vtk-view", # ID for the view, used in callbacks
    pickingModes=["hover"], # Enable hover picking for tooltip
    children=[
        dash_vtk.GeometryRepresentation(
            id="vtk-representation", # ID for the geometry representation
            children=[
                dash_vtk.PolyData(
                    id="vtk-polydata", # ID for the PolyData component
                    points=points, # Initial points data
                    polys=polys, # Initial polygonal connectivity
                    children=[
                        dash_vtk.PointData( # Data associated with points
                            [
                                dash_vtk.DataArray(
                                    id="vtk-array", # ID for the DataArray
                                    registration="setScalars", # Register as scalars for coloring
                                    name="elevation", # Name of the scalar array
                                    values=elevation, # Initial elevation values
                                )
                            ]
                        )
                    ]
                ),
            ],
        ),
        colorMapPreset="erdc_blue2green_muted", # Initial colormap preset
        colorDataRange=color_range, # Initial color data range
        property={"edgeVisibility": True}, # Show edges of the mesh
        showCubeAxes=True, # Show cube axes around the model
        cubeAxesStyle={"axisLabels": ["", "", "Altitude"]}, # Label the Z-axis as Altitude
    ),
    # A separate GeometryRepresentation for displaying the picked point (a sphere).
    dash_vtk.GeometryRepresentation(
        id="pick-rep",
        actor={"visibility": False}, # Initially hidden
        children=[
            dash_vtk.Algorithm(
                id="pick-sphere", # Sphere source for the picked point
                vtkClass="vtkSphereSource",
                state={"radius": 100}, # Initial radius of the sphere
            )
        ],
    ),
],
)

# Define the Layout of the Dash application using Dash Bootstrap Components.
app.layout = dbc.Container(
    fluid=True, # Container takes full width
    style={"height": "100vh", "display": "flex", "flexDirection": "column"}, # Full viewport height, column layout
    children=[
        dbc.Row( # Row for controls (slider, dropdown, checklist)
            [
                dbc.Col( # Column for the scale factor slider
                    children=dcc.Slider(
                        id="scale-factor",
                        min=0.1, max=5, step=0.1, value=1,
                        marks={0.1: "0.1", 5: "5"},
                        className="p-2" # Add some padding
                    ),
                    width=4 # Occupy 4 columns of the 12-column grid
                ),
                dbc.Col( # Column for the colormap preset dropdown
                    children=dcc.Dropdown(
                        id="dropdown-preset",
                        options=list(map(toDropOption, presets)), # Options from dash_vtk presets
                        value="erdc_rainbow_bright", # Default colormap
                        className="p-2"
                    ),
                    width=4
                ),
                dbc.Col( # Column for the cube axes toggle checklist
                    children=dcc.Checklist(
                        id="toggle-cube-axes",
                        options=[{"label": " Show axis grid", "value": "grid"}],
                        value=[], # Initially unchecked
                        labelStyle={"display": "inline-block", "marginLeft": "10px"},
                        className="p-2"
                    ),
                    width=4
                ),
            ],
            style={"height": "12%", "alignItems": "center"}, # Align items vertically in the middle
            className="mb-3" # Margin bottom
        ),
        html.Div( # Div for the main VTK viewer
            vtk_view,
            style={"flexGrow": 1, "width": "100%", "height": "88%"} # Takes remaining height
        )
    ]
)

```



```

    ),
    html.Pre( # Tooltip for picked points
        id="tooltip",
        style={
            "position": "absolute",
            "bottom": "25px",
            "left": "25px",
            "zIndex": 1,
            "color": "white",
            "backgroundColor": "rgba(0,0,0,0.7)",
            "padding": "5px",
            "borderRadius": "5px",
            "pointerEvents": "none", # Ensures tooltip doesn't block interactions
            "display": "none" # Hidden by default
        },
    ),
],
)

# --- Dash Callbacks ---

@app.callback(
    [
        Output("vtk-representation", "showCubeAxes"),
        Output("vtk-representation", "colorMapPreset"),
        Output("vtk-representation", "colorDataRange"),
        Output("vtk-polydata", "points"),
        Output("vtk-polydata", "polys"),
        Output("vtk-array", "values"),
        Output("vtk-view", "triggerResetCamera"), # Trigger camera reset on update
    ],
    [
        Input("dropdown-preset", "value"),
        Input("scale-factor", "value"),
        Input("toggle-cube-axes", "value"),
    ],
)
def updatePresetName(name, scale_factor, cubeAxes):
    """Callback to update terrain based on slider and dropdown selections."""
    # Re-warp the terrain based on the new scale factor
    points, polys, elevation, color_range = updateWarp(scale_factor)

    # Return updated properties for the VTK components
    return [
        "grid" in cubeAxes, # showCubeAxes is true if 'grid' is in the checklist value
        name, # New colormap preset
        color_range, # New color data range
        points, # Updated points data
        polys, # Updated polygons data
        elevation, # Updated elevation values
        random.random(), # Trigger a camera reset by providing a new random value
    ]

@app.callback(
    [
        Output("tooltip", "children"),
        Output("tooltip", "style"), # Update style to control visibility
        Output("pick-sphere", "state"), # Update sphere position
        Output("pick-rep", "actor"), # Update sphere visibility
    ],
    [
        Input("vtk-view", "clickInfo"), # Listen for click events
        Input("vtk-view", "hoverInfo"), # Listen for hover events
    ],
    [
        State("tooltip", "style") # Get current style to toggle visibility
    ],
)
def onInfo(clickData, hoverData, current_style):
    """Callback to display tooltip and sphere on hover/click."""
    info = hoverData if hoverData else clickData # Prefer hover data if available

    if info:
        # Check if the picked element is our terrain representation
        if (
            "representationId" in info
            and info["representationId"] == "vtk-representation"
        ):
            # Display detailed info in the tooltip
            tooltip_text = json.dumps(info, indent=2)

            # Update sphere position to the picked world position
            sphere_state = {"center": info["worldPosition"]}

            # Make sphere visible
            actor_visibility = {"visibility": True}

            # Make tooltip visible
            new_style = {**current_style, "display": "block"}

            return tooltip_text, new_style, sphere_state, actor_visibility

        # If picked something else or no valid pick, hide tooltip and sphere
        return "", {**current_style, "display": "none"}, {}, {"visibility": False}

    # If no hover or click data, hide tooltip and sphere
    return "", {**current_style, "display": "none"}, {}, {"visibility": False}

```

```
# Run the Dash application server.
# For Jupyter/Colab, 'mode='inline'' embeds the app output directly in the notebook.
# 'debug=True' enables hot-reloading and debug tools (set to False for production).
if __name__ == "__main__":
    app.run(debug=False, mode='inline') # Corrected to app.run() for newer Dash versions
```

```
In [4]: import os
import random

import dash
from dash import html # Correct import for html components
from dash import dcc, Input, Output # Correct import for core components
import dash_bootstrap_components as dbc # Dash Bootstrap Components for layout
import itk # ITK for reading medical image files (like VTI)

import dash_vtk
from dash_vtk.utils import to_volume_state # Utility to convert ITK/VTK images to dash-vtk state

import vtk # VTK library, often used by ITK internally for image processing

random.seed(42) # Set random seed for reproducibility

# --- Data Loading and Volume State Creation ---
# The original code used __file__ which is not defined in Jupyter/Colab.
# We assume 'head.vti' has been downloaded to the current working directory.
head_vti_path = "dash-vtk/demos/data/head.vti"

# Load the VTI dataset using VTK's reader
# vtk.vtkXMLImageDataReader is used to read the .vti file.
reader = vtk.vtkXMLImageDataReader()
reader.SetFileName(head_vti_path)
reader.Update() # Execute the reader to load the data

# Convert the VTK dataset to a state object compatible with dash-vtk.
volume_state = to_volume_state(reader.GetOutput())

# Define sliders for interactive control of slicing and window/level.
# These sliders control the i, j, k slices and the color window/level for visualization.
sliders = {
    "Slice i": dcc.Slider(id="slider-i", min=0, max=256, value=128, step=1),
    "Slice j": dcc.Slider(id="slider-j", min=0, max=256, value=128, step=1),
    "Slice k": dcc.Slider(id="slider-k", min=0, max=95, value=47, step=1), # Max value adjusted based on typical head.vti dimensions
    "Color Level": dcc.Slider(id="slider-lvl", min=0, max=4095, value=1000, step=1),
    "Color Window": dcc.Slider(id="slider-window", min=0, max=4095, value=4095, step=1),
}

# Create a Dash Bootstrap Card to group the slider controls.
controls = dbc.Card(
    body=True,
    children=dbc.Row(
        [
            # Create a column for each slider with its label.
            dbc.Col(
                [dbc.Label(label), component],
                width="auto", # Adjust width automatically
                className="d-flex flex-column align-items-center" # Center content in column
            )
            for label, component in sliders.items()
        ],
        justify="around", # Distribute columns evenly
        className="g-0" # Remove gutter between columns
    ),
    className="mb-3" # Margin bottom for the card
)

# Define the Dash VTK View for displaying the slices.
# This view is configured for 2D slicing (camera position, view up, parallel projection).
slice_view = dash_vtk.View(
    id="slice-view",
    # Camera settings for an axial view (looking down Z-axis)
    cameraPosition=[0, 0, 1], # Looking from above
    cameraViewUp=[0, -1, 0], # Y-axis points down on screen
    cameraParallelProjection=True, # Use parallel projection for 2D slices
    background=[0.9, 0.9, 1], # Light blue background
    children=[
        # ShareDataSet allows multiple representations to use the same underlying volume data.
        dash_vtk.ShareDataSet(dash_vtk.Volume(state=volume_state)),
        # SliceRepresentation for the I-slice (sagittal)
        dash_vtk.SliceRepresentation(
            id="slice-repr-i",
            iSlice=128, # Initial slice index
            property={"colorWindow": 4095, "colorLevel": 1000}, # Initial window/level
            children=dash_vtk.ShareDataSet(), # Shares data from parent ShareDataSet
        ),
        # SliceRepresentation for the J-slice (coronal)
        dash_vtk.SliceRepresentation(
            id="slice-repr-j",
            jSlice=128, # Initial slice index
            property={"colorWindow": 4095, "colorLevel": 1000},
            children=dash_vtk.ShareDataSet(),
        ),
        # SliceRepresentation for the K-slice (axial)
        dash_vtk.SliceRepresentation(
            id="slice-repr-k",
            kSlice=47, # Initial slice index
            property={"colorWindow": 4095, "colorLevel": 1000},
            children=dash_vtk.ShareDataSet(),
        ),
    ],
)
```

```
)

# Initialize the Dash application with Bootstrap external stylesheets.
app = dash.Dash(__name__, external_stylesheets=[dbc.themes.BOOTSTRAP])
server = app.server # Standard way to expose the Flask server for deployment.

# Define the main layout of the Dash application.
app.layout = dbc.Container(
    fluid=True, # Container takes full width
    style={"height": "100vh", "display": "flex", "flexDirection": "column"}, # Full viewport height, column layout
    children=[
        html.Div( # Div for header and controls
            style={"height": "20%", "display": "flex", "flexDirection": "column", "alignItems": "center", "justifyContent": "center"},
            children=[
                html.H2("Interactive Medical Image Slicer", className="mb-3 text-center text-gray-800"),
                controls, # The Card containing sliders
            ],
        ),
        html.Div( # Div for the main VTK slice viewer
            slice_view,
            style={"height": "80%", "width": "100%", "flexGrow": 1} # Takes remaining height
        ),
    ],
)

# --- Dash Callbacks ---

@app.callback(
    [
        Output("slice-view", "triggerRender"), # Trigger a re-render of the VTK view
        Output("slice-repr-i", "property"), # Update properties for I-slice
        Output("slice-repr-i", "iSlice"), # Update I-slice index
        Output("slice-repr-j", "property"), # Update properties for J-slice
        Output("slice-repr-j", "jSlice"), # Update J-slice index
        Output("slice-repr-k", "property"), # Update properties for K-slice
        Output("slice-repr-k", "kSlice"), # Update K-slice index
    ],
    [
        Input("slider-i", "value"), # Input from Slice i slider
        Input("slider-j", "value"), # Input from Slice j slider
        Input("slider-k", "value"), # Input from Slice k slider
        Input("slider-lvl", "value"), # Input from Color Level slider
        Input("slider-window", "value"), # Input from Color Window slider
    ],
)
def update_slice_property(i, j, k, level, window):
    """
    Callback to update slice positions and color window/level based on slider inputs.
    A random value is returned for triggerRender to force the VTK view to update.
    """
    render_call = random.random() # Generate a random number to trigger re-render
    slice_prop = {"colorLevel": level, "colorWindow": window} # Create property dict for slices
    return render_call, slice_prop, i, slice_prop, j, slice_prop, k

# Run the Dash application server.
# For Jupyter/Colab, 'mode='inline'' embeds the app output directly in the notebook.
# 'debug=True' enables hot-reloading and debug tools (set to False for production).
if __name__ == "__main__":
    app.run(debug=False, mode='inline', port=8051) # Corrected to app.run() for newer Dash versions
```

Interactive Medical Image Slicer

Slice i

Slice j

Slice k

Color Level

Color Window



```
In [6]: import random

import dash
from dash import html # Correct import for html components
# dash_core_components and dash_bootstrap_components are not used in this specific example.
# from dash import dcc
# import dash_bootstrap_components as dbc
# from dash.dependencies import Input, Output, State # Not used in this specific example

import dash_vtk
from dash_vtk.utils import to_volume_state # Not directly used for synthetic data, but often part of VTK workflows

# vtk is not explicitly imported in the provided snippet but might be implicitly needed by dash_vtk or itk.
# import vtk

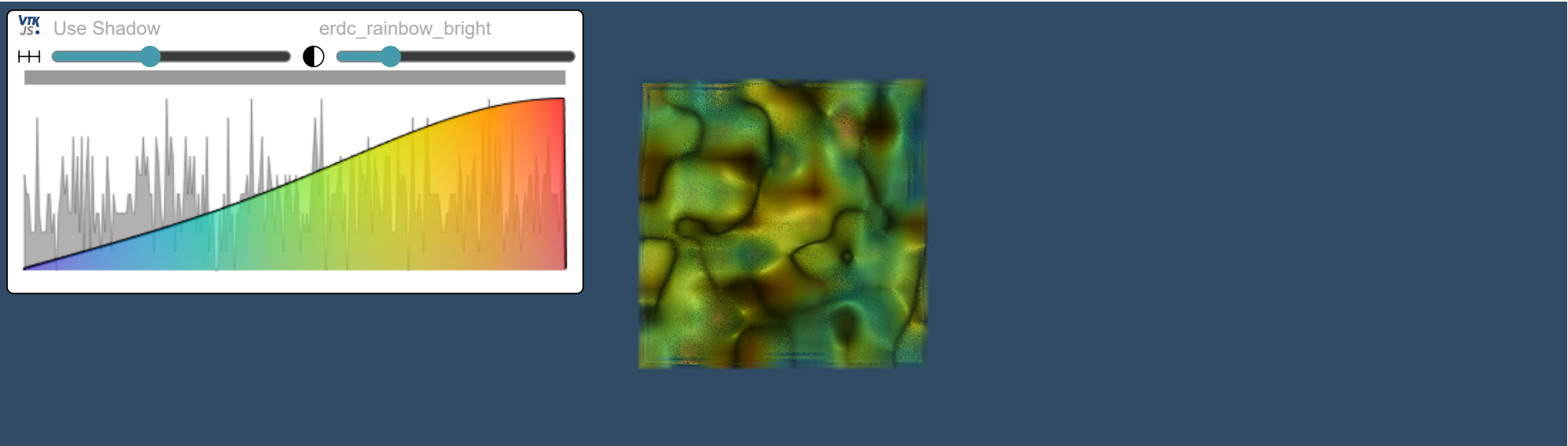
random.seed(42) # Set random seed for reproducibility

# Initialize the Dash application.
app = dash.Dash(__name__)
server = app.server # Standard way to expose the Flask server for deployment.

# Define the Dash VTK View component for displaying a synthetic volume.
volume_view = dash_vtk.View(
    children=dash_vtk.VolumeDataRepresentation(
        # Define the spacing between voxels in X, Y, Z directions.
        spacing=[1, 1, 1],
        # Define the dimensions (resolution) of the 3D volume data.
        # 10x10x10 = 1000 voxels, matching the length of the scalars array.
        dimensions=[10, 10, 10],
        # Define the origin of the volume in 3D space.
        origin=[0, 0, 0],
        # Generate random scalar values for each voxel in the volume.
        # These values will be used for volume rendering (e.g., opacity and color mapping).
        scalars=[random.random() for _ in range(1000)],
        # If set to False, the color map will not be rescaled to the min/max of the current scalars.
        # This means the color mapping might be fixed based on a global range (0-1 for random.random()).
        rescaleColorMap=False,
    )
)

# Define the Layout of the Dash application.
app.layout = html.Div(
    style={"height": "calc(100vh - 16px)", "width": "100%", "backgroundColor": "#f0f0f0"}, # Added background for clarity
    children=[
        html.Div(
            children=volume_view, # Embed the VTK volume view
            style={"height": "100%", "width": "100%", "display": "flex", "justifyContent": "center", "alignItems": "center"} # Center the VTK view
        ),
    ],
)

# Run the Dash application server.
# For Jupyter/Colab, 'mode='inline'' embeds the app output directly in the notebook.
# 'debug=True' enables hot-reloading and debug tools (set to False for production).
if __name__ == "__main__":
    app.run(debug=False, mode='inline', port=8052)
```



```
In [8]: import random

import dash
from dash import html # Correct import for html components
```

```

from dash import dcc # Correct import for core components
import dash_bootstrap_components as dbc # Dash Bootstrap Components for Layout
from dash.dependencies import Input, Output, State # Standard Dash imports for callbacks

import dash_vtk # Dash VTK components

# vtk is not explicitly imported in the provided snippet as vtkConeSource is a vtk.js class
# represented by dash_vtk.Algorithm, but it's good to be aware it's VTK-related.
# import vtk

random.seed(42) # Set random seed for reproducibility

# Initialize the Dash application with Bootstrap external stylesheets for styling.
app = dash.Dash(__name__, external_stylesheets=[dbc.themes.BOOTSTRAP])
server = app.server # Standard way to expose the Flask server for deployment.

# Define the Dash VTK View component for displaying the 3D cone.
vtk_view = dash_vtk.View(
    id="geometry-view", # ID for the view, used in callbacks for camera reset
    children=[
        dash_vtk.GeometryRepresentation( # Represents a geometric actor in the 3D scene
            [
                dash_vtk.Algorithm( # Use dash_vtk.Algorithm to create VTK objects like vtkConeSource
                    id="vtk-algorithm", # ID for the algorithm, used in callbacks to update its state
                    vtkClass="vtkConeSource", # Specifies the VTK class to use (vtk.js equivalent)
                    state={"capping": False, "resolution": 60}, # Initial properties of the cone
                )
            ]
        )
    ],
)

# Define the control card using Dash Bootstrap Components.
# This card contains a slider for cone resolution and a switch for capping.
controls = dbc.Card(
    [
        dbc.CardHeader("Controls"), # Header for the control card
        dbc.CardBody(
            [
                html.P("Resolution:"), # Label for the resolution slider
                dcc.Slider( # Slider for controlling the cone's resolution
                    id="slider-resolution",
                    min=10, max=100, step=1, value=60, # Slider range and initial value
                    marks={10: "10", 100: "100"}, # Marks on the slider
                    className="mb-3" # Margin bottom for spacing
                ),
                html.Br(), # Line break for spacing
                dbc.Checklist( # Checklist (rendered as a switch) for capping
                    options=[{"label": "Capping", "value": "capping"}],
                    value=[], # Initial value (unchecked)
                    id="capping-checklist", # ID for the checklist
                    switch=True, # Render as a switch
                ),
            ]
        ),
    ]
)

# Define the main layout of the Dash application using Dash Bootstrap Components.
app.layout = dbc.Container(
    fluid=True, # Container takes full width
    style={"marginTop": "15px", "height": "calc(100vh - 30px)", "display": "flex", "flexDirection": "column"}, # Layout styling
    children=[
        dbc.Row( # Row for arranging controls and the VTK view
            [
                dbc.Col(width=4, children=controls, className="mb-3 mb-md-0"), # Column for controls, responsive margin
                dbc.Col( # Column for the VTK view
                    width=8,
                    children=[
                        html.Div(
                            vtk_view, # Embed the VTK view component
                            style={"height": "100%", "width": "100%", "minHeight": "400px"}, # Styling for the VTK container
                        )
                    ],
                ),
            ],
            style={"flexGrow": 1}, # Allow row to grow and take available height
            className="g-0" # Remove gutter between columns
        ),
    ],
)

# --- Dash Callbacks ---

@app.callback(
    [Output("vtk-algorithm", "state"), Output("geometry-view", "triggerResetCamera")], # Outputs: algorithm state and camera reset trigger
    [Input("slider-resolution", "value"), Input("capping-checklist", "value")], # Inputs: slider value and checklist value
)
def update_cone(slider_val, checked_values):
    """
    Callback function to update the cone's properties (resolution and capping)
    based on slider and checklist inputs.
    """
    # Create a new state dictionary for the vtkConeSource algorithm.
    # 'resolution' is set directly from the slider value.
    # 'capping' is true if 'capping' is present in the checked_values list.
    new_state = {"resolution": slider_val, "capping": "capping" in checked_values}

    # Return the new state for the algorithm and a random number to trigger a camera reset.

```



```
# Triggering camera reset ensures the view updates correctly after geometry changes.
return new_state, random.random()

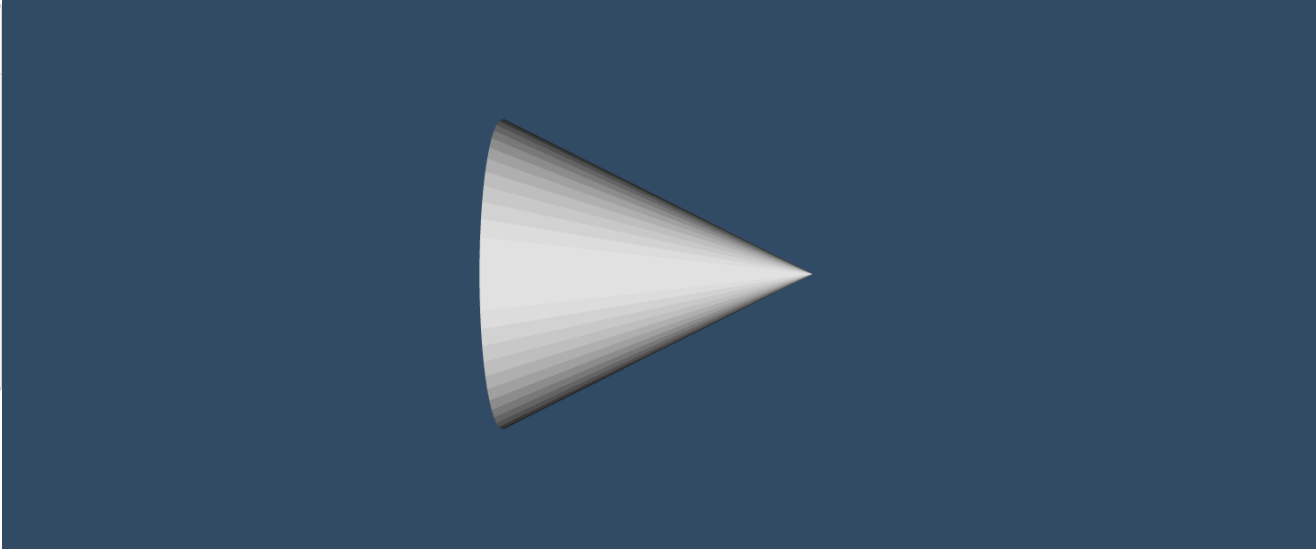
# Run the Dash application server.
# For Jupyter/Colab, 'mode='inline'' embeds the app output directly in the notebook.
# 'debug=True' enables hot-reloading and debug tools (set to False for production).
if __name__ == "__main__":
    app.run(debug=False, mode='inline', port=8053) # Corrected to app.run() for newer Dash versions
```

Controls

Resolution:

10100

☐ Capping



Slide 17

```
In [10]: import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns # For loading a dataset and potentially for cleaner defaults
%matplotlib inline

# Simulate data for User Acquisition Channels
np.random.seed(20)
months_ua = pd.date_range(start='2023-01-01', periods=18, freq='ME')
channels = ['Organic Search', 'Paid Ads', 'Social Media', 'Email Marketing', 'Referral', 'Direct']

# Generate user data with some general trends and one "problematic" channel
ua_data = {}
for i, channel in enumerate(channels):
    base_users = 500 + i * 50 + np.random.normal(0, 50, len(months_ua))
    # Introduce a dip in 'Social Media' performance
    if channel == 'Social Media':
        base_users[8:12] -= 200 # Significant drop
        base_users[12:] += 150 # Recovery

    # Introduce a sharp spike in 'Paid Ads'
    if channel == 'Paid Ads':
        base_users[10] += 500 # A sudden, unusual spike
        base_users[11] += 300

    ua_data[channel] = np.maximum(0, base_users).astype(int) # Ensure no negative users

df_ua = pd.DataFrame(ua_data, index=months_ua)
df_ua.index.name = 'Month'
df_ua_long = df_ua.reset_index().melt(id_vars='Month', var_name='Channel', value_name='Users')

print("Sample User Acquisition Data:")
print(df_ua_long.head())

# --- "Bad" Chart (Matplotlib default, overlapping lines) ---
plt.figure(figsize=(14, 8))

for channel in channels:
    plt.plot(df_ua.index, df_ua[channel], label=channel, linewidth=1.5, alpha=0.8)

plt.title('Monthly User Acquisition Across All Channels', fontsize=18, fontweight='bold')
plt.xlabel('Month', fontsize=14)
plt.ylabel('Number of Users', fontsize=14)
```

```
plt.xticks(rotation=45, ha='right', fontsize=10)
plt.yticks(fontsize=10)
plt.legend(loc='upper left', bbox_to_anchor=(1, 1), ncol=1, fontsize=10)
plt.grid(True, linestyle='-', alpha=0.3)
plt.tight_layout()
plt.show()

# --- "Good" Chart (Seaborn & Matplotlib for Focused Storytelling) ---
plt.figure(figsize=(12, 7))

# Set a professional Seaborn style
sns.set_style("whitegrid")
sns.set_palette("muted") # A more subdued palette for background channels

# Highlight specific channels and de-emphasize others
highlight_channel_1 = 'Social Media' # Channel with a dip and recovery
highlight_channel_2 = 'Paid Ads'     # Channel with a spike

for channel in channels:
    if channel == highlight_channel_1:
        sns.lineplot(x=df_ua.index, y=df_ua[channel], label=highlight_channel_1,
                      color='darkorange', linewidth=3, marker='o', markersize=7, zorder=3, ax=plt.gca())
    elif channel == highlight_channel_2:
        sns.lineplot(x=df_ua.index, y=df_ua[channel], label=highlight_channel_2,
                      color='darkviolet', linewidth=3, marker='x', markersize=7, linestyle='--', zorder=3, ax=plt.gca())
    else:
        sns.lineplot(x=df_ua.index, y=df_ua[channel], color='gray', linestyle=':', linewidth=1, alpha=0.3, zorder=1, ax=plt.gca())

# Add annotations for key insights
plt.annotate('Social Media dip & recovery',
             xy=(df_ua.index[10], df_ua.loc[df_ua.index[10], highlight_channel_1]),
             xytext=(df_ua.index[12], df_ua.loc[df_ua.index[10], highlight_channel_1] - 150),
             arrowprops=dict(facecolor='darkorange', shrink=0.05, width=1, headwidth=6),
             fontsize=10, color='darkorange', fontweight='bold',
             bbox=dict(boxstyle="round,pad=0.3", fc="white", ec="darkorange", lw=0.8, alpha=0.9))

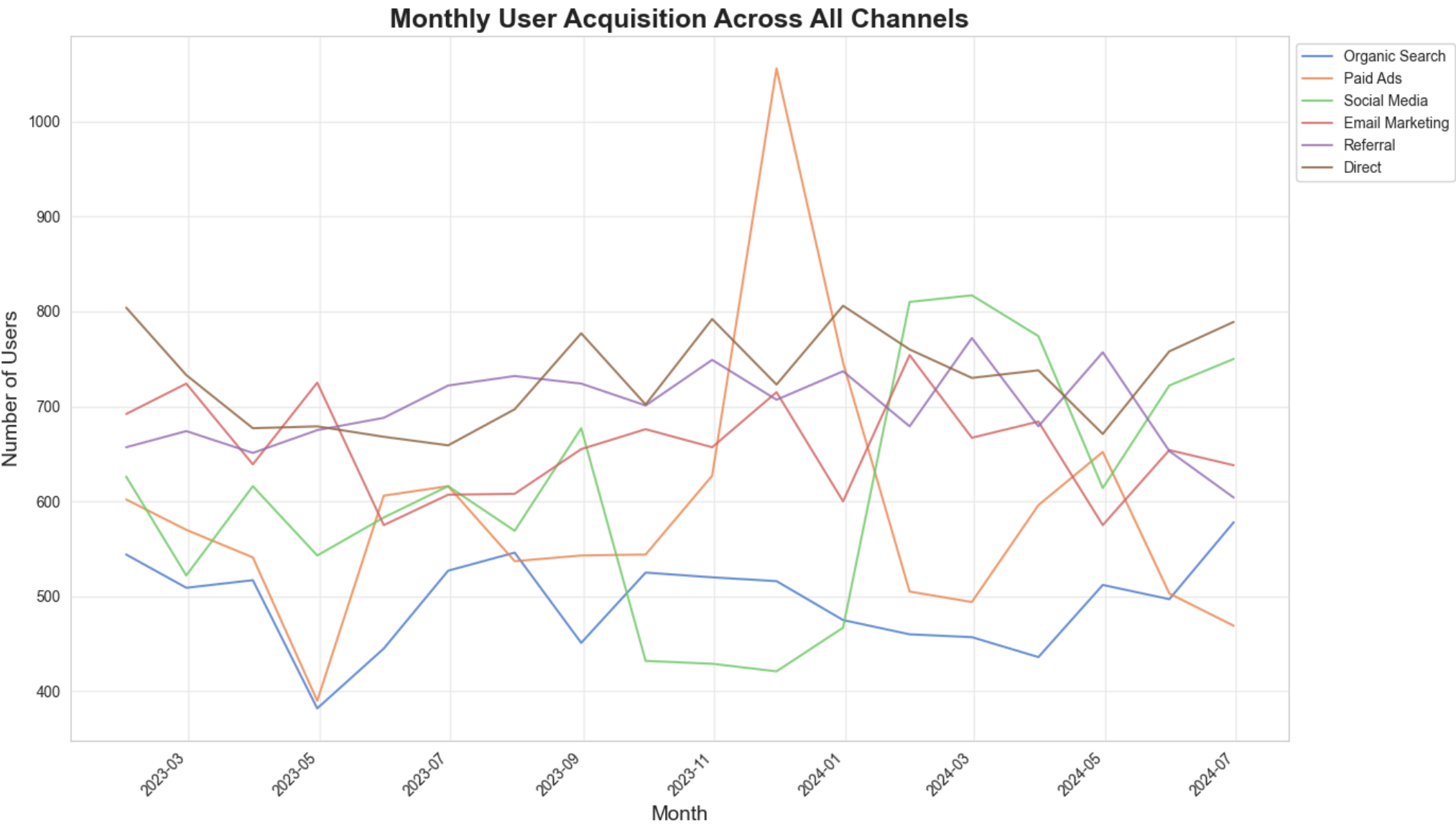
plt.annotate('Unusual spike in Paid Ads!',
             xy=(df_ua.index[10], df_ua.loc[df_ua.index[10], highlight_channel_2]),
             xytext=(df_ua.index[7], df_ua.loc[df_ua.index[10], highlight_channel_2] + 200),
             arrowprops=dict(facecolor='darkviolet', shrink=0.05, width=1, headwidth=6),
             fontsize=10, color='darkviolet', fontweight='bold',
             bbox=dict(boxstyle="round,pad=0.3", fc="white", ec="darkviolet", lw=0.8, alpha=0.9))

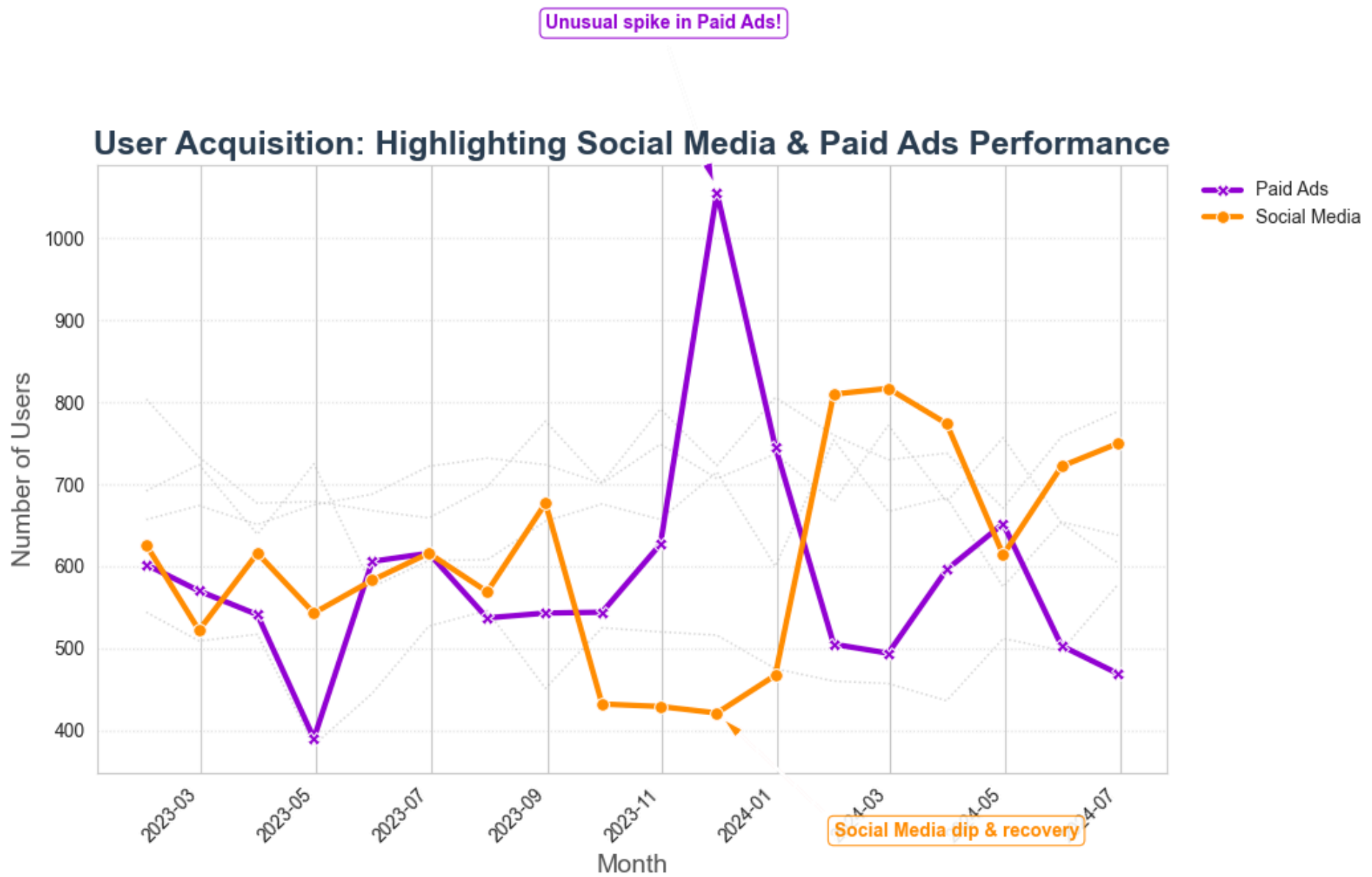
plt.title('User Acquisition: Highlighting Social Media & Paid Ads Performance', fontsize=18, fontweight='bold', color='#2c3e50')
plt.xlabel('Month', fontsize=14, color='#555')
plt.ylabel('Number of Users', fontsize=14, color='#555')
plt.xticks(rotation=45, ha='right', fontsize=10)
plt.yticks(fontsize=10)
plt.legend(loc='upper left', bbox_to_anchor=(1.02, 1), ncol=1, fontsize=10, frameon=False)
plt.grid(axis='y', linestyle=':', alpha=0.5)

plt.tight_layout(rect=[0, 0, 0.88, 1])
plt.show()
```

Sample User Acquisition Data:

	Month	Channel	Users
0	2023-01-31	Organic Search	544
1	2023-02-28	Organic Search	509
2	2023-03-31	Organic Search	517
3	2023-04-30	Organic Search	382
4	2023-05-31	Organic Search	445





Let's Recap 🎓 👁

Data Storytelling: The Art of Impact 📊

Creating beautiful plots is just the **first step**. The **true power of data visualization** lies in **transforming complex analyses into compelling stories that drive understanding and action**. This section highlights key principles to make your visualizations **impactful**.

```
In [11]: import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns # For loading a dataset and potentially for cleaner defaults
%matplotlib inline

In [12]: # Simulate data for demonstration: Monthly sales for several products
np.random.seed(42)
months = pd.date_range(start='2023-01-01', periods=12, freq='ME')
products = [f'Product {i+1}' for i in range(7)] # 7 products

# Generate sales data with some trends and noise
sales_data = {}
for i, product in enumerate(products):
    base_sales = 100 + i * 10 # Different base for each product
    trend = np.linspace(0, 50, 12) # General upward trend
    noise = np.random.normal(0, 15, 12) # Random fluctuations
    seasonal = 10 * np.sin(np.linspace(0, 2 * np.pi, 12)) # Seasonal pattern
    sales_data[product] = base_sales + trend + noise + seasonal
    # Make one product (e.g., Product 4) consistently higher/lower
    if product == 'Product 4':
        sales_data[product] += 50 # Make it a standout
    elif product == 'Product 2':
        sales_data[product] -= 30 # Make it a low performer

df_sales = pd.DataFrame(sales_data, index=months)
df_sales.index.name = 'Month'
df_sales_long = df_sales.reset_index().melt(id_vars='Month', var_name='Product', value_name='Sales')

print("Sample Sales Data:")
print(df_sales_long.head())
```

Sample Sales Data:

	Month	Product	Sales
0	2023-01-31	Product 1	107.450712
1	2023-02-28	Product 1	107.877898
2	2023-03-31	Product 1	127.902557
3	2023-04-30	Product 1	146.380026
4	2023-05-31	Product 1	122.227013

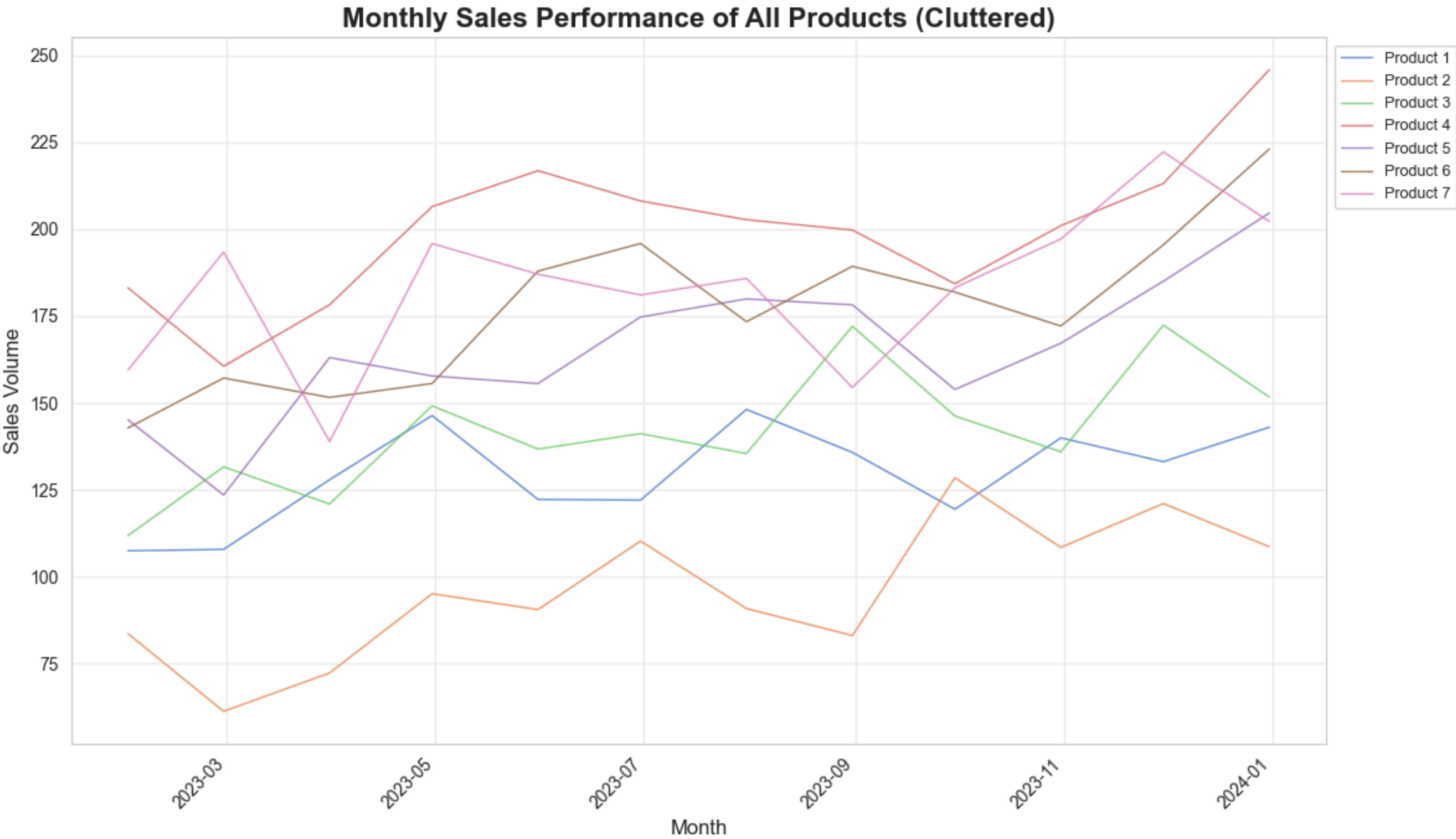
1. The "Bad" Chart: Clutter and Confusion 🤔

This chart is designed to demonstrate **common pitfalls** in data visualization. It's **cluttered**, uses default settings that don't highlight insights, and makes it **hard** to understand the main message.

```
In [13]: # --- "Bad" Chart ---
plt.figure(figsize=(12, 7))

# Plot all products with default Matplotlib colors
for product in products:
    plt.plot(df_sales.index, df_sales[product], label=product, linewidth=1, alpha=0.8)

plt.title('Monthly Sales Performance of All Products (Cluttered)', fontsize=16, fontweight='bold')
plt.xlabel('Month', fontsize=12)
plt.ylabel('Sales Volume', fontsize=12)
plt.xticks(rotation=45, ha='right')
plt.legend(loc='upper left', bbox_to_anchor=(1, 1), ncol=1, fontsize=9) # Legend outside, but still busy
plt.grid(True, linestyle='--', alpha=0.3) # Heavy grid lines
plt.tight_layout()
plt.show()
```



Issues with the "Bad" Chart:

Too many lines: Hard to track individual product performance.

Generic colors: No clear distinction or emphasis.

Busy legend: Takes up space and doesn't immediately convey importance.

Heavy grid: Adds visual noise.

Lack of clear message: What's the main takeaway?

2. The "Good" Chart: Clarity and Impact 👍

Now, let's apply data storytelling principles to the same data. We'll focus on a **clear message**, **strategic use of color**, and **minimizing chartjunk** to make the insights immediately apparent.

```
In [14]: # --- "Good" Chart ---
plt.figure(figsize=(12, 7)) # Slightly larger figure for more detail

# Set a clean Seaborn style for better aesthetics
sns.set_style("whitegrid")
sns.set_palette("viridis") # A visually appealing palette

# Highlight key products and subtly show others
highlight_product_1 = 'Product 4'
highlight_product_2 = 'Product 2' # Second product to highlight (e.g., worst performer)

for product in products:
    if product == highlight_product_1:
        # Highlight best performing product
        sns.lineplot(x=df_sales.index, y=df_sales[product], label=highlight_product_1,
                    color='darkgreen', linewidth=3.5, marker='o', markersize=8, zorder=3, ax=plt.gca())
```

```
elif product == highlight_product_2:
    # Highlight worst performing product
    sns.lineplot(x=df_sales.index, y=df_sales[product], label=highlight_product_2,
                 color='darkred', linewidth=2.5, marker='X', markersize=7, linestyle='--', zorder=2, ax=plt.gca())
else:
    # Subtly show other products as context
    sns.lineplot(x=df_sales.index, y=df_sales[product], color='gray', linestyle=':', linewidth=1, alpha=0.4, zorder=1, ax=plt.gca())

# Add a shaded region to highlight a specific period (e.g., a marketing campaign)
# Let's say a campaign ran from April to July (index 3 to 6)
plt.axvspan(df_sales.index[3], df_sales.index[6], color='blue', alpha=0.08, label='Q2 Campaign')

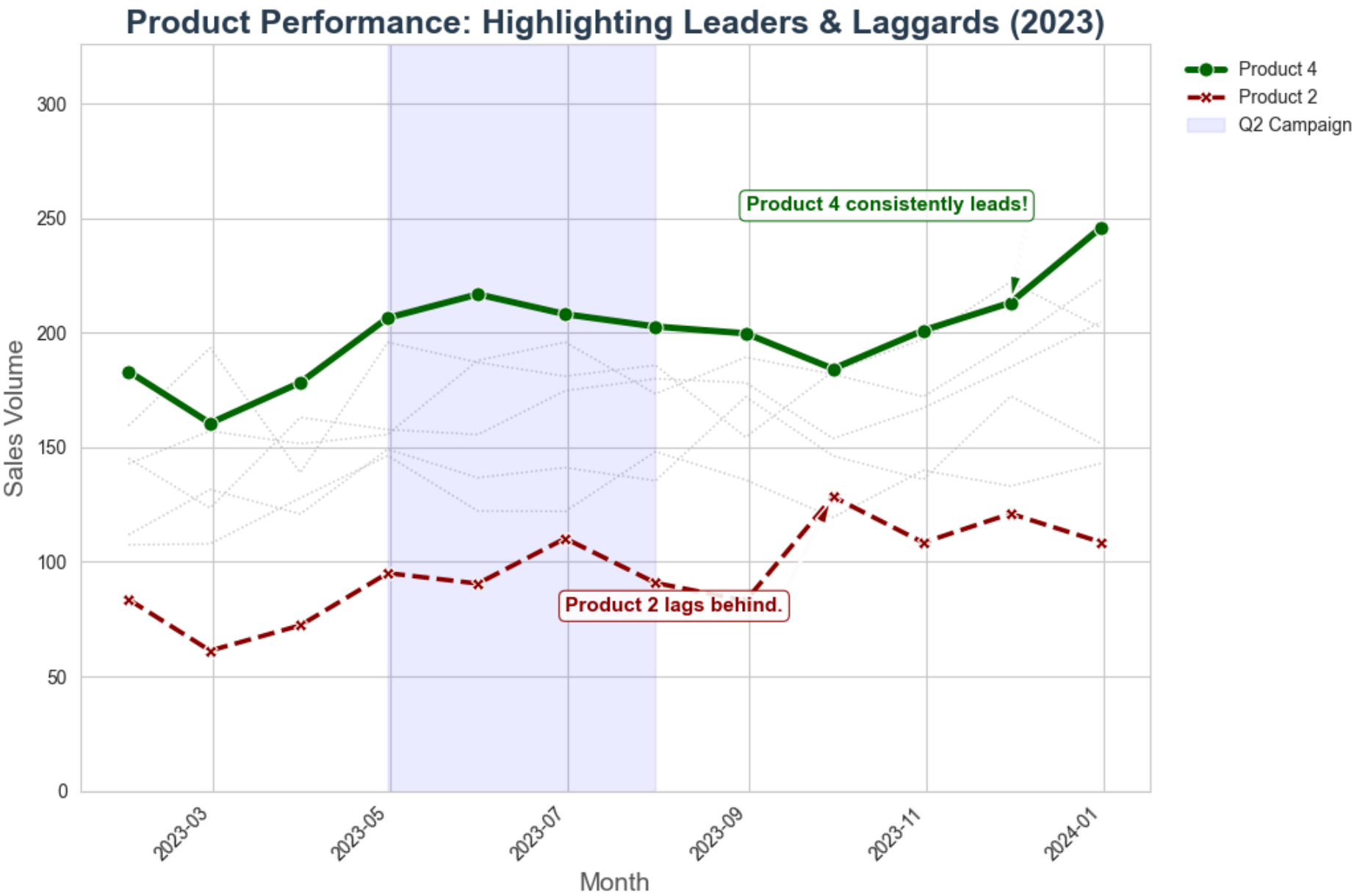
plt.title(f'Product Performance: Highlighting Leaders & Laggards (2023)', fontsize=18, fontweight='bold', color='#2c3e50')
plt.xlabel('Month', fontsize=14, color='#555')
plt.ylabel('Sales Volume', fontsize=14, color='#555')
plt.xticks(rotation=45, ha='right', fontsize=10)
plt.yticks(fontsize=10)
plt.ylim(0, df_sales.max().max() + 80) # Adjust y-limit for better spacing

# Custom Legend to order and emphasize highlighted products
handles, labels = plt.gca().get_legend_handles_labels()
# Reorder Legend to put highlighted products first
order = [labels.index(highlight_product_1), labels.index(highlight_product_2)] + \
        [i for i, label in enumerate(labels) if label not in [highlight_product_1, highlight_product_2, 'Q2 Campaign']] + \
        [labels.index('Q2 Campaign')]
plt.legend([handles[idx] for idx in order], [labels[idx] for idx in order],
          loc='upper left', bbox_to_anchor=(1.02, 1), ncol=1, fontsize=10, frameon=False) # Clean Legend, no frame

# Add annotations for key insights
plt.annotate(f'{highlight_product_1} consistently leads!',
            xy=(df_sales.index[10], df_sales[highlight_product_1].iloc[10]),
            xytext=(df_sales.index[7], df_sales[highlight_product_1].iloc[10] + 40),
            arrowprops=dict(facecolor='darkgreen', shrink=0.05, width=1, headwidth=6),
            fontsize=11, color='darkgreen', fontweight='bold',
            bbox=dict(boxstyle="round,pad=0.3", fc="white", ec="darkgreen", lw=0.8, alpha=0.9))

plt.annotate(f'{highlight_product_2} lags behind.',
            xy=(df_sales.index[8], df_sales[highlight_product_2].iloc[8]),
            xytext=(df_sales.index[5], df_sales[highlight_product_2].iloc[8] - 50),
            arrowprops=dict(facecolor='darkred', shrink=0.05, width=1, headwidth=6),
            fontsize=11, color='darkred', fontweight='bold',
            bbox=dict(boxstyle="round,pad=0.3", fc="white", ec="darkred", lw=0.8, alpha=0.9))

plt.tight_layout(rect=[0, 0, 0.88, 1]) # Adjust Layout to make space for Legend and annotations
plt.show()
```



Key Takeaways from the "Good" Chart:

Clear Message: The title immediately tells the story.

Strategic Color: "Product 4" stands out, while other products provide context without clutter.

Minimal Chartjunk: Clean grid, no unnecessary elements.

Narrative Flow: The annotation guides the viewer to a key insight.

This transformation demonstrates how **intentional design choices** can elevate your data visualizations from **mere plots to powerful storytelling tools, driving actionable insights and influencing decisions.**

Slide 18

Your Journey Continues: Resources for Growth

Congratulations on completing this workshop! You've gained foundational knowledge and practical skills in data visualization with Python. This is just the beginning of your exciting journey into the world of data. To truly master data visualization and become a data-driven professional, continuous learning and hands-on practice are key.

1. Official Documentation: Your Primary Source of Truth

The official documentation for these libraries is the** most authoritative and up-to-date resource.** They provide **comprehensive** guides, API references, and **numerous** examples.

Matplotlib

Website: <https://matplotlib.org/>

Gallery: <https://matplotlib.org/gallery/index.html> (A great source of inspiration and code examples)

Seaborn

Website: <https://seaborn.pydata.org/>

Gallery: <https://seaborn.pydata.org/examples/index.html> (Excellent for statistical plots)

Plotly

Website: <https://plotly.com/python/>

Plotly Express: <https://plotly.com/python/plotly-express/>

Graph Objects: <https://plotly.com/python/graph-objects/>

Dash

Website: <https://dash.plotly.com/>

Dash App Gallery: <https://dash.plotly.com/gallery> (See what's possible!)

2. Online Platforms: Practice, Learn, and Compete

These platforms offer datasets, notebooks, and communities to enhance your practical skills.

Kaggle

Website: <https://www.kaggle.com/>

Why: Access to diverse datasets, participate in competitions, explore thousands of community notebooks, and learn from others' approaches to data analysis and visualization.

Hugging Face Hub (Spaces & Datasets)

Website: <https://huggingface.co/>

Why: A central hub for machine learning models, datasets, and **Gradio/Streamlit Spaces** where you can deploy and interact with ML demos (including interactive visualizations). Great for seeing models in action and sharing your own.

Google Colaboratory (Colab) / Jupyter Notebooks

Website: <https://colab.research.google.com/>

Why: Free cloud-based Jupyter notebooks with GPU access. Perfect for experimenting with code, running tutorials, and sharing your data visualization projects without local setup.

Towards Data Science / Medium

Website: <https://towardsdatascience.com/> (and Medium.com in general)

Why: A vast collection of articles, tutorials, and case studies written by data professionals. Excellent for learning new techniques, understanding practical applications, and staying updated with industry trends.

DrivenData / Zindi

DrivenData: <https://www.drivendata.org/>

Zindi: <https://zindi.africa/>

Why: Platforms focused on data science competitions with a social impact. Great for applying your skills to real-world problems that matter, often involving complex datasets and unique visualization challenges.

3. Book Recommendations: Deepen Your Understanding

Dive deeper into the art and science of data visualization with these highly recommended books.

"Storytelling with Data" by Cole Nussbaumer Knaflic

Focus: Essential for effective communication through data. Teaches you how to craft compelling narratives with your visualizations, focusing on clarity, simplicity, and impact.

Key Takeaway: It's not just about making a chart; it's about telling a story that drives action.

"Python for Data Analysis" by Wes McKinney

Focus: A foundational text for data manipulation and analysis in Python. Written by the creator of the `pandas` library.

Key Takeaway: Master data cleaning, transformation, and preparation, which are crucial steps before visualization.

"Grokking Algorithms" by Aditya Bhargava (Manning Publications)

Focus: An incredibly intuitive and visual guide to common algorithms and data structures. While not strictly about data visualization, understanding algorithms is fundamental to what you visualize.

Key Takeaway: Makes complex algorithms easy to grasp with engaging illustrations and practical examples. Excellent for building foundational computer science knowledge.

"Python for Data Science Handbook" by Jake VanderPlas (O'Reilly)

Focus: A comprehensive reference for essential tools in data science, including NumPy, Pandas, Matplotlib, and Scikit-Learn.

Key Takeaway: Provides a deep dive into the core libraries for data manipulation, scientific computing, and visualization, offering practical examples and best practices. A must-have for any Python data scientist.

" Fundamentals of Data Visualization" by Claus O. Wilke (O'Reilly)

Focus: Explores the principles of good data visualization from a theoretical and practical perspective, covering design choices, common pitfalls, and effective presentation.

Key Takeaway: Teaches you how to create clear, accurate, and compelling visualizations by understanding the underlying principles of visual perception and effective design.

" Data Visualization with Category Theory and Geometry" by Lukas Silvester Barth et al.

Focus: A highly academic and advanced book that explores the deep mathematical principles behind data visualization. It delves into a critical analysis and refinement of algorithms like UMAP.

Key Takeaway: Provides a systematic mathematical foundation for data representation. It's a great resource for researchers and those who want to understand the theoretical underpinnings of visualization and improve upon existing algorithms.

4. Personal Projects: Learn by Doing

The best way to solidify your learning is by applying your skills to real-world problems you care about.

Start with data you're passionate about! Whether it's sports statistics, movie ratings, climate data, or social trends, find data that genuinely interests you.

Build small projects: Don't aim for perfection initially. Focus on completing small, functional projects.

Share your work: Put your projects on GitHub, write about them, and seek feedback. This is invaluable for growth.

Contribute to open source: Look for opportunities to contribute to existing data visualization libraries or projects.

Your journey to becoming a data visualization expert is an exciting one. Keep exploring, keep building, and keep telling impactful stories with data! Good luck    